Square-Root Circuit Bounds for Iterated Functions

Samuel Schlesinger

October 17, 2026

Preliminary draft

Abstract

Let $F : \{0,1\}^n \to \{0,1\}^n$ have a Boolean circuit of size s, and put $A = s + n^3$ and $B = A + \sqrt{n^3 2^n A}$. Every fixed iterate F^T has a bounded-fan-in circuit of size $O(B)$, independently of the magnitude of $T \geq 0$, without assuming injectivity. The construction uses sparse checkpoints and sorting-based lookup, with every access and arithmetic gate charged. The circuit-size bound is nonuniform; constructing its hardwired data can require work exponential in the state width.

For even n, an explicit linear-size permutation has almost all powers of complexity $\Omega(2^{n/2}/n)$ under a uniform exponent modulo its order. This matches the upper bound's exponential scale, but identifies no explicit hard power; hard exponents necessarily have exponentially many binary digits. The upper-bound construction also gives $O(B)$-gate cycle labels, transient depth, pair-input reachability, exact first hitting times, and complete visit descriptors, together with input-time and shared batched versions. Exact target-distance summaries have a local correctness equation, yielding compact safety invariants and termination ranks with coNP verification. Under SETH, general deterministic uniform analysis cannot save a fixed fraction of the state-width exponent, even when the requested powered map has a tiny circuit.

Structured transitions admit stronger bounds. A permutation differing from a counter at r states has $O(n(r+1))$-gate signed fixed powers and navigation, and $O(n(r+m+1))$-gate iteration for an m-bit input time. Its orbit chart can be constructed in polynomial time from an explicit edit list. An autonomous q-bit controller with a p-bit affine payload gives $O(2^q(q+p^2))$-gate fixed powers. Lean checks these core circuit bounds; further extensions state their individual formalization boundaries. The checkpoint and block methods have established antecedents, and historical priority of these circuit formulations remains unresolved.

Keywords. Boolean circuit complexity; function iteration; functional graphs; nonuniform computation; sorting networks; permutation inversion; one-way functions.

1 Introduction

A simple transition rule can describe a computation of enormous duration. How complicated can its final-state map be as a Boolean function of the initial state? We study this question for a fixed map $F : \{0,1\}^n \to \{0,1\}^n$. The computational resource is the total number of Boolean gates, rather than the number of evaluations of F, the number of memory probes, or the time of a random-access algorithm.

Let $N = 2^n$ and suppose that F has a circuit of size s. Directly unrolling T applications gives a circuit of size at most Ts for F^T. Alternatively, hardwiring a complete truth table and testing the input against all N states gives an $O(nN)$-gate circuit, independently of T. The latter bound

ignores the possibility that F itself is much simpler than an arbitrary map on its state space. Our main result exploits that possibility.

Theorem 1.1 (Fixed-power tradeoff)**.** *Let $n \geq 1$, let $F : \{0,1\}^n \to \{0,1\}^n$ satisfy $\mathrm{C}(F) \leq s$, and put $N = 2^n$ and $A = s + n^3$. For every integer $T \geq 0$ and every integer $1 \leq L \leq N$,*

$$\mathrm{C}(F^T) = O\Big(Ls + n^3\left(L + \frac{N}{L}\right)\Big). \tag{1.1}$$

In particular,

$$\mathrm{C}(F^T) = O\Big(A + \sqrt{n^3 NA}\Big). \tag{1.2}$$

The implicit constants are absolute. The circuit may depend on F and T, but the size bound does not depend on the bit length of T.

Here and throughout, circuits use the full fan-in-two Boolean basis and have unrestricted fan-out. The results also hold over the basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$ with a constant-factor change. Circuit size refers to the whole multi-output circuit, not to a separate size bound for each output bit.

Corollary 1.2. *If $\mathrm{C}(F_n) = n^{O(1)}$, then*

$$\sup_{T \geq 0} \mathrm{C}(F_n^T) \leq 2^{n/2 + O(\log n)}. \tag{1.3}$$

The constant hidden in $O(\log n)$ may depend on the polynomial bound for $\mathrm{C}(F_n)$, but not on T.

For each fixed F and T, the circuit works for every initial state.

1.1 Proof overview

The functional graph of F consists of directed cycles with directed trees feeding into them. For a parameter L, we select $O(N/L)$ checkpoints so that every trajectory either encounters one within $2L$ steps or repeats a state within that many steps. A repeated state reveals a period and permits direct recovery of the answer from the initial trajectory.

Set $K = 2L$ and assume $T \geq K$. At each checkpoint h, store $z_h = F^{T-K}(h)$. If the initial trajectory encounters h at time $d \leq K$, then

$$F^{K-d}(z_h) = F^{K-d}\big(F^{T-K}(F^d(x))\big) = F^T(x). \tag{1.4}$$

Thus only a short *forward* correction is needed after the lookup. There is no inverse operation, so the argument applies to noninjective maps.

A separate lookup for each of $\Theta(L)$ visited states against $\Theta(N/L)$ checkpoints would cost $\Theta(N)$ comparisons. Instead, we generate the full initial trajectory before looking anything up and resolve all its queries together. Sorting the queries and dictionary records reduces this cost to $\mathrm{poly}(n)(L + N/L)$ Boolean gates. Combining the initial and final short simulations with this batched lookup gives Equation (1.1).

A worked example. Consider eight states, encoded by three bits, with transitions

$$0 \to 1 \to 2 \to 3 \to 4 \to 5 \to 2, \qquad 6 \to 7 \to 6.$$

Take $L = 3$, $K = 6$, and $H = \{2, 5\}$, the endpoints of the disjoint three-vertex paths $(0, 1, 2)$ and $(3, 4, 5)$. Fix $T = 101$. Starting at $x = 1$, the first checkpoint is $h = 2$, reached at $d = 1$. Its stored payload is $z_2 = F^{95}(2) = 5$, so the forward correction gives

$$F^{K-d}(z_2) = F^5(5) = 2 = F^{101}(1).$$

Starting instead at $x = 6$, the prefix $6, 7, 6$ repeats without meeting a checkpoint. The repeated indices $i = 0$, $j = 2$ give period $p = 2$. The answer is prefix entry $i + ((101 - i) \bmod p) = 1$, namely 7. Thus the two-cycle needs no dictionary entry.

1.2 Further results and sharpness

The exponent $1/2$ is sharp for arbitrary fixed powers. In Section 5, an explicit linear-size permutation has almost all powers of complexity $\Omega(2^{n/2}/n)$, when the exponent is uniform modulo its order. The hard powers are selected by counting, require exponentially long binary exponents, and are not explicitly identified. The same permutation has a polynomial-size iterator for polynomial-width input times.

Write $B = A + \sqrt{n^3 NA}$. The extensions in Sections 6.1 and 6.2 handle input exponents and shared evaluation. The graph-query circuits in Section 7 compute cycle labels, transient depth, reachability, first hitting times, visit descriptors, and synchronous coalescence in $O(B)$ gates. Pair queries take two n-bit states; their exponential scale remains $2^{n/2}$ for polynomial-size transitions. Recognizable targets include eventual membership and recurring or permanent acceptance. For a predicate of size a, put $A_P = s + a + n^3$ and $B_P = A_P + \sqrt{n^3 N A_P}$.

Section 8 gives stronger bounds for autonomous controllers and sparse counter edits. For the latter, an explicit edit list permits polynomial-time chart construction; transport to other coordinates charges their encoding and decoding circuits. Choosing $T = \mathrm{ord}(P) - 1$ also recovers the earlier square-root permutation-inversion bound, compatible with polynomial-size one-wayness; see Section 3.6.

The further extensions separate several questions that the size bound does not settle. Sections 5.2, 5.4 and 5.7 refine counting, approximation, and the complexity of specifying hard exponents. Sections 6.4, 6.5 and 9 study smaller polynomial factors, depth, width, and reversible implementations; the distinguished-point procedure has checked semantics, while its resource bounds remain prose. Section 4 studies optimal checkpoint sets. Sections 10.1 and 10.3 connect compact summaries to certification and the cost of construction, and Section 11 states its oracle-model restrictions and probabilistic hypotheses explicitly.

1.3 Related work and status of the formulation

Preprocessing trajectories to exchange storage for later computation is classical. Hellman's cryptanalytic time–memory tradeoff is an early example of this approach [3]. More directly, Munro, Raman, Raman, and Rao [15, Section 3.1] give a shortcut representation for permutation inversion: with a parameter L, their black-box construction uses $O(L)$ evaluations and $O((N/L)(\log N + \log L))$ auxiliary bits. They also give succinct representations of general functions supporting positive powers with a word-size exponent [15, Section 5]. Those results concern representation space, RAM time, and black-box access, rather than total Boolean gate count.

A closer circuit antecedent is Hirahara, Ilango, and Williams [16, Section 3.2]: they obtain square-root-size permutation-inversion circuits using checkpoints and batched lookup. Their Section 3.2 treats a single-cycle permutation explicitly and notes in one sentence that the construction extends to all permutations through the cycle decomposition. Their lookup theorem [16, Theorem 2.2]

Query	Input parameters	Circuit size
General transitions on n-bit states		
Fixed iterate	x; fixed $T \geq 0$	$O(B)$
Input-time iterate	x, t; $t \in \{0,1\}^m$	$O(\min\{mB, nB + nm\})$
Graph navigation	x or x, y	$O(B)$
Recognizable targets	x; fixed P of size a	$O(B_P)$
Shared fixed powers	k states; common fixed T	$O(kA + \sqrt{kn^3NA})$
Autonomous q-bit controller and p-bit affine payload, $q + p = n$		
Fixed iterate	u, v; fixed $T \geq 0$	$O(2^q(q + p^2))$
Counter permutations with r edited successors		
Signed fixed iterate; labels and navigation	x or x, y; fixed $T \in \mathbb{Z}$	$O(n(r + 1))$
Input-time iterate	x, t; $t \in \{0,1\}^m$	$O(n(r + m + 1))$

Table 1: Circuit bounds for fixed transitions, with $m \geq 1$ and $1 \leq k \leq N$. An m-bit input time multiplies the affine-controller bound by m. Visit descriptors specify all visits; additional deadline-processing circuits are not included. All listed bounds are formalized; Section 12 records the checked boundary.

generalizes a lemma of Paul [6, Lemma 2] on evaluating a Boolean function on many inputs by sorting; that lemma is an ancestor of the sorting-based batched lookups used there and in Lemma 2.1. This draft generalizes that permutation-inversion construction to arbitrary fixed forward powers of possibly noninjective maps. It does not subsume their general-function inversion and compression theorems.

For general functions, Fiat and Naor [4] extend Hellman's method to rigorous time–space tradeoffs for inverting arbitrary functions, in the random-access model; Hirahara, Ilango, and Williams implement their algorithm with circuits [16, Section 3.3]. Lower bounds in this preprocessing model face a known barrier. Corrigan-Gibbs and Kogan [5] show that improving Yao's bound $ST = \widetilde{\Omega}(N)$ for inverting a random function with S bits of advice and T queries would give new lower bounds for depth-two circuits with arbitrary gates, and that strong lower bounds for nonadaptive inverters would give lower bounds for linear-size logarithmic-depth circuits. These barriers concern inversion, not forward powers, but they show that even black-box lower bounds in the preprocessing model can be as hard as circuit lower bounds. The lower bound of Section 5 avoids such questions by counting over exponentially long exponents (Section 5.3). Section 3.7 compares the construction with the treatment of preprocessing in the literature on verifiable delay functions and time-lock puzzles.

Eppstein [17] studies uniform iteration of polynomial-time bijections, with the iteration count supplied in binary, and characterizes the resulting class of functional problems as $\mathsf{FP}^{\mathsf{PSPACE}}$. Our resource question differs: the map is fixed, the circuits are nonuniform, and the size bound remains exponential in the state length. Our result supplies no polynomial-time algorithm for that iteration problem. Section 7.5 gives a nonuniform counterpart for short exponents: hard powers at exponents with polynomially many bits exist exactly when $\mathsf{PSPACE} \not\subseteq \mathsf{P/poly}$.

The present construction uses the same checkpoint and batched-lookup principles. Batcher's sorting networks [1] implement the lookup; the asymptotically smaller networks of Ajtai, Komlós, and Szemerédi [2] save a factor of n (Section 6.4), while the shifted payload in Equation (1.4) and the checkpoint-or-repetition lemma handle general powers and functional graphs. No RAM memory probe is treated as a unit-cost circuit operation. Checkpointing, sorting-based lookup,

and square-root-size permutation inversion are not claimed as new. Historical priority for the full fixed-power theorem and the shared bounds in this draft remains unresolved.

2 Model and circuit primitives

2.1 Conventions

All logarithms are to base two. The notation F^t denotes t-fold composition, with F^0 equal to the identity. The quantity $\mathrm{C}(G)$ is the minimum number of internal gates in an acyclic Boolean circuit computing all output bits of G. Each internal gate computes a Boolean function of at most two bits. Inputs, wires, fan-out, and output designations are not charged as gates. The two constant bits are available; charging a constant number of gates for them would not change any bound.

The circuit for an iterate is allowed to depend on the entire function F and on its fixed exponent. Preprocessing time and the time required to find the circuit are not bounded. Hardwired data do not provide free addressing: every comparison, multiplexer, and routing operation used to access them is realized with Boolean gates and included in our estimates.

All state words have n bits. Because $L \le 2^n$, indices up to $2L$ fit in $n+2$ bits. In the batched construction $r \le 2^n$, so query identifiers and record positions also have $O(n)$ bits. We retain explicit polynomial factors in n for a concrete implementation; these factors are not claimed to be optimal.

2.2 Arithmetic and selection

Comparison, addition, subtraction, and conditional selection of w-bit words have $O(w)$-gate circuits. Binary long division gives an $O(w^2)$-gate circuit for the remainder of a w-bit integer modulo a positive w-bit integer: process the dividend bits from most to least significant, and at each stage conditionally subtract the modulus from the current shifted remainder. There are $O(w)$ stages and $O(w)$ gates per stage.

A circuit can select one of q words of width w using $O(qw)$ gates by a binary tree of word multiplexers, padding to a power of two when necessary. It can also select one flagged record from an ordered list of q records in $O(qw)$ gates. For example, a left-to-right scan maintains a bit indicating whether a flagged record has already been found and retains the first one. This scan is simply an acyclic chain of gates; no depth bound is needed here.

Lemma 2.1 (Batched fixed-dictionary lookup)**.** *Suppose a fixed dictionary consists of M distinct w-bit keys, each with a w-bit payload. Given $Q \ge 1$ query keys, a Boolean circuit can output a match bit and the corresponding payload for every query, in the original query order, using*

$$O\Big((M+Q)(w+\log(M+Q+2))\log^2(M+Q+2)\Big) \tag{2.1}$$

gates. The payload of an unmatched query can be set to zero. In particular, if $w = O(n)$ and $\log(M+Q+2) = O(n)$, the cost is $O(n^3(M+Q))$.

Proof. Let $R = M+Q$. Form one record for each dictionary entry and one for each query. A record contains a key, a dictionary/query tag, a payload field, and an original-position field. The record width is $O(w+\log(R+2))$.

Sort the records lexicographically by key, then by the tag with dictionary records first, and then by original position. A sorting network with $O(R\log^2(R+2))$ comparators exists [1]. A comparator and the resulting conditional swap of the entire records cost $O(w+\log(R+2))$ Boolean gates. Padding uses a separate dummy-record bit so that no actual key is reserved as a sentinel.

In the sorted list, a dictionary record precedes every query with its key. Scan the list, resetting the match bit when the key changes and propagating the payload from a dictionary record through the equal-key run that follows it. If an equal-key run has no dictionary record, every query in that run receives match bit zero. Dictionary keys are distinct, so no run contains two competing payloads. This scan uses $O(R(w + \log(R + 2)))$ gates.

Finally, sort records by a tag that places queries first and then by the query's original position. The first Q records give the requested outputs in order. A second sorting network has the same asymptotic cost as the first. Every operation described is a fixed acyclic circuit. □

Lemma 2.2 (Detecting a repeated state)**.** *Given Q words of w bits, a circuit can return a flag indicating whether two words are equal and, when the flag is one, indices $i < j$ of two equal words. Its size is*

$$O\Big(Q(w + \log(Q + 2)) \log^2(Q + 2)\Big).$$

In particular, for $Q \le 2^{n+O(1)}$ and $w = O(n)$, the size is $O(n^3 Q)$.

Proof. Attach each word's original index and sort by word and then by index. A duplicate exists exactly when two adjacent sorted records have the same word. Compare adjacent words and select one flagged pair. Sorting by index within an equal-word run guarantees that the returned indices satisfy $i < j$. The remaining comparisons and selection use at most $O(Q(w + \log(Q + 2)))$ gates. □

2.3 Sparse checkpoints in functional graphs

The functional graph of a map F has one vertex for each state and the edge $v \to F(v)$ at every vertex. Each weakly connected component has a unique directed cycle, with directed trees feeding into it. For a noncycle vertex, its *depth* is the number of steps needed to reach the cycle.

Lemma 2.3 (Checkpoint or repetition)**.** *Let F be a function on a set of N states, and let $1 \le L \le N$. There is a set H of at most $\lfloor N/L \rfloor$ states such that, for every x, either two of $x, F(x), \dots, F^{L-1}(x)$ are equal or $F^d(x) \in H$ for some $d < 2L$. More generally, the same conclusion holds for starting states in any finite forward-invariant set S, with $|H|L \le |S|$ and $H \subseteq S$. The ambient state type need not be finite.*

Proof. Choose a maximal family of pairwise vertex-disjoint simple directed paths, each consisting of L vertices in S. Put the last vertex of each path into H. Disjointness gives $|H|L \le |S|$. If the first L states of an orbit are distinct, they form such a path in S. By maximality this path meets a chosen path: say $F^i(x) = F^j(a)$, where $i, j < L$ and the chosen path starts at a. Its marked endpoint is reached from x after $i + (L - 1 - j) < 2L$ steps. If the prefix is not simple, it already contains a repetition. Taking S to be the entire state space proves the first claim. The Lean proof implements the same packing argument by induction on $|S|$, removing one simple path at a time. □

Remark 2.4. It is essential that the lemma allows a short repetition instead of a checkpoint. A function can have N fixed points, in which case a set meeting every eventual cycle must contain all N states. Such cycles need no stored shortcut: their behavior is already visible locally. Section 4 determines how small such sets can be and how hard a minimum one is to find.

3 Circuits for fixed powers

We now prove Theorem 1.1. The construction is uniform as a description of a circuit from the chosen constants, but no efficient procedure for computing those constants is asserted.

3.1 Short trajectories and shifted payloads

Fix L and let $K = 2L$. If $T < K$, use T copies of a circuit for F; this costs at most $2Ls$ gates. Hence assume $T \geq K$. Choose H as in Lemma 2.3. For every $h \in H$, hardwire the n-bit payload

$$z_h = F^{T-K}(h). \tag{3.1}$$

The dictionary is fixed independently of the input x and has at most N/L entries.

On input x, first compute

$$x_0 = x, \quad x_1 = F(x), \quad \ldots, \quad x_K = F^K(x). \tag{3.2}$$

This requires Ks gates. Apply Lemma 2.1 to all $K+1$ states in Equation (3.2), using the dictionary $h \mapsto z_h$. Select one matching record, if any, and retain its original trajectory index d. This gives a match flag and a pair (d, z_h) with $x_d = h$ whenever the flag is one.

For the matched case, compute a second short trajectory

$$z_h, F(z_h), \ldots, F^K(z_h)$$

and select its entry at position $K - d$. Its value is

$$\begin{aligned} F^{K-d}(z_h) &= F^{K-d}\big(F^{T-K}(h)\big) \\ &= F^{T-d}(h) \\ &= F^{T-d}\big(F^d(x)\big) = F^T(x). \end{aligned} \tag{3.3}$$

All exponents used here are nonnegative. This second trajectory costs another Ks gates; selection costs $O(nK)$ gates.

3.2 The repeated-state case and very large exponents

Use Lemma 2.2 on Equation (3.2). If no checkpoint was found, Lemma 2.3 guarantees indices

$$0 \leq i < j \leq K, \qquad x_i = x_j.$$

Put $p = j - i$. Determinism implies $x_{u+p} = x_u$ for all $u \geq i$. The number p need not be the least period. Since $T \geq K \geq i$,

$$F^T(x) = x_{\,i+((T-i) \bmod p)}. \tag{3.4}$$

The selected index is at most $i + p - 1 = j - 1$, so the required state already occurs in the initial trajectory.

To implement Equation (3.4) without paying for the bit length of T, hardwire the table

$$R_T(p) = T \bmod p, \qquad 1 \leq p \leq K. \tag{3.5}$$

There are K entries, each of $O(n)$ bits. A multiplexer tree retrieves $R_T(p)$ in $O(nK)$ gates. Compute $i \bmod p$ by binary long division and then compute

$$(T - i) \bmod p = \big(R_T(p) - (i \bmod p)\big) \bmod p$$

using a comparison, subtraction, and a conditional addition of p. These operations take $O(n^2)$ gates. A further $O(nK)$ gates select the state prescribed in Equation (3.4).

A circuit evaluates both branches, so all intermediate operations must also be defined when their output will be discarded. If a repeated pair is absent, supply the dummy values $i = 0$ and $p = 1$ to the arithmetic branch. If a checkpoint is absent, supply a dummy state and $d = 0$ to the second trajectory. The final output selects the checkpoint branch when its match flag is one, and otherwise selects the repetition branch. The checkpoint lemma ensures that the selected branch is always valid. In particular, no division by zero occurs on any input.

3.3 Size bound

The two simulations together use at most $2Ks = 4Ls$ gates. The dictionary and query list contain

$$|H| + (K+1) = O\Big(\frac{N}{L} + L\Big)$$

records of $O(n)$ bits, and the logarithm of this number is $O(n)$. Thus Lemma 2.1 costs $O(n^3(N/L+L))$ gates. Repeated-state detection costs $O(n^3L)$ gates, and all remaining lookup, arithmetic, and selection operations cost $O(nL + n^2)$ gates. Since $n, L \geq 1$, the latter cost is absorbed by $O(n^3L)$. This proves Equation (1.1).

Write the tradeoff as $O(AL + n^3N/L)$, where $A = s + n^3$, and choose

$$L = \max\left\{1, \left\lceil\sqrt{n^3N/A}\right\rceil\right\}. \tag{3.6}$$

Since $A \geq n^3$, this choice satisfies $L \leq \left\lceil\sqrt{N}\right\rceil \leq N$. If $A \leq n^3N$, then $\sqrt{n^3N/A} \leq L \leq 2\sqrt{n^3N/A}$. Hence

$$AL = O(\sqrt{n^3NA}), \qquad n^3N/L = O(\sqrt{n^3NA}).$$

If $A > n^3N$, the choice is $L = 1$ and the cost is $O(A)$. This establishes Equation (1.2) and completes the proof of Theorem 1.1. □

Remark 3.1 (Where the exponent is stored). For $T \geq K$, the information about T used by the circuit is confined to the checkpoint payloads in Equation (3.1) and the residue table in Equation (3.5). Their total length is $O(n(N/L + L))$ bits. The construction does not place the binary expansion of T on a long hidden input tape, and it does not perform uncharged arithmetic on an arbitrarily large integer. Computing the hardwired information is a preprocessing task outside the asserted circuit bound.

Remark 3.2 (Succinct checkpoint sets). A checkpoint set with a small membership circuit, such as the zeros of a hash function, does not reduce the size in general. It shortens the description of H, but the dependence on T sits in the payloads. For the permutation P_n of Section 5, a circuit for P_n^T determines T mod a for every $1 \leq a \leq B$ by Theorem 5.1, hence T mod D_B, which takes at least $B - \log(B+1)$ bits by Lemma 5.2. Theorem 5.4 bounds every circuit for some power, however its checkpoints are chosen. Only compressible payloads could help, and for P_n they are structured, being residues of T, yet some power still needs $\Omega(2^{n/2}/n)$ gates.

3.4 Charging only long trajectories

Let $\delta(v)$ be the distance from v to its eventual cycle, and let $\ell(v)$ be that cycle's length. For $1 \leq L \leq N$, put

$$Q_L = |\{v : \delta(v) \geq L\}| + |\{v : \delta(v) = 0,\ \ell(v) \geq L\}|.$$

The two sets are disjoint. Their union contains the deep transient vertices and all vertices of cycles with length at least L.

Theorem 3.3 (Long-trajectory tradeoff)**.** *For every fixed $T \geq 0$ and $1 \leq L \leq N$,*

$$\mathrm{C}(F^T) = O\Big(Ls + n^3\left(L + \frac{Q_L}{L}\right)\Big).$$

Proof. Apply the path-packing argument of Lemma 2.3 only to paths whose L vertices all lie in the union above. Mark their endpoints. The resulting set H satisfies $|H|L \le Q_L$. Every L-state prefix inside this union either repeats or meets a packed path, and hence reaches its marked endpoint within $2L$ steps. This packing argument does not require the union to be forward invariant.

If $\delta(x) \ge 2L$, the first L states are all deep, so that argument applies immediately. Otherwise the trajectory reaches its cycle before $2L$ steps. A short cycle then repeats before $3L$ steps. On a long cycle all vertices lie in the union, so the packing argument gives a checkpoint or repetition within another $2L$ steps. Thus every trajectory hits H or repeats within a prefix of $4L$ steps.

Use this prefix in the fixed-power construction, with payload $F^{T-4L}(h)$ when $T \ge 4L$ and direct unrolling otherwise. The dictionary has at most Q_L/L entries and the prefix has $O(L)$ states, giving the stated cost. □

Short transient paths leading to short cycles incur no dictionary cost. The bound includes $Q_L = 0$. Since Q_L itself depends on L, one must retain that dependence when choosing the tradeoff parameter.

3.5 Invariant sets and image-sensitive bounds

Theorem 3.4 (Invariant-set tradeoff)**.** *Let $S \subseteq \{0,1\}^n$ be forward invariant under F, with $|S| = M$. For every fixed $T \ge 0$ and $1 \le L \le N$, there is a circuit agreeing with F^T on S of size*

$$O\Big(Ls + n^3\left(L + \frac{M}{L}\right)\Big).$$

The circuit is total on $\{0,1\}^n$; correctness outside S is not asserted.

Proof. Use the invariant-set version of Lemma 2.3, and otherwise apply the fixed-power construction unchanged. A state still occupies n bits, but the dictionary contains only $O(M/L)$ records. □

Corollary 3.5 (Image sensitivity)**.** *Put $M_d = |\operatorname{im}(F^d)|$. For $T \ge d$ and $1 \le L \le M_d$,*

$$\mathrm{C}(F^T) \le \mathrm{C}(F^d) + O\Big(Ls + n^3\left(L + \frac{M_d}{L}\right)\Big).$$

Consequently this is at most $\mathrm{C}(F^d) + O(A + \sqrt{n^3 M_d A})$.

Proof. The image of F^d is forward invariant, since $F(F^d(x)) = F^d(F(x))$. Compute $F^d(x)$ and apply Theorem 3.4 on its image for $T - d$ more steps. Optimize L as in Equation (3.6), replacing N by M_d. □

3.6 Permutation inversion and one-way functions

Corollary 3.6 (Exact permutation inversion)**.** *Let $P : \{0,1\}^n \to \{0,1\}^n$ be a permutation with $\mathrm{C}(P) \le s$, and put $N = 2^n$ and $A = s + n^3$. For every $1 \le L \le N$,*

$$\mathrm{C}(P^{-1}) = O\Big(Ls + n^3\left(L + \frac{N}{L}\right)\Big).$$

In particular,

$$\mathrm{C}(P^{-1}) = O\Big(A + \sqrt{n^3 N A}\Big).$$

If $s = n^{O(1)}$, an inverter of size $2^{n/2+O(\log n)}$ is correct on every input.

Proof. Let $D = \operatorname{ord}(P)$, the least common multiple of its cycle lengths. Then P^D is the identity, so $P^{-1} = P^{D-1}$. Apply Theorem 1.1. The exponent may have a long binary expansion, but the theorem charges only the resulting tables and their access gates. □

This recovers the permutation-inversion circuit bound already discussed by Hirahara, Ilango, and Williams [16, Section 3.2]. It identifies an earlier special case of the fixed-power theorem.

One-wayness against polynomial-size circuits requires every such inverter to have negligible success on the image of a uniform input [20, Section 7.1.1]. The exponential-size exact inverter above is compatible with that requirement. It does rule out security against all circuits of size $2^{(1/2+\varepsilon)n}$, for any fixed $\varepsilon > 0$ and sufficiently large n; it establishes no security at smaller sizes.

For noninjective maps, a forward power need not find a preimage. For example, the map with values $(0, 0, 1, 0)$ on $\{0, 1, 2, 3\}$ has unique preimage 2 of 1, while the forward orbit of 1 stays in $\{0, 1\}$. Hirahara, Ilango, and Williams give a separate general-function inversion bound of $2^{4n/5}\operatorname{poly}(s + n)$ gates [16, Theorem 3.5]. Encoding exhaustive preimage search as iteration retains both an n-bit target and an n-bit candidate, so the present theorem gives only $2^{n+O(\log n)}$ gates for that construction.

Nor does the counting example in Section 5 establish one-way functions. Its update permutation has a linear-size inverse, obtained by decrementing the active cycle coordinate. The selected hard powers have exponential worst-case forward complexity, so cannot themselves form a polynomial-time one-way function. The counting argument gives no average-case inversion hardness.

3.7 Cryptographic consequences

Iterated round functions serve as delay functions. Iterated hashing, which Boneh, Bonneau, Bünz, and Fisch [7] discuss as the basic sequential function behind verifiable delay functions, the slow hash sloth [8], and the candidate MinRoot [9] all evaluate g^T for a public round function g on an n-bit state. The security property, *sequentiality*, asks that g^T cannot be computed much faster than by T sequential applications of g. Theorem 1.1 bounds the nonuniform complexity of g^T, so it limits what a nonuniform sequentiality claim can assert. This subsection records the resulting limits and checks them against concrete parameters. Lean checks only the exponent reduction for sequential squaring and the arithmetic of the table entries (Section 12); the remaining statements are prose. None of them affects a deployed parameter set that we examined.

Depth. Sequentiality concerns depth, while Theorem 1.1 counts gates. For a round circuit g of size s and depth d, the construction in Theorem 6.9 has size $O(Ls + n^3(L + N/L))$ and depth $4Ld + O(n^2 \log n)$: the two trajectories contribute at most $4Ld$, and parallel prefix operations and sorting bound the remaining depth.

Speed-up and size. Unrolling g^T has depth Td. For $T \le \sqrt{N}$, choose L of order T/σ for a speed-up factor σ with $4 \le \sigma \le T$. The depth becomes about $Td \cdot 4/\sigma$, and when the dictionary term dominates the size is of order

$$\sigma \cdot \frac{n^3 N}{T}.$$

Equivalently, depth times size is of order $n^3 N d$, independently of T. In particular, even a constant-factor speed-up costs about $n^3 N/T$ gates in this construction. For $T \ge \sqrt{N}$ the choice Equation (3.6) gives size $2^{n/2}\operatorname{poly}(n, s)$ and depth at most $4\left\lceil\sqrt{N}\right\rceil d + O(n^2 \log n)$, an unbounded speed-up as T grows. All of this is an upper bound for one construction; we make no claim that smaller fast circuits do not exist.

The same tradeoff appears in cryptanalysis as an average-case statement about random-access machines: with S stored iterates spaced L apart, a random challenge meets one within L steps with probability about SL/N. Theorem 1.1 is the every-input, gate-counted form, with a size independent of T.

A design rule. Say that g^T is sequential at level k against circuits if no circuit of size below 2^k computes g^T correctly on every input with depth at most a constant fraction of Td. The construction above then shows the following. If $T \le \sqrt{N}$ and $Ts < 2^k$, sequentiality at level k requires roughly $n + 3\log_2 n - \log_2 T \ge k$. If sequentiality is required for every number of rounds, then, taking T near $\sqrt{N}$ and beyond, it requires $2^{n/2}\,\mathrm{poly}(n, s) \ge 2^k$, that is,

$$n \ge 2k - O(\log k)$$

for polynomial s. The rule is consistent with existing practice: sloth takes a prime $p \ge 2^{2k}$ at security level k [8, Section 3.3], so its state has about $2k$ bits; MinRoot claims 128-bit security with a 512-bit state [9, Claim 1]; and time–memory–data tradeoffs for state recovery [12] are the source of the rule that a stream cipher's state should have at least twice as many bits as its key. We do not claim the rule as new. We found no derivation of it from the circuit size of iterates. For noninjective round functions, Corollary 3.5 gives more. If $|\operatorname{im}(g^d)| = O(N/d)$ for d near $N^{1/3}$, as is expected heuristically for random-like maps, then every g^T has a circuit of size $N^{1/3}\,\mathrm{poly}(n, s)$, and the corresponding rule for unboundedly many rounds would be $n \ge 3k - O(\log k)$. We do not know whether any concrete hash function has this image profile.

Concrete parameters. Table 2 lists $\log_2$ of the two sizes for some state lengths, with constant factors and polynomial factors other than n^3 omitted. We use $T = 2^{40}$ honest rounds, generously; the MinRoot analysis report names $t \approx 2^{30}$ as the practical range [10, Section 4.2.4]. MinRoot adds the round index to the state in every round [10, Section 4.1.2], so its round function is a fixed map only on a state extended by a counter; this makes n larger than 512 and the sizes larger.

Round function	n	k	$\log_2 \sqrt{N}$	$\log_2(n^3 N/T)$, $T = 2^{40}$
MinRoot over a 256-bit field	≥ 512	128	≥ 256	≥ 499
sloth with $p \approx 2^{256}$	256	128	128	240
SHA-256 chain	256	128	128	240
any 128-bit state	128	128	64	109

Table 2: Sizes, as base-2 logarithms, of the circuits of Theorem 1.1: for g^T at every T (column $\sqrt{N}$), and for a constant-factor speed-up at $T = 2^{40}$ (last column). Constant factors and the factors $\mathrm{poly}(n, s)$ in the $\sqrt{N}$ column are omitted; k is the claimed or intended security level.

For MinRoot, sloth, and an iterated 256-bit hash, the speed-up size exceeds 2^k by more than 2^{100}. The every-T size exceeds 2^k by 2^{128} for MinRoot. For sloth and the hash it equals 2^k up to the omitted polynomial factors, which is the boundary $n \approx 2k$ of the design rule. Moreover, the obvious way to compute the payloads evaluates g on essentially all N states, whereas MinRoot allows the adversary 2^{128} precomputation calls [9]. So no deployed parameter that we checked is affected. The last row only illustrates the design rule: a 128-bit state would admit, in principle, a circuit of about 2^{109} gates that evaluates 2^{40} rounds a constant factor faster than unrolling, below a nominal 128-bit nonuniform level.

Where the cryptographic literature stands. Sequentiality definitions bound the preprocessing. In the definition of Boneh, Bonneau, Bünz, and Fisch [7, Definition 4], a preprocessing algorithm A_0 runs in time polynomial in t and the security parameter, and an online algorithm A_1 has parallel time below t on polynomially many processors. Their iterated sequential functions must be sequential for $2^{o(\lambda)}$ rounds with $\lambda = O(\log |X|)$ [7, Definition 7], far fewer than $\sqrt{|X|}$, so Theorem 1.1 does not contradict these definitions. The MinRoot analysis report [10, Chapter 4] models the cube root as a random permutation and bounds the oracle queries of A_0 and A_1. Its Theorem 2 is a precomputation attack of the table-lookup type: A_0 stores d-round outputs of many starting points, and a challenge in the stored set saves d rounds; its advantage is stated as an average over the challenge. The report leaves a tight lower bound in the bounded-advice model open. Mahmoody, Moran, and Vadhan [11] analyse time-lock puzzles in the random oracle model against query-bounded adversaries without auxiliary input. We found no prior statement of the every-input, gate-counted tradeoff for forward iterates in these works, and no sequentiality bound for an iterated random oracle against adversaries with auxiliary input. Whether the checkpoint tradeoff is optimal in that model is open.

Sequential squaring. For a fixed squarefree modulus M of n bits, the map $x \mapsto x^2 \bmod M$ has polynomial-size circuits, so Theorem 1.1 gives $2^{n/2}\,\mathrm{poly}(n)$ gates for $x \mapsto x^{2^T} \bmod M$. Nonuniformly much more is true. With the Carmichael exponent $\lambda(M)$ hardwired, $x^a \equiv x^b \pmod M$ for all x whenever $a, b \geq 1$ and $a \equiv b \pmod{\lambda(M)}$, so $x^{2^T} \bmod M = x^e \bmod M$ with $e = 1 + ((2^T - 1) \bmod \lambda(M))$, which square-and-multiply computes in polynomial size. The congruence holds for non-units because each prime factor p of M divides x or has $x^{\lambda(M)} \equiv 1 \pmod p$, and the shift by one keeps $e \geq 1$: when $\lambda(M)$ divides 2^T, the exponent $2^T \bmod \lambda(M) = 0$ would give $x^0 = 1$ at $x = 0$. Lean checks this reduction for every T and every residue, with $1 \leq e \leq \lambda(M)$; the square-and-multiply circuit is not mechanized. The sequentiality of the time-lock puzzle of Rivest, Shamir, and Wagner [58] and of squaring-based delay functions is therefore a statement about adversaries that cannot factor M, not about circuits that may depend on M. In the definitions above this is enforced by the polynomial time bound on A_0. For a fixed public round function such as sloth's or MinRoot's, by contrast, the model of Theorem 1.1, in which the circuit may depend on g, is the preprocessing model with an exponential rather than a polynomial budget.

Stream ciphers. The state update of Trivium [13] is a fixed map on 288 bits, and that of Grain [14] on 160 bits; the key enters only through the initial state. For every T, Theorem 1.1 gives circuits of size $2^{144}\,\mathrm{poly}(n)$ and $2^{80}\,\mathrm{poly}(n)$ that jump ahead T steps. This is no attack. Jumping ahead helps only for T beyond what can be run directly, and these sizes are at least 2^k for the 80-bit keys, in accordance with the state-size rule above. The theorem does show that a generator on n bits hides no forward sequentiality beyond $2^{n/2}\,\mathrm{poly}(n)$ nonuniform gates, however many steps are taken.

4 Minimum checkpoint sets

The constructions above only need the existence of a sparse checkpoint set. This section asks how small such a set can be for a given map, and how hard it is to find one. For a map given by a circuit, deciding whether a checkpoint set of a given size exists is coNP-complete; for a map given as a table, a minimum set is computable in polynomial time. The proof rests on a min–max duality for directed paths in in-forests. That duality, Lemma 4.3, is checked in Lean as `ForestDuality.forest_duality`, for an arbitrary vertex set of a finite in-forest; the component

formula, the complexity results and the table algorithm are not mechanized. None of the results in this section is used elsewhere in the paper.

Definition 4.1. Let F be a map on a finite set V of N states, and let $K \geq 1$. A *K-trajectory* is a list $x, F(x), \ldots, F^{K-1}(x)$ of K pairwise distinct states. A set $H \subseteq V$ is a *K-checkpoint set* if it meets every K-trajectory, and $\mathrm{opt}_K(F)$ is the least size of a K-checkpoint set.

If $x, \ldots, F^{2L-1}(x)$ are distinct, then so are $x, \ldots, F^{L-1}(x)$, and the set H of Lemma 2.3 contains some $F^d(x)$ with $d < 2L$. So that set is a $2L$-checkpoint set and $\mathrm{opt}_{2L}(F) \leq \lfloor N/L \rfloor$. This is tight up to a factor of two: on a single cycle of length $N \geq K$, each state lies in exactly K of the N K-trajectories, so $\mathrm{opt}_K \geq N/K$. For other maps opt_K can be far smaller; it is 0 for a map with only fixed points.

Lemma 4.2 (Long trajectories)**.** *If $\mathrm{opt}_K(F) \leq m$, then every list $x, F(x), \ldots, F^{R-1}(x)$ of R pairwise distinct states has $R < (m+1)K$. In particular every cycle of F has length less than $(m+1)K$.*

Proof. The list contains $\lfloor R/K \rfloor$ disjoint consecutive blocks of K states, each a K-trajectory, and each needs its own point of a K-checkpoint set. So $\lfloor R/K \rfloor \leq m$. □

4.1 Duality in in-forests

An *in-forest* is a finite set V with a partial successor map $\sigma : V \to V \cup \{\perp\}$ that has no cycles. A *path* is a list $v_1, \ldots, v_j$ with $\sigma(v_i) = v_{i+1}$ for $i < j$; it *ends* at v_j and has length j. A vertex $u \neq v$ is an *ancestor* of v if $\sigma^i(u) = v$ for some $i \geq 1$. Fix a requirement $r : V \to \{1, 2, \ldots\}$. A path is *required* if its length is at least r of its end. Let τ_r be the least size of a set meeting every required path, and ν_r the largest number of pairwise vertex-disjoint required paths. Write τ_K, ν_K when $r \equiv K$.

Lemma 4.3 (Duality)**.** *In every in-forest, $\tau_r = \nu_r$. A minimum cover is obtained as follows. Process the vertices in an order in which every vertex follows all of its ancestors, maintaining a set G. At v, let*

$$d(v) = 1 + \max\{d(u) : \sigma(u) = v,\ u \notin G\},$$

with the maximum of the empty set equal to 0, and add v to G if $d(v) \geq r(v)$.

Proof. By induction in the processing order, $d(v)$ is the largest length of a path ending at v whose vertices other than v lie outside G; the status of those vertices is final when v is processed, because they are ancestors of v.

G is a cover. Let Q be a required path ending at v and avoiding G. Then $d(v) \geq |Q| \geq r(v)$, so $v \in G$, a contradiction.

For each $v \in G$ choose a *witness* W_v: the last $r(v)$ vertices of a path of length $d(v)$ realizing the definition. It is a required path ending at v, and $W_v \cap G = \{v\}$, since its other vertices are ancestors of v and lie outside G. Witnesses are pairwise disjoint. Suppose $u \in W_v \cap W_w$ with $v \neq w$. Both v and w lie on the unique forward path from u, say v strictly before w. Then v lies on W_w between u and w, and $v \in G$, contradicting $W_w \cap G = \{w\}$.

Hence $|G| \leq \nu_r$. Disjoint required paths need distinct points of a cover, so $\nu_r \leq \tau_r \leq |G|$. □

4.2 The component formula

A K-trajectory follows F, so it lies in one weakly connected component of the functional graph. Hence $\mathrm{opt}_K(F)$ is the sum of opt_K over the components. Fix a component Q with cycle Z of length

ℓ. For a cycle vertex c, let $Q - c$ be the in-forest on $Q \setminus \{c\}$ with $\sigma(v) = F(v)$ unless $F(v) = c$, in which case $\sigma(v) = \perp$. Let $Q - Z$ be the in-forest on $Q \setminus Z$ with $\sigma(v) = F(v)$ unless $F(v) \in Z$, in which case $\sigma(v) = \perp$, and give it the requirement $r_Z(v) = K - \ell$ if $F(v) \in Z$ and $r_Z(v) = K$ otherwise.

Lemma 4.4 (Component formula)**.** *Let* $\mathrm{opt}(Q)$ *be the least size of a set meeting every* K*-trajectory in* Q*.*

1. *If* $\ell \geq K$*, let* W *consist of any* K *consecutive vertices of* Z*. Then* $\mathrm{opt}(Q) = \min_{c \in W}\big(1 + \nu_K(Q - c)\big)$*.*
2. *If* $\ell < K$*, then* $\mathrm{opt}(Q) = \min\big(\nu_{r_Z}(Q - Z),\ \min_{c \in Z}(1 + \nu_K(Q - c))\big)$*.*

Proof. First fix a cycle vertex c. Because $Q - c$ has no cycle, its paths of length K are lists of K distinct consecutive states avoiding c, that is, the K-trajectories of Q avoiding c; conversely every such trajectory is a path of $Q - c$. So the least size of a set containing c and meeting every K-trajectory of Q is $1 + \tau_K(Q - c)$, which equals $1 + \nu_K(Q - c)$ by Lemma 4.3.

(1) The K vertices of W are distinct consecutive states, so W is a K-trajectory and every feasible set contains some $c \in W$. The formula is the minimum of the previous paragraph over $c \in W$.

(2) A feasible set either contains a cycle vertex, which is covered by the first paragraph, or avoids Z. Let x have depth a, so that $x, \ldots, F^{a-1}(x)$ are tree vertices and $F^a(x) \in Z$. The first $a + \ell$ states of the orbit are distinct and the next one repeats, so x starts a K-trajectory exactly when $a + \ell \geq K$. A set avoiding Z meets this trajectory exactly when it meets its tree part $x, \ldots, F^{\min(a,K)-1}(x)$.

These tree parts are required paths of $Q - Z$ under r_Z. If $a \geq K$ the part has K vertices. If $K - \ell \leq a < K$, it has $a \geq K - \ell$ vertices and ends at a vertex u with $F(u) \in Z$.

Conversely, every required path of $Q - Z$ contains such a tree part. A required path ending at a vertex v with $F(v) \notin Z$ contains a path of exactly K vertices ending at v, which is the tree part of its first vertex, whose depth is at least K. A required path ending at u with $F(u) \in Z$ contains the path of its last $K - \ell$ vertices, which is the tree part of a state of depth exactly $K - \ell$.

So a set avoiding Z is feasible exactly when it covers the required paths of $Q - Z$, and the least such size is $\tau_{r_Z}(Q - Z) = \nu_{r_Z}(Q - Z)$ by Lemma 4.3. □

4.3 Maps given by circuits

Let CKPT be the problem: given a circuit for $F : \{0,1\}^n \to \{0,1\}^n$ and integers K and m in unary, decide whether $\mathrm{opt}_K(F) \leq m$. The defining formula, that some list H of at most m states meets the first K states of every orbit with no repetition among them, places CKPT in Σ_2^p. Duality brings it down one level.

Theorem 4.5. CKPT *is* coNP*-complete. For every fixed* $m \geq 0$ *it remains* coNP*-complete with* m *fixed and* $K = 2$*. Consequently* CKPT *is not* Σ_2^p*-hard under polynomial-time reductions unless* NP = coNP*, in which case the polynomial hierarchy collapses to* NP *[53].*

Proof. Membership. We give polynomially checkable certificates for $\mathrm{opt}_K(F) > m$. Put $M = (m + 1)K$. A certificate is either

(i) a list $x, F(x), \ldots, F^{M-1}(x)$ of M distinct states, or

(ii) at most $m + 1$ components, each given by a state z, an integer $\ell < M$ and an integer $b \geq 1$, together with packings.

In (ii), with $W = \{z, \ldots, F^{K-1}(z)\}$ when $\ell \geq K$, the verifier checks that $F^\ell(z) = z$ and $F^j(z) \neq z$ for $0 < j < \ell$, so that $Z = \{z, \ldots, F^{\ell-1}(z)\}$ is a cycle of length ℓ. It checks that the cycles of different listed components are disjoint, and that the listed b sum to at least $m+1$. For each option of Lemma 4.4, the certificate lists pairwise disjoint required paths in the corresponding in-forest: $b-1$ of them for an option c, and b of them for the option $Q - Z$. Each path is an explicit list of at most K states. The verifier checks the F-steps, the avoidance of c or of Z, the length condition, and that the path lies in the component: from each vertex, at most $M + \ell$ applications of F reach Z. There are at most K options per component, so everything is polynomial in the size of F, K and m.

Soundness. A certificate of type (i) works by Lemma 4.2. For type (ii), the listed components are distinct, since their cycles are disjoint. In each, a packing of $b-1$ paths gives $1 + \tau_K(Q - c) \geq b$, and a packing of b paths gives $\tau_{r_Z}(Q - Z) \geq b$. Every feasible set falls under one of the options, as in the proof of Lemma 4.4, so $\mathrm{opt}(Q) \geq b$. Summing over components gives $\mathrm{opt}_K(F) \geq m+1$.

Completeness. Suppose $\mathrm{opt}_K(F) > m$ and no certificate of type (i) exists. Then every orbit has fewer than M distinct states, so every cycle has length less than M and every depth is less than M. Choose components with $\mathrm{opt}(Q) \geq 1$ until their optima sum to at least $m+1$; at most $m+1$ are needed. Set $b = \mathrm{opt}(Q)$, lowering the last value so that the sum is exactly $m+1$. Each option of Lemma 4.4 has value at least $\mathrm{opt}(Q) \geq b$, and the maximum packings of Lemma 4.3 supply the paths. Shortening every path to exactly r of its end keeps them disjoint and required, with at most K states each.

Hardness. Reduce from the tautology problem for Boolean circuits φ on n' variables, with $K = 2$. Use states (y, j) with $y \in \{0,1\}^{n'}$ and $j \in \{0,1,2\}$, encoded in $n'+2$ bits; the unused encodings are fixed points. Put $F(y, j) = (y, j)$ if $\varphi(y) = 1$ or $j = 2$, and $F(y, j) = (y, j+1)$ otherwise. If $\varphi(y) = 0$, then $(y, 0), (y, 1)$ is a 2-trajectory. If φ is a tautology, every state is fixed, so there is no 2-trajectory. Hence $\mathrm{opt}_2(F) = 0$ exactly when φ is a tautology. For fixed $m \geq 1$, add a flag bit and, on the flagged states, m disjoint cycles of length 2, all other flagged states being fixed. Each new cycle is a 2-trajectory and one point per cycle meets all of its trajectories, so opt_2 increases by exactly m. The circuit for F has size polynomial in that of φ.

If CKPT were Σ_2^p-hard, then $\Sigma_2^p \subseteq \mathsf{coNP}$, hence $\mathsf{NP} \subseteq \mathsf{coNP}$ and $\mathsf{NP} = \mathsf{coNP}$. □

Corollary 4.6. *For every function $\alpha \geq 1$, computing a number in $[\mathrm{opt}_K(F), \alpha \cdot \mathrm{opt}_K(F)]$ from a circuit for F and K in unary is* coNP*-hard.*

Proof. Such a number is 0 exactly when $\mathrm{opt}_K(F) = 0$, which is the coNP-hard case $m = 0$ of Theorem 4.5. □

Proposition 4.7 (Search)**.** *Given a circuit for F and K, m in unary with $\mathrm{opt}_K(F) \leq m$, the value $\mathrm{opt}_K(F)$ and a minimum K-checkpoint set are computable in polynomial time with an* NP *oracle.*

Proof. The value takes $m+1$ queries to the coNP language of Theorem 4.5. For the set, put $M = (m+1)K$; by Lemma 4.2, every orbit has fewer than M distinct states. Membership of a state in the component of a known cycle is therefore decidable in polynomial time, and each search below is an NP question whose witness is found bit by bit.

Find the components that contain a K-trajectory one at a time: ask for a K-trajectory whose orbit reaches none of the cycles found so far, and iterate it to find its cycle. Each such component has $\mathrm{opt}(Q) \geq 1$, so there are at most m of them, and the other components need no point.

For each such component and each option of Lemma 4.4, compute a minimum cover of the in-forest (Y, σ, r) of that option as follows. Call a vertex *violated* if some required path ending at it avoids the current set G. Repeatedly find a violated vertex, then move to a violated proper ancestor while one exists, and add the final vertex to G. Each move strictly increases the depth, which stays

below M, so each round makes fewer than M moves. Stop when no vertex is violated, or as soon as $|G|$ exceeds m, in which case the option is not optimal. Output the best option over all components.

This rule produces a minimum cover. Each added vertex v has a witness: a required path W_v ending at v that avoided G when v was added. Suppose two witnesses share a vertex u. Their ends lie on the forward path of u; call the earlier one a and the later one b, so a is a proper ancestor of b and $a \in W_b$. If a was added before b, then W_b did not avoid G when b was added. If a was added after b, then W_a avoided G already when b was added, since G only grows; so a was a violated proper ancestor of b, contradicting the choice of b. So the witnesses are disjoint, and $|G| \le \nu_r = \tau_r$. □

4.4 Maps given as tables

Theorem 4.8. *Given F as a table of N entries and $K \ge 1$, a minimum K-checkpoint set can be computed in time*

$$O\Big(N + \sum_Z \ell_Z \min(K, \ell_Z)\Big) \subseteq O(NK)$$

on a word RAM, where Z ranges over the cycles and ℓ_Z is the length of Z. On an in-forest with an arbitrary requirement, time $O(N)$ suffices.

Proof. For an in-forest, run the greedy rule of Lemma 4.3, processing the vertices by decreasing distance to their root, which is computed by one traversal. Ancestors lie strictly farther from the root.

For a general F, find the cycles and the depths of all tree vertices in time $O(N)$. Let T be the in-forest of tree vertices, with $\sigma(v) = F(v)$ unless $F(v)$ is a cycle vertex, and requirement K.

Step 1. Run the greedy rule on T, in decreasing order of depth, obtaining G. Put $\bar{d}(u) = 0$ for $u \in G$ and $\bar{d}(u) = d(u)$ otherwise. So $\bar{d}(u)$ is the largest length of a path of T ending at u that avoids G, and $\bar{d}(u) \le K - 1$.

Step 2. For each cycle $Z = (v_0, \dots, v_{\ell-1})$ and each i, let a_i be the maximum of $\bar{d}(u)$ over tree vertices u with $F(u) = v_i$, or 0 if there is none. If $w_i = K - a_i \le \ell$, record the arc $I_i = \{v_i, v_{i+1}, \dots, v_{i+w_i-1}\}$, indices mod ℓ.

Step 3. On each cycle, compute a minimum set P_Z of cycle vertices meeting all recorded arcs. Some arc I_s of least width must contain a point of every feasible P_Z. For each $p \in I_s$, discard the arcs containing p. Cut the circle at p, which turns the remaining arcs into intervals on a line, and run the classical line greedy: scan intervals by right endpoint, choosing the right endpoint of each interval not yet met. Take the best $1 +$ (greedy size) over $p \in I_s$. With bucket sorting each cut costs $O(\ell)$, and $|I_s| \le \min(K, \ell)$.

Output $G \cup \bigcup_Z P_Z$. We show that this is optimal.

Feasibility. Every K-trajectory contained in T is a required path of T, so it meets G (Lemma 4.3). Any other K-trajectory avoiding G starts at some x, runs through a tail $x, \dots, F^{a-1}(x)$ of $a \le K-1$ tree vertices avoiding G, enters the cycle at v_i, and continues through the $K - a \le \ell$ distinct vertices $v_i, \dots, v_{i+K-a-1}$. If $a \ge 1$ the tail ends at a tree vertex u with $F(u) = v_i$, so $a \le \bar{d}(u) \le a_i$. Hence the arc I_i is recorded, lies inside the trajectory, and is met by P_Z.

Every recorded arc forces a point. The arc I_i is the cycle part of a K-trajectory avoiding G. If $a_i \ge 1$, prefix I_i with a path of a_i tree vertices that ends at a preimage of v_i and avoids G. If $a_i = 0$, the trajectory starting at v_i is I_i itself.

Some minimum set contains G. Let the picks of Step 1 be $g_1, g_2, \dots$ in processing order, and suppose a minimum set S contains $g_1, \dots, g_{j-1}$. The witness W of g_j is a K-trajectory with $W \cap G = \{g_j\}$, so S contains some $u \in W$. Replace u by g_j. A K-trajectory met by u but not by g_j follows W from u and stops before g_j. So it is a path of T of length K ending at an ancestor e of

g_j, and it contains a pick g. Its vertices are ancestors of e or e itself, so g was processed before g_j and lies in S, and $g \neq u$ because $u \notin G$. The new set is therefore still feasible.

Optimality. Let $S \supseteq G$ be minimum. Replace each tree vertex $u \in S \setminus G$ by the cycle vertex at which the orbit of u enters its cycle. A K-trajectory met by u either lies in T, and is met by G, or reaches the cycle, which it enters at that vertex. So the new set is feasible, no larger, and of the form $G \cup P$ with P on the cycles. By the previous paragraph $P \cap Z$ meets every recorded arc of Z. Hence $|S| \geq |G| + \sum_Z |P_Z|$.

The cost is $O(N)$ for Steps 1 and 2 and $O(\ell_Z \min(K, \ell_Z))$ for Step 3 on each cycle. □

For a map given by a small circuit the components can be exponentially large. In Theorem 4.5, duality replaces the existential quantifier over H by packings, which certify lower bounds. The constructions of this paper need no such optimization: Lemma 2.3 is within a factor of two of the worst case, and that factor only changes constants. The undirected relative, the k-path vertex cover problem, is NP-hard in general graphs and linear on trees [54]. The out-degree-one structure is what makes duality and the polynomial table algorithm available here. The script `verify_checkpoint_sets.py` compares Lemmas 4.3 and 4.4 and Theorem 4.8 with brute force on small random maps.

5 A matching exponent for arbitrary fixed powers

This section gives an explicit linear-size permutation whose collection of powers is large enough that some member has circuit complexity at least $2^{n/2}/\operatorname{poly}(n)$. The counting argument is elementary and does not require an asymptotic estimate for the maximal order of a permutation.

5.1 A permutation with cycles of all lengths up to $2^{n/2} - 1$

Let $n = 2k$ and $B = 2^k - 1$. Write a state as a pair of k-bit integers (a, b). Define

$$P_n(a,b) = \begin{cases} (a, b+1 \bmod a), & 1 \leq a \leq B \text{ and } 0 \leq b < a, \\ (a,b), & \text{otherwise.} \end{cases} \tag{5.1}$$

For each $1 \leq a \leq B$, the states $(a,0), (a,1), \ldots, (a,a-1)$ form one cycle of length a; every other state is fixed. Hence P_n is a permutation and

$$\operatorname{ord}(P_n) = D_B := \operatorname{lcm}(1, 2, \ldots, B). \tag{5.2}$$

Despite the notation in Equation (5.1), the update does not require a general division circuit. Compare b with a, increment b, and, when $b < a$, replace b by zero exactly when the increment equals a. Otherwise retain the appropriate increment or original value. Comparison, increment, and selection each have $O(k)$-gate circuits. The first coordinate is passed through unchanged. Thus

$$\mathrm{C}(P_n) = O(n), \tag{5.3}$$

and the family has a direct polynomial-time circuit construction.

Theorem 5.1 (Powers and fixed-integer remainders)**.** *For a fixed $T \geq 0$, define $r_T(a) = T \bmod a$ when $a > 0$ and $r_T(0) = 0$, using k bits for both input and output. Then*

$$\mathrm{C}(r_T) \leq \mathrm{C}(P_n^T) \leq \mathrm{C}(r_T) + O(n).$$

Proof. Restrict the power circuit to inputs $(a, 0)$ and retain its second coordinate. This computes r_T, including the zero-modulus branch. Conversely, on an active state compute $b + r_T(a)$ using $k+1$ bits. Both summands are below a, so a comparison and at most one subtraction of a give the result modulo a. The spare bit retains the carry. Select the original b on inactive states and copy a to the output. All additional operations have linear-size Boolean circuits. The first inequality uses the paper's free constant-wire convention; the native Lean model charges one shared zero gate for the restriction. □

Thus the arithmetic object behind the lower bound is the remainder of a single fixed integer as a function of its input modulus. The theorem does not identify a particular hard integer.

Lemma 5.2. *For every integer $B \geq 1$,*

$$\operatorname{lcm}(1, 2, \ldots, B) \geq \frac{2^B}{B+1}. \tag{5.4}$$

Proof. We show that every binomial coefficient $\binom{B}{j}$ divides D_B. For a prime p, factorial valuations give

$$\nu_p\binom{B}{j} = \sum_{a \geq 1} \left(\left\lfloor \frac{B}{p^a} \right\rfloor - \left\lfloor \frac{j}{p^a} \right\rfloor - \left\lfloor \frac{B-j}{p^a} \right\rfloor \right).$$

Each summand is zero or one, and it is zero when $p^a > B$. Therefore

$$\nu_p\binom{B}{j} \leq \left\lfloor \log_p B \right\rfloor = \nu_p(D_B),$$

where both sides are zero for $p > B$. This proves divisibility. Since $\sum_{j=0}^{B} \binom{B}{j} = 2^B$, some binomial coefficient is at least $2^B/(B+1)$, and D_B is at least that large. □

Lemma 5.3 (Counting multi-output circuits)**.** *The number of functions $\{0,1\}^n \to \{0,1\}^n$ computable with at most q fan-in-two Boolean gates is at most*

$$(q+1)16^q(n+q+2)^{2q+n} = 2^{O((q+n)\log(n+q+2))}. \tag{5.5}$$

Proof. Topologically order the gates. At each gate choose one of at most 16 Boolean operations and two predecessors among the inputs, the two constants, and gates. Allowing all $n+q+2$ possible predecessors at each position only increases the count. Choose the n output wires from these same possibilities. Sum over gate counts from zero through q; each summand is bounded by the expression without its factor $q+1$. Different encodings may compute the same function, which only helps the upper bound. □

Theorem 5.4 (Hard powers of a simple permutation)**.** *For all sufficiently large even n, the permutation in Equation* (5.1) *satisfies*

$$\max_{T \geq 0} \mathrm{C}(P_n^T) = \Omega\left(\frac{2^{n/2}}{n}\right). \tag{5.6}$$

Together with Theorem 1.1, this gives

$$\log_2\left(\max_{T \geq 0} \mathrm{C}(P_n^T)\right) = \frac{n}{2} + O(\log n). \tag{5.7}$$

Proof. There are exactly D_B distinct powers of P_n, with representatives $0 \le T < D_B$. By Lemma 5.2,

$$\log_2 D_B \ge B - \log_2(B+1).$$

Let $q = \lfloor cB/n \rfloor$ for a sufficiently small positive absolute constant c. Since $B = 2^{n/2} - 1$, Lemma 5.3 bounds the logarithm of the number of circuits of size at most q by

$$O((q+n)\log(n+q+2)) = O(cB + n^2).$$

Choose c so that the coefficient of B is less than $1/2$. For all sufficiently large even n, this quantity is smaller than $B - \log_2(B+1)$. Thus there are fewer size-q circuits than distinct powers of P_n. At least one power has complexity greater than q, proving Equation (5.6).

For the upper bound, substitute $\mathrm{C}(P_n) = O(n)$ into Equation (1.2). This gives $\max_T \mathrm{C}(P_n^T) = O(n^3 2^{n/2})$. Combining the upper and lower bounds and taking logarithms proves Equation (5.7). □

Theorem 5.5 (Almost every power is hard)**.** *There is an absolute $c > 0$ such that, for all sufficiently large even n,*

$$\Pr_{T \sim \mathrm{Unif}\{0,\ldots,D_B-1\}} \left[\mathrm{C}(P_n^T) \ge c\frac{2^{n/2}}{n} \right] \ge 1 - 2^{-\Omega(2^{n/2})}.$$

Proof. Set $U = 2^{n/2}$. The preceding counting argument bounds the number of powers with circuits of size at most $\lfloor cU/n \rfloor$ by $2^{\alpha U}$, for some fixed $\alpha < 1$ after decreasing c. On the other hand, $D_B \ge 2^{U-1}/U$. Divide the two counts to bound the exceptional fraction by $U 2^{1-(1-\alpha)U} = 2^{-\Omega(U)}$. □

The probability concerns exponent residues modulo the whole group order. It does not concern short exponent descriptions or average-case hardness over inputs to any one power. In particular, easy powers remain among the exceptional residues.

Corollary 5.6. *For all sufficiently large even n, some power of P_n has an output coordinate of circuit complexity $\Omega(2^{n/2}/n^2)$.*

Proof. For the power supplied by Theorem 5.4, join optimal circuits for all n output coordinates. Their total size is at most n times the largest coordinate complexity and is an upper bound on the complexity of the whole map. □

5.2 The exact constant and average-case behaviour

Theorems 5.4 and 5.5 determine $\max_T \mathrm{C}(P_n^T)$ up to a polynomial factor. This subsection determines its leading constant and describes how well a fixed power can be approximated on most states. The results refine the mechanized statements above but are proved here in prose only; they are not covered by the Lean formalization. Throughout, $n = 2k$, $B = 2^k - 1$, and $\pi(x)$ is the number of primes at most x.

Two classical estimates replace Lemmas 5.2 and 5.3. First, $\ln D_B = \psi(B) = \sum_{p^j \le B} \ln p$ is Chebyshev's function, so the prime number theorem with the de la Vallée Poussin error term [40] gives

$$\log_2 D_B = \frac{B}{\ln 2}\left(1 + O(e^{-c\sqrt{k}})\right) \tag{5.8}$$

for an absolute $c > 0$; similarly $\pi(x) = (1 + O(1/\ln x))\, x/\ln x$ and, by Mertens' second theorem, $\sum_{p \le x} 1/p = \ln\ln x + M + O(1/\ln x)$ for a constant M. Second, the circuit count of Lemma 5.3 can be sharpened by the classical symmetry argument [27, 29].

Lemma 5.7 (Sharp circuit count)**.** *For $q \geq 1$, the number of functions $\{0,1\}^\ell \to \{0,1\}^m$ with circuit complexity at most q is at most*

$$(q+1)\left(\frac{16e(\ell+q+2)^2}{q}\right)^q (\ell+q+2)^m.$$

In particular, for $q \geq \ell$ its base-two logarithm is $q\log_2 q + O(q) + O(m\log_2 q)$.

Proof. Let $\mathrm{C}(f) = s$ and fix a circuit for f with exactly s gates. Its gates compute pairwise distinct functions, none equal to an input variable or a constant. Indeed, if a gate g computes the same function as an input or constant wire, redirect every use of g, including output designations, to that wire and delete g. If two gates g, h compute the same function and h is not an ancestor of g, redirect every use of h to g and delete h; since g does not depend on h, the result is still acyclic. Either operation contradicts minimality.

A *description* with s labelled gates lists, for each label, one of 16 operations and two predecessors among the ℓ inputs, two constants, and s labels, and then lists m output wires from the same $\ell+s+2$ possibilities. There are at most $(16(\ell+s+2)^2)^s(\ell+s+2)^m$ descriptions. Each of the $s!$ ways to label a minimum circuit for f gives a description of a circuit computing f. These descriptions are distinct: the description determines the function computed at each label, and because gate functions are pairwise distinct this determines the labelling. Descriptions of circuits for different functions are different. Hence the number of f with $\mathrm{C}(f) = s$ is at most $(16(\ell+s+2)^2)^s(\ell+s+2)^m/s!$. Use $s! \geq (s/e)^s$, bound $\ell+s+2 \leq \ell+q+2$, and note that $s \mapsto (c/s)^s$ increases on $1 \leq s \leq c/e$, where here $c = 16e(\ell+q+2)^2 > eq$. Summing over $0 \leq s \leq q$ gives the bound. For $q \geq \ell$, $(\ell+q+2)^2/q \leq 16q$, which gives the logarithmic form. □

Lemma 5.3 omits the factor $1/s!$ and therefore charges about $2q\log_2 q$ bits to a size-q circuit instead of $q\log_2 q$, and Lemma 5.2 gives $\log_2 D_B \geq B - O(k)$ instead of $(1+o(1))B/\ln 2$. Each loss costs a constant factor in Theorem 5.4. With both estimates sharpened, the counting bound meets a matching construction.

Theorem 5.8 (Exact constant for powers of P_n)**.** *For every $T \geq 0$,*

$$\mathrm{C}(r_T) \leq \left(1 + O\left(\tfrac{\log k}{k}\right)\right)\frac{2^k}{k\ln 2}.$$

For all but a $2^{-\Omega(2^k/k)}$ fraction of $T \in \{0,\dots,D_B-1\}$,

$$\mathrm{C}(r_T) \geq \left(1 - O\left(\tfrac{1}{k}\right)\right)\frac{2^k}{k\ln 2}.$$

Consequently, by Theorem 5.1,

$$\max_{T\geq 0} \mathrm{C}(P_n^T) = \left(\frac{2}{\ln 2} + O\left(\tfrac{\log n}{n}\right)\right)\frac{2^{n/2}}{n},$$

and the lower bound holds for almost every T in the same sense.

Proof. Lower bound. The maps r_T for $0 \leq T < D_B$ are pairwise distinct, since r_T determines $T \bmod a$ for every $a \leq B$ and hence $T \bmod D_B$. Put $\Lambda = \log_2 D_B$ and $q = \lfloor (1 - C/k)\Lambda/k \rfloor$ for an absolute constant C. By Equation (5.8), $\log_2 q \leq k+1$ for large k, so by Lemma 5.7 with $\ell = m = k$ the number of maps of complexity at most q is at most

$$2^{q(k+1)+c_1 q + c_2 k^2} \leq 2^{\Lambda(1-C/k)(1+(1+c_1)/k)+c_2k^2} \leq 2^{\Lambda - \Lambda/k}$$

once $C \geq c_1 + 3$ and k is large, because Λ/k grows like $2^k/k$. Thus at most a $2^{-\Lambda/k}$ fraction of the D_B residues T have $\mathrm{C}(r_T) \leq q$. Finally $q = (1 - O(1/k))2^k/(k \ln 2)$ by Equation (5.8).

Upper bound. Let $s_0 = \left\lfloor \sqrt{B} \right\rfloor$ and call a prime *large* if it exceeds s_0. Every $1 \leq a \leq B$ factors uniquely as $a = a_s p$, where a_s has no large prime factor and p is either 1 or a large prime: two large prime factors would give $a \geq (s_0 + 1)^2 > B$. Then $\gcd(a_s, p) = 1$. The circuit has four stages.

(i) Smooth part. Start with $(A, r) = (1, 0)$. For each prime $d \leq s_0$ in turn, compute $e = \nu_d(a)$ by testing $d^j \mid a$ for $j \leq \lfloor \log_d B \rfloor$, select the hardwired constants d^e and $T \bmod d^e$ with a multiplexer over at most $k + 1$ entries, and replace (A, r) by (Ad^e, ρ), where $\rho < Ad^e$ satisfies $\rho \equiv r \pmod{A}$ and $\rho \equiv T \pmod{d^e}$. The Chinese remainder step uses an inverse of A modulo d^e, computed by an extended-Euclid circuit. All numbers are below 2^k, so each prime costs $\mathrm{poly}(k)$ gates, and there are $\pi(s_0) \leq 2^{k/2}$ primes. At the end $A = a_s$ and $r = T \bmod a_s$. One division gives $p = a/A$.

(ii) Index of the large prime. Let $L = 2^{\lceil k/2 \rceil}$ and $h = \lfloor p/L \rfloor$, the top $\lfloor k/2 \rfloor$ bits of p. Read $\pi(hL - 1)$ from a hardwired table with $\lfloor k/2 \rfloor$ input bits, using $O(k2^{k/2})$ gates. Add the number of primes in $[hL, p)$ using L primality circuits of size $\mathrm{poly}(k)$ and a running counter. Such circuits exist by the AKS algorithm [37] and the Pippenger–Fischer simulation [38], or nonuniformly by Miller–Rabin with $O(k)$ hardwired bases chosen by Adleman's argument [39]. Subtracting $\pi(s_0)$ yields the index $\iota(p) \in [0, M)$ of p among the large primes, where $M = \pi(B) - \pi(s_0)$. This stage costs $2^{k/2}\,\mathrm{poly}(k)$ gates.

(iii) Residue table. Let $G(i) = T \bmod p_i$, where p_i is the large prime of index i. Write $M = \sum_{t \in S} 2^t$ in binary; this partitions $[0, M)$ into aligned dyadic blocks I_t of length 2^t, one for each $t \in S$, and membership of i in I_t is an equality test on the bits of i above position t. On I_t, each of the k output bits of G is a total function of the low t bits of i. Compute it with Lupanov's bound $\mathrm{C} \leq (1 + \alpha_t)2^t/t$, where $\alpha_t = O(\log t/t)$ [28, 29], and with an $O(2^t/t)$ bound for small t. An AND–OR selection combines the blocks with $O(k^2)$ gates. Let $j = \lfloor \log_2 M \rfloor$ and $t_0 = j - \lceil \log_2 j \rceil$. Blocks with $t \geq t_0$ cost at most $k(1 + \sup_{t \geq t_0} \alpha_t)M/t_0$, and blocks with $t < t_0$ cost $O(k2^{t_0}/t_0) = O(kM/j^2)$. By the prime number theorem, $M = (1 + O(1/k))2^k/(k \ln 2)$, so $j = k - \log_2 k + O(1)$, $t_0 = k - O(\log k)$, and this stage costs $(1 + O(\log k/k))M$ gates.

(iv) Combination. If $a = 0$, output 0. If $p = 1$, output r. Otherwise output the $\rho < a$ with $\rho \equiv r \pmod{a_s}$ and $\rho \equiv G(\iota(p)) \pmod{p}$, again by a $\mathrm{poly}(k)$-size Chinese remainder circuit. On $a = 1$ stage (i) gives $a_s = 1$, $p = 1$ and $r = 0$, as required. Values computed on discarded branches are ignored.

The total is $(1 + O(\log k/k))2^k/(k \ln 2) + 2^{k/2}\,\mathrm{poly}(k)$, which proves the upper bound. Substituting $k = n/2$ and using Theorem 5.1, whose additive $O(n)$ is negligible, gives the statement for P_n^T. □

The primes in $(\sqrt{B}, B]$ carry the leading-order independent information about T. The residues modulo the prime powers $d^e \leq B$ with $d \leq \sqrt{B}$ are further independent data, stored separately in stage (i), but they occupy only $2^{k/2}\,\mathrm{poly}(k)$ bits; every other residue is reconstructed from these two kinds by the Chinese remainder theorem. Storing $T \bmod p$ in k bits instead of $\log_2 p$ bits wastes only a $1 + O(1/k)$ factor, because most large primes lie near B. This is why the two bounds agree in their leading constant $2/\ln 2 \approx 2.885$. The construction improves the general bound of Theorem 1.1, which gives $O(n^3 2^{n/2})$ for this family, by a factor of order n^4.

Approximation on most states. For a map g on $\{0,1\}^n$, let $\mathrm{err}_T(g)$ be the fraction of states $x \in \{0,1\}^n$ with $g(x) \neq P_n^T(x)$. Hardness of P_n^T does not survive this relaxation by much: every power can be approximated by circuits a polynomial factor below the worst case, with error $\Theta(\log n/n)$, and this is sharp for random powers.

Proposition 5.9 (Every power is approximable)**.** *Fix $c > 1$. For every T there is a circuit of size $(1 + o(1))2^k/k^c$ with*

$$\mathrm{err}_T \le (1 + o(1))\frac{(c-1)\log_2 k}{2k}.$$

Proof. Let y be the least integer with $\pi(y) - \pi(s_0) \ge 2^k/k^c$; by the prime number theorem, $\log_2 y = k - (c-1)\log_2 k + O(1)$, and $y > \sqrt{B}$ for large k. Run the circuit of Theorem 5.8, but place in stage (iii) only the large primes at most y. Its cost is $(1 + O(\log k/k))2^k/k^c + 2^{k/2}\,\mathrm{poly}(k)$. Wrap it as in Theorem 5.1, and output the input unchanged whenever $p > y$. The result is correct on every inactive state and on every active state (a, b) whose a has no prime factor in $(y, B]$. Since $y \ge \sqrt{B}$, such an a has exactly one such factor p, and $a = pu$ with $u \le \lfloor B/p \rfloor$. The number of wrong states is therefore at most

$$\sum_{y<p\le B}\ \sum_{u\le\lfloor B/p\rfloor} pu \le \frac{B^2}{2}\sum_{y<p\le B}\frac{1}{p} + \frac{B\pi(B)}{2}.$$

By Mertens' theorem the sum of reciprocals is $\ln(\ln B/\ln y) + O(1/k) = (1 + o(1))(c-1)\log_2 k/k$, and $B\pi(B) = O(4^k/k)$. Dividing by $2^n = 4^k$ gives the claim. □

Theorem 5.10 (Random powers are hard on average)**.** *Fix $c > 1$ and let $q = \left\lfloor 2^k/k^c \right\rfloor$. For T uniform in $\{0, \dots, D_B - 1\}$, with probability $1 - 2^{-\Omega(qk)}$ every map g with $\mathrm{C}(g) \le q$ satisfies*

$$\mathrm{err}_T(g) \ge (1 - o(1))\frac{(c-1)\log_2 k}{2k}.$$

Proof. Put $K = k^{c-1}$, $y = B/K$, and let $\mathcal{P}$ be the set of primes in $(y, B]$. For $p \in \mathcal{P}$ let C_p be the set of active states (a, b) with $p \mid a$, and $W_p = |C_p|$. Since $y \ge \sqrt{B}$ for large k, the sets C_p are disjoint, and $W_p \le B(K+1)/2$ because $\lfloor B/p \rfloor < K$.

Fix g. At an active state (a, b), $P_n^T(a, b) = (a, (b + T) \bmod a)$. If $g(a, b) = (a, v)$ with $v < a$, correctness at (a, b) forces $T \equiv v - b \pmod a$, and in particular fixes $T \bmod p$ for $p \mid a$; otherwise g is wrong at (a, b). So for each $p \in \mathcal{P}$ and each residue ρ modulo p, let $Y_p(\rho)$ be the number of states in C_p whose forced residue modulo p is ρ. The number of correct states in C_p is at most $Y_p(T \bmod p)$, and $\sum_\rho Y_p(\rho) \le W_p$. Call p *good* if $Y_p(T \bmod p) > \varepsilon W_p$. Fewer than $1/\varepsilon$ residues are good, and the residues $T \bmod p$ for $p \in \mathcal{P}$ are independent and uniform, since each such p divides D_B exactly once. Hence the events "p good" are independent, each of probability at most $1/(\varepsilon y) = K/(\varepsilon B)$. If fewer than $2q$ primes are good, then g errs on at least $(1 - \varepsilon)(W(\mathcal{P}) - 2q\max_p W_p)$ states, where $W(\mathcal{P}) = \sum_{p\in\mathcal{P}} W_p$.

Using $|\mathcal{P}| \le \pi(B) \le 2B/k$ for large k, the probability that at least $2q$ primes are good is at most

$$\binom{|\mathcal{P}|}{2q}\Big(\frac{K}{\varepsilon B}\Big)^{2q} \le \Big(\frac{eK}{\varepsilon k q}\Big)^{2q} = 2^{-2q(k - O(\log k) - \log_2(1/\varepsilon))}.$$

By Lemma 5.7 with $\ell = m = n$, at most $2^{q(k - c\log_2 k) + O(q)}$ maps have $\mathrm{C}(g) \le q$. Take $\varepsilon = 1/\log_2 k$. A union bound shows that, with probability $1 - 2^{-\Omega(qk)}$, fewer than $2q$ primes are good for every such g.

On that event every such g errs on at least $(1 - \varepsilon)(W(\mathcal{P}) - qB(K+1))$ states. Since $\lfloor B/p \rfloor \ge B/p - 1$, we have $W_p \ge (B^2/p - B)/2$, so by Mertens' theorem

$$W(\mathcal{P}) \ge \frac{B^2}{2}\Big(\ln\frac{\ln B}{\ln y} - O\big(\tfrac{1}{k}\big)\Big) - \frac{B\pi(B)}{2} = (1 - o(1))\frac{B^2}{2}\cdot\frac{(c-1)\log_2 k}{k}.$$

Moreover $qB(K+1) = O(B^2/k)$, which is of smaller order. Dividing by 4^k completes the proof. □

Together, Proposition 5.9 and Theorem 5.10 show that the optimal error at size $2^k/k^c$ is $(1+o(1))(c-1)\log_2 k/(2k)$ for almost every power: constant average-case hardness, i.e. a constant lower bound on the approximation error, is impossible at size $2^{n/2}/\operatorname{poly}(n)$. The same arguments give constant average-case hardness at somewhat smaller sizes.

Corollary 5.11 (Constant average-case hardness at size $2^{(1-\delta)k}$)**.** *Fix $0 < \delta \le 1/3$ and let $q = 2^{(1-\delta)k}$. Every power has a circuit of size $(1+o(1))q$ with error at most $\frac{1}{2}\ln\frac{1}{1-\delta} + o(1)$, and for almost every T every circuit of size at most q has error at least $\frac{1}{2}\ln\frac{1}{1-\delta} - o(1)$, with probability $1-o(1)$ over uniform $T \in \{0,\dots,D_B-1\}$.*

Proof. For the upper bound, repeat the proof of Proposition 5.9 with y the least integer such that $\pi(y) - \pi(s_0) \ge (1-\delta)q$. Stage (iii) then has $t_0 = (1-\delta)k - O(\log k)$, so its k output bits cost $(1+o(1))k(1-\delta)q/t_0 = (1+o(1))q$ gates; stages (i) and (ii) cost $2^{k/2}\operatorname{poly}(k) = o(q)$ because $\delta < 1/2$. Moreover $\ln y = (1-\delta)k\ln 2 + O(\log k)$ and $y \ge \sqrt{B}$. For the lower bound, repeat the proof of Theorem 5.10 with $\log_2 K = \delta k - \sqrt{k}$ and fixed small ε. The union bound requires $2(\log_2 q - \log_2 K) > (1+o(1))\log_2 q$, that is, $\log_2 K < (1-\delta)k/2 - \omega(\log k)$, which holds because $\delta \le (1-\delta)/2$. The subtracted term qBK is $o(B^2)$ because $\log_2 K < \delta k - \omega(1)$, and $y = B/K \ge \sqrt{B}$. In both bounds the error is $\frac{1}{2}\ln(\ln B/\ln y) + o(1)$ by Mertens' theorem; letting $\varepsilon \to 0$ slowly gives the statement. □

For $\delta > 1/3$ the two arguments no longer meet, and for $\delta > 1/2$ the moduli may have two prime factors above y; we do not pursue these ranges. The hardness of a typical power of P_n is spread thinly over the cycles whose length has a prime factor near B. A permutation whose cycles all have comparable prime lengths would avoid this dilution; we leave such constructions to future work.

5.3 Why the lower bound does not apply to short input times

The lower bound selects T from $\{0,\dots,D_B-1\}$; it places no polynomial bound on the bit length of that exponent. Indeed, Lemma 5.2 shows that $\log D_B \ge 2^{n/2} - O(n)$. The choice of a power therefore has exponentially many available bits. The theorem does not provide a polynomial-time procedure that, given n, identifies a hard power.

For the very same family, the iterator with an m-bit time input has size $O(n(m+1))$. On an active cycle,

$$P_n^t(a,b) = (a, (b + (t \bmod a)) \bmod a). \tag{5.9}$$

An m-stage remainder computation with a k-bit running remainder costs $O(mk)$ gates, and the remaining comparison, addition, and selection cost $O(n)$ gates as in Theorem 5.1. On inactive states the answer is the input itself; use a dummy positive modulus when $a = 0$. Consequently, for $m = \operatorname{poly}(n)$ this iterator has polynomial size.

Corollary 5.12 (Bit length of hard exponents)**.** *Let T_n be exponents satisfying $\mathrm{C}(P_n^{T_n}) = \Omega(2^{n/2}/n)$ for sufficiently large even n. Their binary lengths $m_n = \max\{1, \lceil\log_2(T_n+1)\rceil\}$ satisfy*

$$m_n = \Omega(2^{n/2}/n^2).$$

Proof. Hardwire the m_n-bit exponent into the iterator just described. This gives $\mathrm{C}(P_n^{T_n}) = O(n(m_n+1))$. Comparing with the assumed lower bound proves the claim. □

Thus the hard powers necessarily require exponentially long binary exponents; long descriptions are not merely allowed by the counting proof. Theorem 5.4 establishes sharpness for arbitrary fixed powers. It gives no hardness result for the bounded-bit-time problem.

5.4 Explicit hard exponents

By Corollary 5.12, a hard power of P_n has an exponent with $2^{n/2}/\operatorname{poly}(n)$ binary digits. The natural notion of an explicit hard power therefore allows time polynomial in that length. Call a sequence of exponents $(T_n)_{n \text{ even}}$ *explicit* if a deterministic algorithm outputs the binary expansion of T_n from 1^n in time $2^{O(n)}$. Write $\mathsf{E} = \mathsf{DTIME}(2^{O(m)})$ for languages of strings of length m, and L_m for the length-m slice of a language L.

We measure the hardness of a power of P_n on the scale 2^k, where $k = n/2$. This is the natural scale: by Theorem 1.1 every power has $\mathrm{C}(P_n^T) \le 2^{k+O(\log k)}$, just as every m-input Boolean function has circuits of size $O(2^m/m)$. On these scales, explicit hard powers and circuit lower bounds for E are equivalent, with matching exponents.

Theorem 5.13 (Explicit hard powers and lower bounds for E)**.** *Let $0 < \varepsilon, \delta \le 1$. Read "for infinitely many" in both hypothesis and conclusion of each part, or "for all sufficiently large" in both.*

1. *If explicit exponents (T_n) satisfy $\mathrm{C}(P_n^{T_n}) \ge 2^{\varepsilon k}$, $k = n/2$, for infinitely many (for all sufficiently large) even n, then for every $\delta < \varepsilon$ some $L \in \mathsf{E}$ satisfies $\mathrm{C}(L_m) \ge 2^{\delta m}$ for infinitely many (for all sufficiently large) m.*
2. *If some $L \in \mathsf{E}$ satisfies $\mathrm{C}(L_m) \ge 2^{\delta m}$ for infinitely many (for all sufficiently large) m, then for every $\varepsilon < \delta$ there are explicit exponents (T_n) with $\mathrm{C}(P_n^{T_n}) \ge 2^{\varepsilon k}$, $k = n/2$, for infinitely many (for all sufficiently large) even n.*

Consequently, the supremum of the ε attained by explicit powers of P_n equals the supremum of the δ for which E has a language of circuit complexity $2^{\delta m}$, under either reading.

The second part uses a moment bound for sums of t-wise independent variables.

Lemma 5.14 (Bellare and Rompel [30, Lemma 2.3])**.** *Let $t \ge 4$ be even, and let $X = \sum_i X_i$ be a finite sum of t-wise independent random variables with values in $[0,1]$ and mean $\mu = \mathbb{E}X$. For every $A > 0$,*

$$\Pr[|X - \mu| \ge A] \le 8\left(\frac{t\mu + t^2}{A^2}\right)^{t/2}.$$

In particular, if $\mu \ge t$ then $\Pr[X < \mu/2] \le 8(8t/\mu)^{t/2}$.

The particular case takes $A = \mu/2$ and uses $4t^2/\mu^2 \le 4t/\mu$.

Proof of Theorem 5.13, part 1. For $m \ge 2$ let $k(m)$ be the largest $k \ge 1$ with $k + \lceil \log_2 k \rceil \le m$. Since $k \mapsto k + \lceil \log_2 k \rceil$ is strictly increasing, every $k \ge 1$ equals $k(m)$ for $m = k + \lceil \log_2 k \rceil$, and $m < (k(m)+1) + \lceil \log_2 (k(m)+1) \rceil$, so $m - k(m) \le \log_2 m + 2$.

A string of length m is read as (a, i, w), with $a \in \{0,1\}^k$ for $k = k(m)$, $i \in \{0,1\}^{\lceil \log_2 k \rceil}$, and padding w. Let L contain (a, i, w) exactly when $a \ge 1$, $i < k$, and bit i of $T_{2k} \bmod a$ is one. To decide L, compute $k(m)$, compute T_{2k} in time $2^{O(k)}$, and reduce it modulo a by long division in time $2^{O(k)}$. Hence $L \in \mathsf{E}$.

Hardwiring $i = 0, \dots, k-1$ and constant padding into k copies of a circuit for L_m computes $r_{T_{2k}}$ on all k-bit inputs, including $r_{T_{2k}}(0) = 0$. By Theorem 5.1,

$$k\,\mathrm{C}(L_m) \ge \mathrm{C}(r_{T_{2k}}) \ge \mathrm{C}(P_{2k}^{T_{2k}}) - O(k).$$

If $\mathrm{C}(P_{2k}^{T_{2k}}) \ge 2^{\varepsilon k}$, then $\mathrm{C}(L_m) \ge (2^{\varepsilon k} - O(k))/k$, which is at least $2^{\delta m}$ once k is large, because $m \le k + \log_2 k + 3$ and $\delta < \varepsilon$. For the "infinitely many" reading apply this at $m = k + \lceil \log_2 k \rceil$ for each hard k. For the "all sufficiently large" reading every large m has a large $k(m)$, and the hypothesis applies at $n = 2k(m)$. □

Proof of Theorem 5.13, part 2. Fix an even $t \geq 4$ with $\delta t/(t+2) > \varepsilon$.

Parameters. For $k \geq 2$ let $m(k)$ be the largest integer $m \geq 0$ with

$$\varphi_k(m) := m + \lceil 2(m+4)/t + \log_2(16tk) \rceil \leq k-1,$$

when such an m exists, and put $g = g(k) = k - 1 - m(k)$. Then $g \geq 2(m+4)/t + \log_2(16tk)$; in particular $g \geq 1$. The function φ_k is strictly increasing in m, and $\varphi_k(m) \leq \varphi_{k+1}(m) \leq \varphi_k(m) + 1$. Hence $\varphi_{k+1}(m(k)) \leq k$, so $m(k+1) \geq m(k)$; and $\varphi_{k+1}(m(k)+2) \geq \varphi_k(m(k)+1) + 1 \geq k+1$, so $m(k+1) \leq m(k) + 1$. Thus, once defined, $m(k)$ is nondecreasing, increases by at most one per step, and takes every sufficiently large integer value. Finally $\varphi_k(m) \leq (1 + 2/t)m + 8/t + \log_2(16tk) + 1$ gives

$$m(k) \geq \frac{t}{t+2}\, k - O(\log k). \tag{5.10}$$

A hashed partition of the k-bit integers. Every $p \in [2^{k-1}, 2^k)$ has a unique representation $p = 2^{k-1} + y2^g + z$ with $0 \leq y < 2^m$ and $0 \leq z < 2^g$, because $m + g = k - 1$. Since $g \geq 1$, p is even when z is even; so every prime in this range has z in the set Z of odd $z < 2^g$, and $|Z| = 2^{g-1}$. Let $w = \max(m, g)$, and identify $\{0,1\}^w$ with the field $\mathrm{GF}(2^w)$ through the lexicographically first irreducible polynomial of degree w, found by exhaustive search in time $2^{O(w)}$. Embed $z < 2^g$ and $y < 2^m$ into $\{0,1\}^w$ by zero-padding. For a polynomial f over $\mathrm{GF}(2^w)$ of degree less than t, let $G_f(z) \in \{0,1\}^m$ be the first m bits of $f(z)$, and define the *class* of $p = 2^{k-1} + y2^g + z$ as $r_f(p) = y \oplus G_f(z) \in \{0,1\}^m$. The class r is

$$\{2^{k-1} + (r \oplus G_f(z))2^g + z : 0 \leq z < 2^g\},$$

with exactly one element for each z, and the 2^m classes partition $[2^{k-1}, 2^k)$.

Write $\pi(x)$ for the number of primes at most x, and put

$$\mu = 2^{-m}(\pi(2^k) - \pi(2^{k-1})),$$

the average number of primes per class. By the prime number theorem (Chebyshev's estimates already suffice), $\pi(2^k) - \pi(2^{k-1}) \geq 2^{k-1}/(2k)$ for all large k, so $\mu \geq 2^g/(2k) \geq 8t$ by the choice of g. Call f *good* if every class contains at least $\mu/2$ primes.

A good f exists. Choose the t coefficients of f uniformly and independently. For any t distinct field elements the values of f are uniform and independent (the Vandermonde matrix is invertible), so the values $G_f(z)$ for distinct $z < 2^g$ are t-wise independent and uniform in $\{0,1\}^m$. Fix a class r and let $X_z = [2^{k-1} + (r \oplus G_f(z))2^g + z \text{ is prime}]$ for $z < 2^g$. These indicators are t-wise independent. For each fixed z the string $r \oplus G_f(z)$ is uniform, so $\mathbb{E}X_z = 2^{-m}\#\{y < 2^m : 2^{k-1} + y2^g + z \text{ is prime}\}$, and summing over z gives $\mathbb{E}\sum_z X_z = \mu$ exactly. No equidistribution of primes among residues is needed. Since $\mu \geq t$, Lemma 5.14 bounds the probability that class r contains fewer than $\mu/2$ primes by

$$8\left(\frac{8t}{\mu}\right)^{t/2} \leq 8\left(16tk\, 2^{-g}\right)^{t/2} \leq 8 \cdot 2^{-(m+4)} = 2^{-m-1},$$

where the last step uses $g - \log_2(16tk) \geq 2(m+4)/t$. A union bound over the 2^m classes shows that f is good with probability at least $1/2$.

The explicit exponent. For large even $n = 2k$ for which $m(k)$ is defined and $L_{m(k)}$ is to be encoded, do the following. Compute the truth table of L_m, $m = m(k)$, in time $2^{O(m)}$, and sieve the integers below 2^k. Try all at most $2^{tw} \leq 2^{tk}$ polynomials f in lexicographic order, and stop at

the first good one; checking a candidate means computing the class of each prime in $[2^{k-1}, 2^k)$ and counting, in time $2^{O(k)}$. Let T_n be the least nonnegative integer with

$$\begin{aligned} T_n &\equiv L_m(r_f(p)) \pmod{p} && \text{for every prime } p \in [2^{k-1}, 2^k), \\ T_n &\equiv 0 \pmod{p} && \text{for every prime } p < 2^{k-1}. \end{aligned}$$

The moduli are distinct primes and each prescribed residue is 0 or 1, so the Chinese remainder theorem gives T_n, smaller than the product of the primes below 2^k, hence below 4^{2^k}. The reconstruction is polynomial in 2^k. For all other even n, and if no good f exists, put $T_n = 0$. Since t is constant, the whole computation takes time $2^{O(k)} = 2^{O(n)}$, so (T_n) is explicit.

The reduction. Primality of k-bit integers is decidable in deterministic polynomial time [37], and a multitape machine running in time τ has circuits of size $O(\tau \log \tau)$ [38]. Hence some circuit PRIME_k of size $\mathrm{poly}(k)$ decides primality of k-bit integers. Evaluating G_f by Horner's rule costs t multiplications in $\mathrm{GF}(2^w)$, that is $O(tk^2)$ gates.

Let C be a circuit for r_{T_n}. Consider the randomized circuit that, on input $r \in \{0,1\}^m$, draws $u = 4km$ independent uniform $z_1, \ldots, z_u \in Z$; forms $p_j = 2^{k-1} + (r \oplus G_f(z_j))2^g + z_j$, a k-bit integer; tests each p_j with PRIME_k; selects the first prime p_j, or p_1 if there is none; and outputs the low bit of $C(p_j)$. Every p_j lies in class r, because $r_f(p_j) = (r \oplus G_f(z_j)) \oplus G_f(z_j) = r$. So whenever some p_j is prime, the output is the low bit of $T_n \bmod p_j = L_m(r)$, which is $L_m(r)$. Since f is good, class r contains at least $\mu/2$ primes, all with $z \in Z$; so each draw yields a prime with probability at least $(\mu/2)/2^{g-1} = \mu/2^g \geq 1/(2k)$. All u draws fail with probability at most $(1 - 1/(2k))^{4km} \leq e^{-2m} < 2^{-2m}$. A union bound over the 2^m inputs r shows that some fixed choice of $z_1, \ldots, z_u$ is correct on every input [39]. Hardwiring it gives

$$\mathrm{C}(L_m) \leq \mathrm{C}(r_{T_n}) + u \ \mathrm{poly}(k) \leq \mathrm{C}(P_n^{T_n}) + \mathrm{poly}(k),$$

by Theorem 5.1.

Conclusion. If $\mathrm{C}(L_m) \geq 2^{\delta m}$ at $m = m(k)$, then by Equation (5.10)

$$\mathrm{C}(P_n^{T_n}) \geq 2^{\delta m(k)} - \mathrm{poly}(k) \geq 2^{\delta \frac{t}{t+2} k - O(\log k)} - \mathrm{poly}(k) \geq 2^{\varepsilon k}$$

for all large k, since $\delta t/(t+2) > \varepsilon$. Because $m(k)$ eventually takes every large value and increases by at most one per step, infinitely many hard lengths m yield infinitely many hard even n, and hardness at all large m yields hardness at all large even n. □

The "all sufficiently large" reading of the second part's hypothesis, for some $\delta > 0$, is the hardness assumption of Impagliazzo and Wigderson [36]. By Theorem 5.13, that assumption holds exactly when some explicit family of powers of the linear-size permutation P_n has circuit complexity $2^{\Omega(n)}$ at all large even n. In particular, unconditionally exhibiting an explicit hard power of P_n is as hard as proving an exponential circuit lower bound for E.

The proof avoids primes in arithmetic progressions altogether. It needs no Linnik-type bound and no Bombieri–Vinogradov theorem, whose constant is ineffective; an encoding by progressions modulo $q \leq 2^{k/2}$ would also lose half of the exponent. The number theory used is Chebyshev's estimate, polynomial-time primality testing, and the Chinese remainder theorem. The hashing replaces a deterministic map from indices to primes, which is the obstacle in a direct encoding: a randomized circuit only has to find *some* prime in a prescribed class, and a 2^m-entry table of primes would cost about as much as the hardness to be transferred.

Together with Theorem 5.5 and Corollary 5.12, this describes the powers of P_n by the exponent's description. Polynomially long exponents give polynomial-size powers. Explicit exponents of

exponential length give powers that are hard exactly as far as E is hard. Arbitrary exponents of exponential length give powers that are hard unconditionally, by counting.

The amount of hardwired information is also forced. Distinct residues of T modulo D_B give distinct powers, and Theorem 5.1 recovers the remainder table r_T, hence T mod D_B, from P_n^T. Any description of P_n^T from which the map can be evaluated, whether a circuit, a table, or a checkpoint dictionary, therefore determines T mod D_B. By Lemma 5.2, for some T such a description has at least $\log_2 D_B \geq 2^{n/2} - O(n)$ bits. The dictionary of Section 3, with $O(n2^{n/2})$ bits at the balanced parameter, is within a polynomial factor of this minimum.

The Lean development does not formalize uniform complexity classes, so the results of this subsection are outside its checked boundary.

5.5 Short descriptions of hard exponents

Long binary expansions do not mean that hard exponents lack short descriptions. This subsection separates the length of an exponent from the complexity of describing it, and identifies the resource that matters. Its results are prose results; they are outside the Lean formalization of Section 12.

Fix a universal Turing machine U. For integers $T, n \geq 0$, the conditional Kolmogorov complexity $K(T \mid n)$ is the length of a shortest program p such that U, given p and the binary expansion of n, halts with the binary expansion of T on its output tape [43]. A different universal machine changes K by an additive constant only.

Proposition 5.15 (Hard exponents of constant description length)**.** *There is a constant c_U such that, for all sufficiently large even n, some exponent $T_n^* < D_B$ satisfies*

$$K(T_n^* \mid n) \leq c_U \qquad \text{and} \qquad \mathrm{C}(P_n^{T_n^*}) \geq c\,\frac{2^{n/2}}{n},$$

where $c > 0$ is the constant of Theorem 5.5. Consequently no bound of the form $\mathrm{C}(F^T) \leq \beta(n, \mathrm{C}(F), K(T \mid n))$ with β a polynomial holds, and any valid bound of this form satisfies $\beta(n, O(n), O(1)) = \Omega(2^{n/2}/n)$. The same holds for the unconditional complexity, since $K(T_n^*) \leq 2\log_2 n + O(1)$.

Proof. Decreasing c if necessary, take c rational. Let $\theta_n = \left\lceil c2^{n/2}/n \right\rceil$, and let T_n^* be the least $T \geq 0$ with $\mathrm{C}(P_n^T) \geq \theta_n$. By Theorem 5.5 such a T exists below D_B for all sufficiently large even n. The following fixed program computes T_n^* from n. For $T = 0, 1, 2, \ldots$ in turn, compute the truth table of P_n^T by iterating Equation (5.1) on each of the 2^n states. Then enumerate every circuit with fewer than θ_n gates and evaluate it on all 2^n inputs. Output the first T for which no such circuit agrees with the truth table. Each stage is a finite computation, and the search stops at T_n^*. Let c_U be the length of this program. For the unconditional bound, prepend a self-delimiting encoding of n. Since $\mathrm{C}(P_n) = O(n)$, these exponents defeat every polynomial β. □

By Corollary 5.12, the exponent T_n^* still has $\Omega(2^{n/2}/n^2)$ binary digits. So hard exponents are long but can be simple. The upper bound of Theorem 1.1 does not depend on T at all, so in terms of description complexity the two results match up to $2^{O(\log n)}$ factors.

The program in the proof is expensive. It tries up to D_B exponents, each against $2^{O(\theta_n \log \theta_n)}$ circuits, so it runs in time $2^{2^{O(n)}}$. It uses only space $2^{O(n)}$: at any moment it stores one exponent below D_B, one candidate circuit, and one truth table. We do not claim that a substantially faster short program for a hard exponent cannot exist. The next statements show what a cheaper program would imply.

For a family of exponents T_n, a *description in time* $t(n)$ (respectively *in space* $S(n)$) is a family of programs p_n of length $\mathrm{poly}(n)$ such that U, given p_n and n, outputs T_n within $t(n)$ steps (respectively, halts using at most $S(n)$ cells of work tape; the write-only output tape is not charged). For a family of maps, membership in $\mathsf{PSPACE}/\mathrm{poly}$ or $\mathsf{EXP}/\mathrm{poly}$ refers to the language of pairs (x, i) such that bit i of the image of x is one, with polynomial-length advice for each n.

Definition 5.16 (Residue profile)**.** For $T \geq 0$ and $n \geq 1$, the *residue profile* $\rho_{T,n} : \{0, 1, \ldots, 2^n\} \to \{0, 1, \ldots, 2^n\}$ is given by $\rho_{T,n}(p) = T \bmod p$ for $1 \leq p \leq 2^n$, and $\rho_{T,n}(0) = \min(T, 2^n)$.

Proposition 5.17 (Residue profiles control powers)**.** *Let* $F_n : \{0,1\}^n \to \{0,1\}^n$ *have* $\mathrm{C}(F_n) \leq \mathrm{poly}(n)$*, and let* $T_n \geq 0$*.*

(i) If the family $\rho_{T_n,n}$ *is in* $\mathsf{PSPACE}/\mathrm{poly}$*, then so is the family* $F_n^{T_n}$*. The same holds with* $\mathsf{EXP}/\mathrm{poly}$ *in place of* $\mathsf{PSPACE}/\mathrm{poly}$*.*

(ii) For the permutations of Equation (5.1)*, the family* $P_n^{T_n}$ *is in* $\mathsf{PSPACE}/\mathrm{poly}$ *if and only if the family of remainder maps* r_{T_n} *of Theorem 5.1 is.*

Proof. (i) Write $N = 2^n$, $T = T_n$, $F = F_n$, and take as advice the circuit of F together with the advice for $\rho_{T,n}$. Given x, compute $y = F^N(x)$ by N evaluations of F with an $(n+1)$-bit counter. The state y lies on the cycle reached from x. Its cycle length λ is the least $j \geq 1$ with $F^j(y) = y$. The tail length $\mu < N$ is the least i with $F^i(x) = F^{i+\lambda}(x)$. Each of these searches stores $O(n)$ bits besides one evaluation of F. Read $m = \rho_{T,n}(0)$. If $m < \mu$, then $T = m$, and the output is $F^m(x)$. Otherwise $T \geq \mu$, because $\mu < N$, and

$$F^T(x) = F^{\mu + (((T \bmod \lambda) - \mu) \bmod \lambda)}(x),$$

which uses the value $T \bmod \lambda = \rho_{T,n}(\lambda)$ and at most $\mu + \lambda \leq 2N$ further evaluations. Each query to $\rho_{T,n}$ is answered by recomputing the requested bits in polynomial space, so the whole computation runs in polynomial space with polynomial advice. The computation also runs in time $2^{O(n)}\,\mathrm{poly}(n)$ plus the cost of the queries, which gives the $\mathsf{EXP}/\mathrm{poly}$ statement.

(ii) The reductions in the proof of Theorem 5.1 are explicit polynomial-time computations in both directions: $r_T(a)$ is the second coordinate of $P_n^T(a, 0)$, and $P_n^T(a, b)$ is obtained from a, b, and $r_T(a)$ by one addition, one comparison, and one selection. □

Corollary 5.18 (Cheap descriptions give membership in a uniform class)**.** *Let* F_n *and* T_n *be as in Proposition 5.17.*

(i) If T_n *has a description in space* $\mathrm{poly}(n)$*, then* $F_n^{T_n}$ *is in* $\mathsf{PSPACE}/\mathrm{poly}$*. Hence, if* $\mathsf{PSPACE} \subseteq \mathsf{P}/\mathrm{poly}$*, then* $\mathrm{C}(F_n^{T_n}) \leq \mathrm{poly}(n)$*.*

(ii) If T_n *has a description in time* $2^{\mathrm{poly}(n)}$*, then* $F_n^{T_n}$ *is in* $\mathsf{EXP}/\mathrm{poly}$*. Hence, if* $\mathsf{EXP} \subseteq \mathsf{P}/\mathrm{poly}$*, then* $\mathrm{C}(F_n^{T_n}) \leq \mathrm{poly}(n)$*.*

Proof. (i) A description in space $S = \mathrm{poly}(n)$ halts within $2^{O(S)}$ steps, so it emits at most $2^{O(S)}$ output bits. To compute $T \bmod p$ for $p \leq 2^n$, rerun the program and fold each output bit into a running remainder as it appears. For most-significant-first output, replace r by $2r + \text{bit} \bmod p$. For least-significant-first output, also maintain $2^j \bmod p$. A running value capped at 2^n gives $\min(T, 2^n)$ in the same way: for least-significant-first output, it suffices to record whether a one appears at position n or later. This uses $S + O(n)$ cells, so $\rho_{T_n,n}$ is in $\mathsf{PSPACE}/\mathrm{poly}$ with the program as advice, and Proposition 5.17 applies. (ii) The same streaming computation runs in time $2^{\mathrm{poly}(n)}$. In both parts, the final implication follows by folding the polynomial advice into the input: the resulting language lies in PSPACE (respectively EXP), so it has polynomial-size circuits under the stated inclusion, and fixing the advice gives circuits for $F_n^{T_n}$. □

So among short descriptions, space rather than length or time is the relevant resource. Constant-length descriptions in space $2^{O(n)}$ reach hard exponents by Proposition 5.15. Polynomial-space descriptions reach a hard power only if $\mathsf{PSPACE} \not\subseteq \mathsf{P}/\operatorname{poly}$. Time-$2^{\operatorname{poly}(n)}$ descriptions reach one only if $\mathsf{EXP} \not\subseteq \mathsf{P}/\operatorname{poly}$.

Long but easy exponents. Several exponents with exponentially long expansions have residue profiles in PSPACE, so by Proposition 5.17 their powers of any polynomial-size map are in $\mathsf{PSPACE}/\operatorname{poly}$:

- $T = 2^{2^n}$: compute $2^{2^n} \bmod p$ by 2^n modular squarings with an n-bit counter.
- Power towers $T = 2 \uparrow\uparrow h$ of any height $h < 2^{\operatorname{poly}(n)}$, with h given as advice: reduce exponents modulo iterated totients, using $a^b \equiv a^{(b \bmod \varphi(p)) + \varphi(p)} \pmod p$ for $b \geq \log_2 p$. Whenever an intermediate exponent $2 \uparrow\uparrow j$ is below n, it is computed exactly and used directly, so the condition is needed only where it holds. The iterated totient of $p \leq 2^n$ reaches 1 after $O(n)$ steps, so only $O(n)$ levels need explicit work, and each totient is found by trial division in polynomial space.
- $T = D_B - 1$: $D_B \bmod p$ is a product of the largest prime powers not exceeding B, enumerated in polynomial space.

The last example is the inversion exponent of P_n, so $P_n^{D_B - 1} = P_n^{-1}$. As noted in Section 3.6, this power even has a linear-size circuit. Membership in $\mathsf{PSPACE}/\operatorname{poly}$ gives only an upper bound. These examples show that neither the length of an exponent nor its numerical size forces hardness.

5.6 The counting barrier

The lower bound of Theorem 5.4 counts the distinct powers of one map. This subsection shows how far such counting can go for an arbitrary map, which maps admit it at all, and why the exponent $1/2$ is also the limit of oracle-based constructions. Lean checks Lemma 5.19 for arbitrary maps on finite types (`iterate_eq_iterate_iff`, `card_range_iterate`) and the counting step behind Proposition 5.20 in its native basis: if the ordered description budget for q gates, or the explicit bound $2^{3q + h(2q+n+1)}$ with $n + q + 1 \leq 2^h$, is below $\mu(F) + \lambda(F)$, then some F^T with $T < \mu(F) + \lambda(F)$ has no circuit with at most q gates (`exists_hard_power_of_budget_lt`, `exists_hard_power`). The final arithmetic of Proposition 5.20 in the paper's basis and the remaining results of this subsection are not mechanized.

For $F : \{0,1\}^n \to \{0,1\}^n$ write $\mu(F)$ for the largest pre-period (transient depth) of a state, $\lambda(F)$ for the least common multiple of the cycle lengths of its functional graph, and $D(F)$ for the number of distinct maps among $F^0, F^1, F^2, \ldots$.

Lemma 5.19 (Number of powers)**.** *For integers $0 \leq a < b$, $F^a = F^b$ holds if and only if $a \geq \mu(F)$ and $\lambda(F) \mid b - a$. Consequently $D(F) = \mu(F) + \lambda(F)$.*

Proof. If $a \geq \mu$, then $F^a(x)$ lies on a cycle for every x, and the length of that cycle divides λ; so $\lambda \mid b - a$ gives $F^b(x) = F^a(x)$. Conversely, suppose $a < \mu$ and let x have pre-period μ. Then $F^a(x)$ has pre-period $\mu - a > 0$, while $F^b(x)$ has pre-period $\max(\mu - b, 0) < \mu - a$, so the two states differ. If $a \geq \mu$ but some cycle length ℓ does not divide $b - a$, then any x on that cycle has $F^a(x) \neq F^b(x)$. Since λ divides $b - a$ exactly when every cycle length does, this proves the equivalence. The powers $F^0, \ldots, F^{\mu + \lambda - 1}$ are therefore pairwise distinct, because two of them with $a < b$ would need $a \geq \mu$ and $0 < b - a < \lambda$; and every F^T with $T \geq \mu + \lambda$ equals $F^{T - \lambda}$. □

Proposition 5.20 (Counting over the powers of one map)**.** *For every $n \geq 4$ and every $F : \{0,1\}^n \to \{0,1\}^n$,*

$$\max_{T \geq 0} \mathrm{C}(F^T) \;\geq\; \frac{\log_2 D(F) - n^2 - n - 1}{3n} - 1.$$

Proof. Let $q = \lfloor (\log_2 D - n^2 - n - 1)/(3n) \rfloor$; if $q < 0$ there is nothing to prove. Every cycle length is at most N, so $\lambda \leq N!$ and $\log_2 D \leq \log_2(N + N!) \leq nN + 1$. Hence $q \leq N/3 + 1$, and $n + q + 2 \leq N$ for $n \geq 4$. By Lemma 5.3, at most

$$(q+1)16^q(n+q+2)^{2q+n} \leq 2^{\log_2(q+1)+4q+(2q+n)n} \leq 2^{3nq+n^2+n}$$

maps have complexity at most q, using $4q \leq nq$ and $\log_2(q+1) \leq n$. By the choice of q this is less than D, so by Lemma 5.19 some power has complexity greater than q. □

For $F = P_n$, where $\mu = 0$ and $\log_2 D = \log_2 D_B \geq 2^{n/2} - O(n)$, this recovers Theorem 5.4 up to the constant factor.

Corollary 5.21 (A ceiling on the number of powers)**.** *If $\mathrm{C}(F) \leq s$ and $A = s + n^3$, then*

$$\log_2 \lambda(F) \leq \log_2 D(F) = O\Big(nA + n^{5/2}\sqrt{NA}\Big).$$

Proof. Combine Proposition 5.20 with the upper bound Equation (1.2), which holds for every power. □

For comparison, let $g(N)$ be Landau's function, the largest order of a permutation of N points. The cycle lengths of any map on N states sum to at most N, so $\lambda(F) \leq g(N)$ for every F, and Landau's theorem [46], $\ln g(N) \sim \sqrt{N \ln N}$, gives

$$\log_2 g(N) = (1 + o(1))\frac{\sqrt{N \ln N}}{\ln 2} = \Theta(2^{n/2}\sqrt{n}).$$

Thus Corollary 5.21 says that a map with a polynomial-size circuit can have at most as many powers as an arbitrary map, up to $\mathrm{poly}(n)$ factors in the exponent, and Equation (5.4) shows that P_n, with $\log_2 D_B \geq 2^{n/2} - O(n)$, is extremal up to such factors. Conversely, a counting argument through Lemma 5.3 can certify hardness q only when $3nq + n^2 + n < \log_2 D(F) \leq \log_2(N + g(N))$, so it never certifies more than $O(2^{n/2}/\sqrt{n})$ for any map on n bits. Counting over the powers of a single map therefore cannot go beyond the exponent $1/2$ of Theorem 1.1; the matching exponent of Theorem 5.4 is a counting barrier, not only a limit of the construction.

The ceiling also says which maps have hard powers by counting. Let $c(F)$ be the number of distinct cycle lengths. Distinct lengths sum to at most N, so $c(F)(c(F)+1)/2 \leq N$, that is, $c(F) \leq \sqrt{2N}$, and by the arithmetic–geometric mean inequality $\log_2 \lambda(F) \leq c(F) \log_2(N/c(F))$. A bound of $2^{n/2}/n$ from Proposition 5.20 needs $\log_2 D(F) \geq 3 \cdot 2^{n/2}$, hence $c(F) = \Omega(2^{n/2}/n)$, within a factor $O(n)$ of the maximum; P_n has $c(P_n) = 2^{n/2} - 1$. Typical maps are far from this. For a uniformly random permutation of N points, $\ln \mathrm{ord}$ is asymptotically normal with mean $\frac{1}{2}(\ln N)^2$ [47], so $\log_2 \mathrm{ord} \approx \frac{\ln 2}{2} n^2 \approx 0.35\, n^2$ with high probability; for a uniformly random map, $\ln \lambda$ has mean asymptotic to $\frac{1}{8}(\ln N)^2$ [48], giving about $\frac{\ln 2}{8} n^2 \approx 0.09\, n^2$ bits. Both have $O(n)$ cycles in expectation. For such maps $\log_2 D(F) = O(n^2)$, every power equals $F^{T'}$ with an exponent T' of $O(n^2)$ bits, and counting gives nothing. Hard powers obtained by counting are therefore a non-generic phenomenon; for typical maps the question is that of Section 5.3, about exponents of polynomial length.

The image profile is the right parameter. Corollary 3.5 charges $|\operatorname{im}(F^d)|$ in place of N, and this is sharp. For $n = a + b$ with a even, let $F(u, v) = (P_a(u), 0^b)$ on $\{0,1\}^a \times \{0,1\}^b$. Then $\mathrm{C}(F) = O(n)$ and $|\operatorname{im}(F)| = 2^a$, so Corollary 3.5 with $d = 1$ gives $\max_T \mathrm{C}(F^T) = O(n^3 2^{a/2})$, however large b is. On the other hand $F^T(u, 0^b) = (P_a^T(u), 0^b)$ for $T \geq 1$, so a circuit for F^T yields one for P_a^T by fixing $v = 0^b$ and projecting, and Theorem 5.4 supplies $T \geq 1$ with $\mathrm{C}(F^T) = \Omega(2^{a/2}/a)$. Permutations, for which no image shrinks, are the hardest case, which is why the sharpness example of Section 5 is a permutation.

For random-like maps the image shrinks substantially. If $|\operatorname{im}(F^d)| \leq cN/d$ at $d = \left\lceil N^{1/3} \right\rceil$, then Corollary 3.5, with F^d unrolled in ds gates and $T < d$ handled by unrolling, gives

$$\mathrm{C}(F^T) = O\Big(N^{1/3}\big(s + n^{3/2}\sqrt{cA}\big)\Big) \qquad \text{for every } T.$$

For a uniformly random map and each fixed d, $\mathbb{E}|\operatorname{im}(F^d)|/N \to \tau_d$ as $N \to \infty$, where $\tau_0 = 1$, $\tau_{d+1} = 1 - e^{-\tau_d}$ and $\tau_d \sim 2/d$ [49]; so random maps heuristically have the profile $2N/d$, with $c \approx 2$. This is a statement about expectations for fixed d, not about any explicit family, and extending it to $d = \left\lceil N^{1/3} \right\rceil$ is a heuristic. Whether a natural explicit family of polynomial-size maps satisfies such an image profile is open. The implication itself is unconditional.

Black-box tightness. The construction uses F only through evaluations. In the preprocessing model, where an unbounded preprocessing stage produces an advice string of S bits about F and an online stage makes t oracle queries to F, the construction of Section 3 has $S = O(n(N/L + L))$ bits (payloads and residue table) and $t = O(L)$ queries for every T; the batched lookup makes no queries. So $S\,t = O(nN)$ for $L \leq \sqrt{N}$.

Proposition 5.22 (Black-box tightness)**.** *Let P be a uniformly random permutation of $\{0,1\}^n$. Suppose that an algorithm with S bits of advice about P and t oracle queries outputs $P^T(x)$ for every $T \geq 0$, or merely for $T = \operatorname{ord}(P) - 1$, with probability at least ε over P and a uniform x. Then $S\,t \geq \widetilde{\Omega}(\varepsilon N)$.*

Proof. $P^{\operatorname{ord}(P)-1} = P^{-1}$, and the preprocessing may compute $\operatorname{ord}(P)$. The algorithm therefore inverts P at a uniform point with probability ε, and the time–space lower bound for inverting a random permutation with advice [50, 51] gives the claim. □

Hence the tradeoff $S\,t = O(N \log N)$ of the construction, and its balancing point $L \approx \sqrt{N}$, are optimal up to polylogarithmic factors among all schemes that use the transition only as an oracle. Mazor and Pass [52, Theorem 5.8] give a related statement for circuits with oracle gates for the permutation. Improving the exponent of Corollary 3.6 for a specific permutation therefore requires using its circuit, not only its values. Whether the same holds for a fixed cycle type, such as a random relabelling $\sigma \circ P_n \circ \sigma^{-1}$ of P_n, does not follow directly from these bounds, which concern uniformly random permutations, and is left open. For noninjective maps the black-box question is also open: the image bound above gives advice times queries squared of $\widetilde{O}(N)$ for random-like maps, but verifying $y = F^T(x)$ costs about $\sqrt{N}$ queries, and the standard presampling and compression arguments do not apply directly.

5.7 A permutation near the counting ceiling

The counting argument of Theorem 5.4 is limited by the number of distinct powers. For P_n this number is $D_B = \operatorname{lcm}(1, \ldots, B)$, and $\log_2 D_B = O(B)$ by Chebyshev's bound on $\pi(B)$, so counting cannot certify more than $O(2^{n/2}/n)$ for P_n. This subsection gives a polynomial-size permutation

with $2^{\Theta(2^{n/2}\sqrt{n})}$ distinct powers whose hardest power has complexity $\Theta(2^{n/2}/\sqrt{n})$, within a constant factor of what any counting argument over powers can give. The family is nonuniform only through $O(n)$ hardwired bits. Lean checks only the combinatorial core; Remark 5.26 lists what is mechanized.

Construction. Fix a constant $C \geq 1$. For a large integer K put

$$g = \lceil \log_2(CK) \rceil, \qquad m = K - 1 - g, \qquad n_K = m + K = 2m + 1 + g.$$

States are pairs (r, b) with $r \in \{0,1\}^m$ and $b \in \{0,1\}^K$, read as integers. Fix an irreducible polynomial of degree m over GF(2) to represent $\mathrm{GF}(2^m)$, and for $\alpha, \beta \in \mathrm{GF}(2^m)$ let $f(z) = \alpha z + \beta$, where $z < 2^g \leq 2^m$ is embedded in $\mathrm{GF}(2^m)$ by its binary digits. Every integer $p \in [2^{K-1}, 2^K)$ is written uniquely as

$$p = 2^{K-1} + y\,2^g + z, \qquad 0 \leq y < 2^m,\ 0 \leq z < 2^g,$$

because $2^{K-1} = 2^m 2^g$. Assign p to the class $y \oplus f(z)$. Class r is therefore

$$S_r = \{\, p_r(z) = 2^{K-1} + (r \oplus f(z))2^g + z :\ 0 \leq z < 2^g \,\},$$

a set of exactly 2^g integers, and the classes partition $[2^{K-1}, 2^K)$. Let ℓ_r be the prime $p_r(z)$ with least z, or $\ell_r = 1$ if S_r contains no prime. Define

$$F(r,b) = \begin{cases} (r,\ b+1 \bmod \ell_r), & \ell_r > 1 \text{ and } b < \ell_r, \\ (r, b), & \text{otherwise.} \end{cases}$$

Lemma 5.23 (Structure)**.** *F is a permutation of $\{0,1\}^{n_K}$ with $\mathrm{C}(F) = \mathrm{poly}(K)$. It has one cycle of length ℓ_r for each r with $\ell_r > 1$, and every other state is fixed. The lengths $\ell_r > 1$ are distinct primes in $[2^{K-1}, 2^K)$, so*

$$\mathrm{ord}(F) = \lambda := \prod_{r:\,\ell_r > 1} \ell_r,$$

and the powers F^T, $0 \leq T < \lambda$, are pairwise distinct.

Proof. For fixed r with $\ell_r > 1$ the second coordinate moves around the cycle $0, 1, \ldots, \ell_r - 1$; the states with $b \geq \ell_r$, and all states with $\ell_r = 1$, are fixed. So F is a permutation with the stated cycles. Different classes are disjoint, so the primes ℓ_r are distinct, and the order of F is their product. Distinct residues of T modulo λ give distinct powers, since some cycle length ℓ_r does not divide their difference.

For the circuit, compute the $2^g \leq 2CK$ candidates $p_r(z)$; each needs one multiplication in $\mathrm{GF}(2^m)$, $O(m^2)$ gates. Test each for primality with a $\mathrm{poly}(K)$-size circuit, obtained from the Agrawal–Kayal–Saxena algorithm [37] by the Pippenger–Fischer simulation [38]. Select the first prime by a prefix-OR scan, compare b with ℓ_r, and update b with an incrementer and an equality test. The total is $\mathrm{poly}(K)$. □

Write ν for the fraction of classes r with $\ell_r > 1$, and

$$\mu = 2^{-m}(\pi(2^K) - \pi(2^{K-1}))$$

for the average number of primes per class. By the prime number theorem, $\pi(2^K) - \pi(2^{K-1}) = (1 + o(1))2^{K-1}/(K \ln 2)$, so

$$\mu = (1 + o(1))\frac{2^g}{K \ln 2} \geq (1 - o(1))\frac{C}{\ln 2}.$$

The factor $1 - o(1)$ is needed: only the prime number theorem, not Chebyshev's bound, gives the constant $1/\ln 2$.

Lemma 5.24 (Most classes contain a prime)**.** *For some α, β, the fraction of classes containing a prime is at least $\mu/(1+\mu)$. Hence $\nu \geq \mu/(1+\mu) \geq c_C - o(1)$ for this choice, where $c_C = C/(C+\ln 2)$.*

Proof. Choose α, β uniformly and fix a class r. Let $X_z = [p_r(z) \text{ is prime}]$ and $X = \sum_{z<2^g} X_z$. For each z the value $f(z)$ is uniform, so $p_r(z)$ is uniform among the 2^m integers of $[2^{K-1}, 2^K)$ with low part z; summing over z gives $\mathbb{E}X = \mu$. For $z_1 \neq z_2$ the pair $(f(z_1), f(z_2))$ is uniform, so the X_z are pairwise independent and $\operatorname{Var} X \leq \sum_z \mathbb{E}X_z = \mu$. By the Cauchy–Schwarz inequality,

$$\Pr[X > 0] \geq \frac{(\mathbb{E}X)^2}{\mathbb{E}X^2} \geq \frac{\mu^2}{\mu + \mu^2} = \frac{\mu}{1+\mu}.$$

The expected number of nonempty classes is therefore at least $\mu 2^m/(1+\mu)$, and some (α, β) attains the expectation. The function $x \mapsto x/(1+x)$ is increasing, which gives the bound on ν. □

The lower bound uses the sharp circuit count of Lemma 5.7, which discounts relabellings of minimum circuits.

Lemma 5.3 counts every encoding of a circuit and so carries an extra factor of about 2 in the exponent; that would halve the constant in the next theorem.

Theorem 5.25 (Hard powers near the counting ceiling)**.** *With α, β as in Lemma 5.24,*

$$\nu(1 - o(1))\, 2^m \leq \max_{T \geq 0} \mathrm{C}(F^T) \leq (1 + o(1))\, 2^m, \qquad 2^m = \Theta\left(\frac{2^{n_K/2}}{\sqrt{n_K}}\right).$$

Consequently, for every constant $C \geq 1$ and every large n there is a permutation F_n of $\{0,1\}^n$ with $\mathrm{C}(F_n) = \mathrm{poly}(n)$ and

$$\max_{T \geq 0} \mathrm{C}(F_n^T) = \Theta\left(\frac{2^{n/2}}{\sqrt{n}}\right).$$

Proof. Upper bound. For every T,

$$F^T(r,b) = \begin{cases} (r,\, (b + (T \bmod \ell_r)) \bmod \ell_r), & \ell_r > 1 \text{ and } b < \ell_r, \\ (r, b), & \text{otherwise.} \end{cases}$$

The table $r \mapsto T \bmod \ell_r$, with value 0 when $\ell_r = 1$, has m input bits and K output bits. By Lupanov's bound [28], each output bit has a circuit of size $(1+o(1))2^m/m$. Since $K = m + O(\log m)$, the K bits together cost $(1 + o(1))2^m$. The circuit for ℓ_r from Lemma 5.23, the comparison, the addition and the reduction modulo ℓ_r add $\mathrm{poly}(K) = o(2^m)$ gates.

Lower bound. Let $q = \max(\max_T \mathrm{C}(F^T), n_K)$. If $q > 2^{m+1}$ there is nothing to prove. Otherwise $\log_2 q \leq m + 1$, and by Lemmas 5.7 and 5.23 with both input and output width n_K,

$$\nu 2^m (K-1) \leq \log_2 \lambda \leq q \log_2 q + O(q) + O(n_K \log q) \leq q(m + O(1)) + O(m^2),$$

using $\ell_r \geq 2^{K-1}$ for each of the $\nu 2^m$ nonempty classes. Since $K - 1 = m + g$ and ν is bounded below by Lemma 5.24, $q \geq \nu(1 - O(1/m))2^m - O(m) = \nu(1 - o(1))2^m$. This exceeds n_K for large K, so $q = \max_T \mathrm{C}(F^T)$.

Every length. We have $2^m = 2^{(n_K - 1 - g)/2}$ and $CK \leq 2^g < 2CK$ with $K = n_K/2 + O(\log n_K)$, which gives $2^m = \Theta(2^{n_K/2}/\sqrt{n_K})$, with constants depending on C. As K increases by one, n_K

increases by one or two, so every large n is n_K or $n_K + 1$ for some K. In the second case let $F_n(x, e) = F(x)$ on the first n_K bits and leave the last bit e fixed. Each power of F_n restricts to the same power of F when $e = 0$, and is computed by a circuit for that power of F with e passed through, so $\max_T \mathrm{C}(F_n^T) = \max_T \mathrm{C}(F^T)$. □

Remark 5.26 (Mechanized part). The module `PrimeCycles.lean` checks the following for every function f with values below 2^m, not only for the affine maps. The lengths $\ell_r > 1$ are distinct primes in $[2^{K-1}, 2^K)$ (`classPrime_coprime`). For arbitrary lengths below 2^K, F^T is given by the formula in the proof of Theorem 5.25 and depends on T only through the table $r \mapsto T \bmod \ell_r$ (`power_apply_mod`). For pairwise coprime lengths, $F^S = F^T$ exactly when $S \equiv T \pmod{\lambda}$, so λ is the order of F and its number of distinct powers (`iterate_eq_iterate_iff`, `card_range_iterate`). Finally, `near_landau_lower_bound` proves that F is a bijection with $\lambda \geq 2^{(K-1)c}$, where $c = \nu 2^m$ is the number of classes containing a prime, and that if $3q + h(2q + n_K + 1) < (K - 1)c$ and $n_K + q + 1 \leq 2^h$, then some F^T with $T < \lambda$ has no native circuit with at most q gates. This step counts ordered circuit descriptions as in Lemma 5.3, not minimum circuits as in Lemma 5.7. With $h = m + O(1)$ it therefore gives the lower bound of Theorem 5.25 with $\nu/2$ in place of ν. The circuit for F, the prime number theorem estimate, Lemma 5.24, Lemma 5.7, the upper bound, and Corollary 5.28 remain prose results.

Remark 5.27 (The counting ceiling). The lower bound above, like that of Theorem 5.4, uses only the number $D(F)$ of distinct powers, and it gives q with $q \log_2 q \approx \log_2 D(F)$. For every map F on $N = 2^n$ points, $D(F) \leq N + g(N)$, where $g(N)$ is the largest order of a permutation of N points: a map has $\mu + \lambda$ distinct powers, $\mu \leq N$, and λ is the lcm of cycle lengths summing to at most N. Landau's theorem [46] gives $\ln g(N) = (1 + o(1))\sqrt{N \ln N}$, so $\log_2 \log_2 D(F) = (1/2 + o(1))n$ and this argument can never certify more than

$$(1 + o(1))\frac{\log_2 D(F)}{\log_2 \log_2 D(F)} \leq \left(\frac{2}{\sqrt{\ln 2}} + o(1)\right)\frac{2^{n/2}}{\sqrt{n}} \approx 2.40 \cdot \frac{2^{n/2}}{\sqrt{n}}.$$

Theorem 5.25 reaches this ceiling up to a constant factor that depends on C and on the rounding of g. The general upper bound of Theorem 1.1 is $2^{n/2}\,\mathrm{poly}(n)$; whether some polynomial-size map has $\max_T \mathrm{C}(F^T)$ much larger than $2^{n/2}/\sqrt{n}$ is beyond counting.

Unlike P_n, the cycles of F all have lengths of the same order and carry a constant fraction of the states. This yields constant average-case hardness for random powers.

Corollary 5.28 (Constant average-case hardness)**.** *Let $0 < \delta < \nu/4$ be constant and let T be uniform modulo λ. With probability $1 - o(1)$, every circuit of size at most $\delta 2^m$ disagrees with F^T on at least a $\nu/2 - 2\delta - o(1)$ fraction of the 2^{n_K} states.*

Proof. Put $q = \lfloor \delta 2^m \rfloor$ and $\varepsilon = 1/\log_2 K$. Fix a map $G : \{0,1\}^{n_K} \to \{0,1\}^{n_K}$ and a class r with $\ell = \ell_r > 1$. On the cycle states (r, b), $b < \ell$, G is correct exactly when $G(r, b) = (r, v)$ with $v < \ell$ and $v \equiv b + T \pmod{\ell}$. So each such state names at most one residue of T modulo ℓ, and at most $1/\varepsilon$ residues are named by more than $\varepsilon\ell$ states. Call r *good* for (G, T) if $T \bmod \ell_r$ is one of them. Since T is uniform modulo $\lambda = \prod \ell_r$, the residues $T \bmod \ell_r$ are independent and uniform, so the classes are good independently, each with probability at most $1/(\varepsilon\ell_r) \leq 2/(\varepsilon 2^K)$. With at most 2^m nonempty classes,

$$\Pr[\text{at least } 2q \text{ good classes}] \leq \binom{2^m}{2q}\left(\frac{2}{\varepsilon 2^K}\right)^{2q} \leq \left(\frac{e\,2^{m-K}}{\varepsilon q}\right)^{2q} \leq \left(\frac{e}{2CK\varepsilon q}\right)^{2q},$$

because $2^{m-K} = 2^{-1-g} \le 1/(2CK)$. By Lemma 5.7, at most $2^{q\log_2 q+O(q)+O(m^2)}$ maps have circuits of size at most q. A union bound over them gives total probability at most $2^{-q(\log_2 q+2\log_2(K\varepsilon)-O(1))+O(m^2)} = o(1)$.

So with probability $1-o(1)$, every circuit of size at most q has fewer than $2q$ good classes, and is wrong on at least a $1-\varepsilon$ fraction of the cycle states of each other nonempty class. The cycle states of the nonempty classes number at least $\nu 2^m 2^{K-1} = (\nu/2)2^{n_K}$, and the good classes contain at most $2q\,2^K \le 2\delta 2^{n_K}$ of them. The error fraction is therefore at least $(1-\varepsilon)(\nu/2-2\delta) = \nu/2-2\delta-o(1)$. □

For P_n, Proposition 5.9 and Theorem 5.10 instead show that at size $2^{n/2}/\operatorname{poly}(n)$ the optimal approximation error tends to zero. Thus the near-ceiling construction also changes the average-case behavior, not only the polynomial factor in worst-case complexity.

Remark 5.29 (Uniformity). Apart from the irreducible polynomial, F is determined by the $2m$ bits of (α,β). The polynomial and a good pair can be found deterministically in time $2^{O(n)}$ by exhaustive search, after sieving $[2^{K-1},2^K)$ once. A uniformly random pair has at most $2\cdot 2^m/(1+\mu)$ empty classes with probability at least $1/2$, by Markov's inequality, so randomness also works when $\mu>1$. A deterministic polynomial-time construction with the same parameters would need an efficiently computable injective map from about $2^K/K$ indices to K-bit primes that hits a constant fraction of all of them. This is related to the construction of primes: Chen, Lu, Oliveira, Ren and Santhanam give a polynomial-time pseudodeterministic construction of primes for infinitely many lengths [55], which is not enough here. Replacing the hash classes by the intervals $[2^{K-1}+rCK,\ 2^{K-1}+(r+1)CK)$ would give an explicit family. Under the Hardy–Littlewood prime k-tuples conjecture, Gallagher [56] shows that the number of primes in such intervals is asymptotically Poisson distributed, which yields a constant fraction of nonempty intervals and the same $\Theta(2^{n/2}/\sqrt{n})$ bound. This route is conditional.

6 Input times and shared evaluation

6.1 An iteration count supplied as input

Definition 6.1. For $m \ge 1$, define

$$\mathrm{Iter}_{F,m} : \{0,1\}^n \times \{0,1\}^m \longrightarrow \{0,1\}^n, \qquad \mathrm{Iter}_{F,m}(x,t) = F^{\mathrm{val}(t)}(x),$$

where $\mathrm{val}(t)$ is the nonnegative integer encoded by t.

Theorem 6.2 (Variable-time iteration)**.** *Under the assumptions of Theorem 1.1, for every $m \ge 1$,*

$$\mathrm{C}(\mathrm{Iter}_{F,m}) = O\Big(m\left(A+\sqrt{n^3NA}\right)\Big). \tag{6.1}$$

In particular,

$$\mathrm{C}(\mathrm{Iter}_{F,n}) = O\Big(n\left(A+\sqrt{n^3NA}\right)\Big).$$

If s and m are polynomial in n, then $\mathrm{C}(\mathrm{Iter}_{F,m}) \le 2^{n/2+O(\log n)}$.

Proof. Write $t = \sum_{j=0}^{m-1} t_j 2^j$. For each j, apply Theorem 1.1 directly to the original map F and the fixed exponent 2^j, obtaining a circuit for $G_j = F^{2^j}$. Starting with $v_0 = x$, use

$$v_{j+1} = \begin{cases} G_j(v_j), & t_j = 1, \\ v_j, & t_j = 0. \end{cases}$$

Inductively, $v_j = F^{\sum_{a<j} t_a 2^a}(x)$, so $v_m = F^t(x)$. There are m circuits of the size in Equation (1.2) and m n-bit multiplexers. The extra $O(mn)$ gates are absorbed by the claimed bound. Each G_j is constructed from F using the theorem; it is not obtained by duplicating the circuit for G_{j-1}. □

Theorem 6.3 (Long input times)**.** *Write $B = A + \sqrt{n^3 NA}$. For every $m \geq 1$,*

$$\mathrm{C}(\mathrm{Iter}_{F,m}) = O(\min\{mB,\ nB + nm\})\,.$$

Thus the cost of processing additional time bits is additive once the state width is fixed.

Proof. The first bound is Theorem 6.2. For the second, compute the exact eventual cycle length $\ell = \ell(x)$ in $O(B)$ gates using Theorem 7.1. Replace the input time by

$$u = \begin{cases} t, & t < N, \\ N + (t - N) \bmod \ell, & t \geq N. \end{cases}$$

Then $u < 2N$ and $F^u(x) = F^t(x)$: in the second case both times are past cycle entry and have the same residue modulo ℓ.

Compute $t \bmod \ell$ by streaming its bits from most significant to least significant, retaining an $O(n)$-bit remainder. Each step doubles the remainder, adds one bit, and conditionally subtracts ℓ; a spare bit prevents overflow. Shared-carry addition and subtraction give $O(n)$ gates per input bit, including comparison and selection. The bounded modular correction by N and the final selection cost $O(n^3 + m)$ with the arithmetic circuits already used in the proof. Hence forming u costs $O(nm + n^3)$, and its $n + 1$ bits feed Theorem 6.2 at cost $O(nB)$. Since $n^3 \leq B$, the second bound follows. If $m \leq n$, one may simply use the original m-stage construction. □

6.2 Sharing a dictionary among independent trajectories

Let

$$B_{F,T,r}(x_1, \ldots, x_r) = (F^T(x_1), \ldots, F^T(x_r)).$$

The inputs $x_1, \ldots, x_r$ are independent n-bit words. The following construction uses one checkpoint dictionary for all of them.

Theorem 6.4 (Batched fixed powers)**.** *Under the assumptions of Theorem 1.1, for every $1 \leq r \leq N$, $T \geq 0$, and $1 \leq L \leq N$,*

$$\mathrm{C}(B_{F,T,r}) = O\Big(rLs + n^3\left(rL + \frac{N}{L}\right)\Big)\,. \tag{6.2}$$

Consequently,

$$\mathrm{C}(B_{F,T,r}) = O\Big(rA + \sqrt{rn^3 NA}\Big)\,. \tag{6.3}$$

Proof. Set $K = 2L$. If $T < K$, unroll T copies of F independently for each input. Otherwise use the checkpoint set and dictionary from Section 3. Generate the first $K + 1$ states of all r trajectories. Together with one copy of the dictionary, these give

$$R = |H| + r(K + 1) = O(N/L + rL)$$

records. Attach the trajectory number and within-trajectory position to each query. Because $r \leq N$ and $L \leq N$, the record width and $\log(R + 2)$ are $O(n)$. One application of Lemma 2.1 therefore resolves all queries in $O(n^3(N/L + rL))$ gates and returns them in their original order.

Select a matching checkpoint separately within each trajectory. Run repeated-state detection separately on each trajectory as well; this costs $O(rn^3L)$ gates and does not confuse a collision between two different trajectories with a repetition along one trajectory. The correction trajectory and output selection are also performed separately for each input. The fixed residue table may be implemented with a separate $O(nL)$-gate lookup for each trajectory, whose total cost is absorbed by $O(rn^3L)$. The two simulation phases together cost $O(rLs)$ gates. This proves Equation (6.2).

Choose

$$L = \max\left\{1, \left\lceil \sqrt{n^3N/(rA)} \right\rceil \right\}.$$

As before $1 \le L \le N$. If $rA \le n^3N$, then $rAL = O(\sqrt{rn^3NA})$ and $n^3N/L = O(\sqrt{rn^3NA})$. If $rA > n^3N$, the choice is $L = 1$ and the cost is $O(rA)$. This gives Equation (6.3). □

Corollary 6.5 (Independent input times)**.** *For $1 \le r \le N$ and $m \ge 1$, the map*

$$\big((x_1, t_1), \ldots, (x_r, t_r)\big) \longmapsto \big(F^{t_1}(x_1), \ldots, F^{t_r}(x_r)\big), \qquad t_i \in \{0,1\}^m,$$

has a circuit of size

$$O\Big(m\left(rA + \sqrt{rn^3NA}\right)\Big). \tag{6.4}$$

Proof. Use the binary-power construction of Theorem 6.2. At stage j, apply Theorem 6.4 with update map F and exponent 2^j on the r current states, then use the jth bit of each t_i to select whether to retain or update that state. The $O(mrn)$ selection gates are absorbed by Equation (6.4). □

For polynomial s and m, and $r = \lfloor 2^{\alpha n} \rfloor$ with fixed $0 \le \alpha \le 1$, Equation (6.4) is at most

$$2^{(1+\alpha)n/2 + O(\log n)}.$$

This is an upper-bound sharing statement. It is not a direct-sum separation against optimal circuits for a single trajectory.

6.3 Batched powers of the counting permutation

For the permutation P_n of Section 5, sharing costs much less than Theorem 6.4 suggests. Lean checks Proposition 6.6 for the modulus counter in its native basis, in the theorems `batched_counter_power` and `batch_lower_bounds` of the namespace `ModulusCounter`. The upper bound is $40000(k+1)^3(2^k + r)$ gates for $r \le N$, with Batcher networks in the shared lookup. One block of a batched circuit gives a circuit for P_n^T of no larger size, and a batched circuit has at least r gates when P_n^T is not the identity. `batched_hard_counter_power` combines both bounds for the hard exponents of Theorem 5.4. The AKS variant and the counting discussion below are not mechanized.

Proposition 6.6 (Batched powers of P_n)**.** *Let $n = 2k$ be even, $1 \le r \le N$, and $T \ge 0$. Then*

$$\mathrm{C}(B_{P_n,T,r}) = O\Big(n^3(2^{n/2} + r)\Big).$$

If P_n^T is not the identity, then also

$$\mathrm{C}(B_{P_n,T,r}) \ge \max\{\mathrm{C}(P_n^T),\, r\}.$$

Proof. For the upper bound, use the dictionary with keys $a = 1, \ldots, B$ and payloads $r_T(a) = T \bmod a$, in the notation of Theorem 5.1. It has $B < 2^{n/2}$ entries of k bits. The first coordinates $a_1, \ldots, a_r$ of the r inputs are the queries. Since $r \le N$, Lemma 2.1 returns all $r_T(a_i)$ in $O(n^3(2^{n/2}+r))$ gates. The query $a_i = 0$ is unmatched and receives payload $0 = r_T(0)$. Each input is then completed as in the proof of Theorem 5.1 with $O(n)$ further gates.

For the lower bound, fixing all inputs but the first block to zero and keeping the first output block gives a circuit for P_n^T. For the bound r, suppose that every output bit of the second coordinate of P_n^T is a constant or an input variable. On the inputs with $a = 0$ every state is fixed, so output bit j agrees with b_j there; it is therefore not constant, and the only input variable that agrees with b_j on these inputs is b_j. Hence P_n^T is the identity. Otherwise, some output bit of each of the r blocks is computed by a gate. These r output functions depend on disjoint sets of variables and are nonconstant, so they are pairwise distinct and need r distinct gates. □

With Lemma 6.7 the factor n^3 becomes n^2. For the hard exponents of Theorem 5.4, the batched cost of P_n is therefore between $\Omega(2^{n/2}/n + r)$ and $O(n^2(2^{n/2} + r))$. Each additional input costs $\mathrm{poly}(n)$ gates. The generic bound of Theorem 6.4 is $O(n^3\sqrt{r}\,2^{n/2})$ for P_n, since $A = \Theta(n^3)$, so the factor $\sqrt{r}$ is not attained here.

Counting cannot show more. As T varies, the number of distinct maps $B_{F,T,r}$ equals the number $D(F)$ of distinct powers of F, since the first output block is F^T. By Theorem 1.1, every power has a circuit of size $S^* = O(A + \sqrt{n^3 NA})$, so Lemma 5.3 gives $\log D(F) = O((S^* + n)\log(n + S^* + 2))$. For polynomial s this is $2^{n/2}\,\mathrm{poly}(n)$. A counting argument over exponents compares $D(F)$ with the number of circuits on rn inputs, which only grows with r. It therefore certifies at most $2^{n/2}\,\mathrm{poly}(n)$ gates for every r, and counting over all maps of polynomial size as well adds only $\mathrm{poly}(n)$ to the logarithm. For P_n, counting gives $\Omega(2^{n/2}/n)$ while $r \le 2^{n/2}/n^2$, which the restriction to one block already gives. Once r exceeds a suitable polynomial in n, no counting argument over powers of polynomial-size maps reaches the upper bound of Theorem 6.4.

Mass production. Uhlig showed that for every $f : \{0,1\}^n \to \{0,1\}$ and every $r = 2^{o(n/\log n)}$, evaluating f on r independent inputs has circuits of size $(1 + o(1))2^n/n$ [61, 62]. Applied to each output bit, this gives $\mathrm{C}(B_{F,T,r}) \le (1+o(1))2^n$ for every map F in that range of r, with no assumption on $\mathrm{C}(F)$. So Boolean circuits have no direct-sum behaviour for repeated evaluation even without structure. For polynomial s, Equation (6.3) is smaller, $2^{n/2+o(n)}$, throughout this range of r, and it remains $O(2^n)$ for all $r \le N/(n^3 A)$.

An open question. Is there a family F_n of polynomial size, a constant $0 < \alpha < 1$, and exponents T_n such that, for $r = \lfloor 2^{\alpha n} \rfloor$,

$$\mathrm{C}(B_{F_n,T_n,r}) \ge 2^{(1+\alpha)n/2 - o(n)}?$$

By Equation (6.3) this exponent would be the largest possible. For P_n the exponent is $\max\{1/2, \alpha\}$, which is smaller, and counting cannot answer the question. We know no polynomial-size family for which any exponent strictly between $\max\{1/2, \alpha\}$ and $(1 + \alpha)/2$ is proved.

6.4 A polynomial improvement with AKS sorting networks

The factor n^3 in Theorem 1.1 has two sources: $O(n)$ gates per comparator on $O(n)$-bit records, and the $O(\log^2 R) = O(n^2)$ comparators per record of Batcher's networks. Ajtai, Komlós, and Szemerédi constructed comparator networks sorting R keys with $O(R \log R)$ comparators [2]. Replacing

Batcher's networks by theirs saves one factor of n. Their constants are very large, so this is an asymptotic statement only. The Lean development mechanizes the Batcher version; nothing in this subsection is mechanized.

Lemma 6.7 (Lookup and repetition with AKS networks)**.** *The dictionary lookup of Lemma 2.1 and the repeated-state detection of Lemma 2.2 have circuits of sizes*

$$O((M+Q)(w+\log(M+Q+2))\log(M+Q+2)) \quad \textit{and} \quad O(Q(w+\log(Q+2))\log(Q+2)),$$

respectively. In particular, for $w = O(n)$ and $\log(M+Q+2) = O(n)$ both costs are $O(n^2(M+Q))$.

Proof. Repeat the proofs of Lemmas 2.1 and 2.2 with every sorting network replaced by an AKS network on the same records, padded with dummy records as before. The networks use $O(R\log(R+2))$ comparators, each of cost $O(w+\log(R+2))$. The scans and the selection of a flagged record cost $O(R(w+\log(R+2)))$ gates and are unchanged. □

Theorem 6.8 (AKS-strengthened tradeoff)**.** *Under the assumptions of Theorem 1.1, put $A_2 = s+n^2$. For every integer $T \geq 0$ and every integer $1 \leq L \leq N$,*

$$\mathrm{C}(F^T) = O\Big(Ls + n^2\left(L + \frac{N}{L}\right)\Big), \qquad \textit{hence} \qquad \mathrm{C}(F^T) = O\Big(A_2 + n\sqrt{NA_2}\Big).$$

Proof. Use the construction of Section 3 unchanged, and implement its lookup and repeated-state detection by Lemma 6.7. These now cost $O(n^2(N/L+L))$ and $O(n^2L)$ gates. Every other component already fits this bound: the two simulations cost $O(Ls)$; the residue-table multiplexer, the trajectory selections, and the final output selection cost $O(nL)$; and the long division and modular correction of Equation (3.4) cost $O(n^2)$. For the optimized bound, choose $L = \max\{1, \lceil\sqrt{n^2N/A_2}\rceil\}$ and argue as for Equation (3.6), with n^3 replaced by n^2 and A by A_2. Then $L \leq \lceil\sqrt{N}\rceil \leq N$ since $A_2 \geq n^2$, and $\sqrt{n^2NA_2} = n\sqrt{NA_2}$. □

The proofs of the following results use the lookup and repetition circuits only through the per-record cost of Lemmas 2.1 and 2.2, together with $O(Ls)$ simulation, $O(nL)$ selection, and $O(n^2)$ arithmetic. Each therefore holds with n^3 replaced by n^2 in its parameterized bound and with A replaced by A_2 in its optimized bound: the long-trajectory, invariant-set and image-sensitive bounds (Theorems 3.3 and 3.4 and Corollary 3.5), exact permutation inversion (Corollary 3.6), variable-time iteration (Theorem 6.2, by m applications of Theorem 6.8), and batched fixed powers and independent input times (Theorem 6.4 and Corollary 6.5). For the batched bound, the shared lookup costs $O(n^2(N/L+rL))$ and the r separate repetition tests cost $O(rn^2L)$, giving

$$\mathrm{C}(B_{F,T,r}) = O\Big(rA_2 + n\sqrt{rNA_2}\Big).$$

For the permutation of Section 5, $\mathrm{C}(P_n) = O(n)$ gives $A_2 = O(n^2)$ and $\max_T \mathrm{C}(P_n^T) = O(n^2 2^{n/2})$, within a factor $O(n^3)$ of the lower bound in Theorem 5.4. We do not claim the improvement for the navigation and target results of Section 7 or for Theorem 6.3, whose proofs contain further n^3 terms that we have not re-examined.

6.5 Depth

The constructions above were stated for size only; for instance, the selection scan in the primitives is a sequential chain. Every component except the two short simulations of F can be made shallow

without increasing its size by more than a constant factor, so the depth of a fixed-power circuit is governed by the $O(L)$ sequential applications of F. Nothing in this subsection is mechanized.

Theorem 6.9 (Depth–size tradeoff)**.** *Let F have a circuit of size s and depth d, and let $S \subseteq \{0,1\}^n$ be forward invariant under F with $|S| = M$; the case $S = \{0,1\}^n$ gives $M = N$. For every $T \geq 0$ and $1 \leq L \leq M$ there is a circuit agreeing with F^T on S of size*

$$O\Big(Ls + n^3\left(L + \frac{M}{L}\right)\Big) \quad \text{and depth} \quad O\Big(Ld + n^2 \log n\Big).$$

With the comparator networks of Section 6.4 the size is $O(Ls + n^2(L + M/L))$ and the depth is $O(Ld + n \log n)$.

Proof. Use the construction of Section 3, with the invariant-set checkpoints of Theorem 3.4, and implement each component as follows; all record widths are $w = O(n)$ and all record counts satisfy $\log(R + 2) = O(n)$, since $M \leq N$ and $L \leq N$.

Arithmetic. Addition, subtraction and comparison of w-bit words have circuits of size $O(w)$ and depth $O(\log w)$ by parallel prefix computation of the carries [44]. The long division of Equation (3.4) is a chain of $O(w)$ conditional subtractions, of size $O(w^2)$ and depth $O(w \log w) = O(n \log n)$. The modular correction that follows has depth $O(\log n)$.

Selection. A multiplexer tree over q words has depth $O(\log q)$. To select the first flagged record of a list of q, compute prefix ORs of the flags with a size-$O(q)$, depth-$O(\log q)$ prefix circuit [44], mark the unique record whose flag is one and whose preceding prefix OR is zero, and combine the marked record by a balanced OR tree; size $O(qw)$ and depth $O(\log q)$. This replaces the sequential scan described in the primitives.

Sorting. Batcher's networks [1] have $O(\log^2 R)$ comparator layers. A comparator on w-bit records compares keys in depth $O(\log w)$ and then swaps by multiplexers in depth $O(1)$. Each sorting network therefore has depth $O(\log^2 R \cdot \log w) = O(n^2 \log n)$. The networks of [2] have $O(\log R)$ layers, giving depth $O(n \log n)$.

Payload propagation. The scan in Lemma 2.1 assigns to each query the payload of the dictionary record at the start of its equal-key run, if there is one. Compute the run boundaries by comparing adjacent keys, in depth $O(\log w)$. Each record then carries a triple (boundary bit, dictionary bit, payload), and the required values are the prefix products of these triples under the associative operation of a segmented copy scan, which keeps the right operand when it starts a run or carries a dictionary record and otherwise keeps the left payload. This operation has a constant-depth circuit of size $O(w)$, so a prefix circuit [44] computes all values with size $O(Rw)$ and depth $O(\log R + \log w) = O(n)$. Repeated-state detection (Lemma 2.2) is a sorting network, adjacent comparisons and a first-flag selection, so it has the same depth bounds as the lookup.

Simulations. If $T < K = 2L$, the circuit consists of T copies of F in series, of depth at most $2Ld$. Otherwise the computation has one sequential spine: the trajectory Equation (3.2), of depth Kd; then the lookup and the selection of the checkpoint index d; then the second trajectory of length K, of depth Kd; then the selection of its entry $K - d$ and the final branch selection. The repetition branch runs in parallel with the lookup and the second trajectory, and consists of repeated-state detection, the residue multiplexer, the division and a selection. Summing along the spine gives depth $4Ld + O(n^2 \log n)$ with Batcher's networks and $4Ld + O(n \log n)$ with the networks of [2].

Each replacement changes the size of its component by at most a constant factor, so the sizes are those of Theorem 3.4 and of its AKS version in Section 6.4. □

For example, with $S = \{0,1\}^n$ and $L = \lceil N^{1/4} \rceil$, every T-step map of an n-bit iteration has a circuit of depth $O(N^{1/4}d + n^2 \log n)$ and size $O(N^{1/4}s + n^3 N^{3/4})$, whereas unrolling has depth

Td. More generally, for $d + n^2 \log n \le D \le Nd$ the choice $L = \Theta(D/d)$ gives depth $O(D)$ and size $O(Ds/d + n^3(D/d + Nd/D))$.

The same accounting applies to the batched construction of Theorem 6.4. Its single shared lookup has $R = O(N/L + rL)$ records with $\log(R + 2) = O(n)$, and the r trajectories, repetition tests and corrections run in parallel, so $B_{F,T,r}$ has circuits of the size in Equation (6.2) and depth $O(Ld + n^2 \log n)$. For Theorem 6.2 the m stages are composed in series, which gives depth $m \cdot O(Ld + n^2 \log n)$; we do not claim anything better for input times.

Corollary 6.10 (Cellular automata)**.** *Let G be a cellular automaton on m cells, arranged on a ring or with fixed boundary values, with finite alphabet Σ and a fixed local rule of constant radius. Encode each cell by $\lceil \log_2 |\Sigma| \rceil$ bits. For every $T \ge 0$ the T-step map of G on configurations in Σ^m has circuits of size $|\Sigma|^{m/2} \operatorname{poly}(m)$, and also circuits of depth $|\Sigma|^{m/4} \operatorname{poly}(m)$ and size $|\Sigma|^{3m/4} \operatorname{poly}(m)$. The polynomials depend on $|\Sigma|$ and the radius, but not on T.*

Proof. Let $n = m \lceil \log_2 |\Sigma| \rceil$ and let F apply the local rule to every cell, mapping invalid cell codes to a fixed symbol. Each output cell is a fixed function of a constant number of input bits, so F has size $O(m)$ and depth $O(1)$, and the set S of valid encodings, of size $|\Sigma|^m$, is forward invariant. Apply Theorem 6.9 with $M = |\Sigma|^m$, $s = O(m)$ and $n = O(m)$: choosing L as in Equation (3.6), with N replaced by M, gives size $O(m^3\sqrt{M})$, and $L = \left\lceil M^{1/4} \right\rceil$ gives depth $O(M^{1/4} + m^2 \log m)$ and size $O(mM^{1/4} + m^3 M^{3/4})$. □

For instance, Wolfram's Rule 30 [45] on a ring of m cells has, for every T, a T-step circuit of size $2^{m/2} \operatorname{poly}(m)$. Whether the exponent $1/2$ is sharp for cellular automata is not established here. We expect that a radius-one automaton simulating an in-place implementation of P_n inherits a lower bound of the form $2^{m/2}/\operatorname{poly}(m)$ from Theorem 5.4, but we do not prove it.

7 Functional-graph navigation

Write $B = A + \sqrt{n^3 NA}$. For a state x, let $\mu(x)$ be its distance to its eventual cycle, let $\ell(x)$ be that cycle's length, and let $c(x)$ be the lexicographically least state on the cycle. Define $\phi(x) \in \{0, \dots, \ell(x) - 1\}$ by

$$F^t(x) = F^{(\phi(x)+t) \bmod \ell(x)}(c(x)) \qquad (t \ge \mu(x)). \tag{7.1}$$

The phase is an eventual-orbit label; it does not assert that a transient state itself lies on the cycle. These labels are unique. Cycle lengths are encoded with $n + 1$ bits, since a cycle may have length N.

Theorem 7.1 (Cycle labels)**.** *A circuit of size $O(B)$ computes $(\ell(x), c(x), \phi(x))$ for every x.*

Proof. Fix L and use the checkpoint construction with $K = 2L$. Hardwire the three labels for each checkpoint. If $F^d(x) = h$ for $d \le K$, output

$$\ell(x) = \ell(h), \qquad c(x) = c(h), \qquad \phi(x) = (\phi(h) - d) \bmod \ell(h).$$

This follows by applying Equation (7.1) after both transients have ended. All records still have $O(n)$ bits.

If there is no checkpoint hit, the prefix repeats. Sort prefix records by state and then by original index. Among adjacent equal-state pairs, choose the pair with smallest second index j, retaining its first index i. This is the first repetition in the original trajectory: any earliest second occurrence is adjacent to its first occurrence in its equal-state run. The states before j are distinct, so $i = \mu(x)$

and $[i, j)$ consists of exactly one cycle traversal. Set $\ell = j - i$, and select the minimum state $c = x_p$ over $i \le p < j$. Then $\phi(x) = (-p) \bmod \ell$. Minimum selection and the interval mask cost $O(nK)$ gates, and word arithmetic costs $O(n^2)$ gates. Sorting and lookup cost $O(n^3(L + N/L))$, while the prefix costs $O(Ls)$. Optimizing L proves the claim. Discarded branches use safe positive dummy moduli, as in the fixed-power construction. □

Theorem 7.2 (Transient depth)**.** *The exact transient depth $\mu(x)$ is computable by a circuit of size $O(B)$.*

Proof. Put $D = N!$ and $R = F^D$. Every trajectory is cyclic after N steps, every cycle length divides D, and $D \ge N$. Thus R maps into the cyclic states, fixes each such state, and satisfies $R^2 = R$. It commutes with all iterates of F, so

$$F^i(x) = F^i(R(x)) \quad \iff \quad i \ge \mu(x). \tag{7.2}$$

Indeed, equality is equivalent to $F^i(x) = R(F^i(x))$, which is exactly cyclicity. The projection costs $O(B)$ gates by Theorem 1.1.

Fix a checkpoint parameter L and set $K = 2L$. Retain the first $K + 1$ states of both trajectories and compare corresponding states. If the states at position K agree, select the first equality; its index is $\mu(x)$. Otherwise $\mu(x) > K$, so the original prefix has no repetition and must meet a checkpoint $h = F^d(x)$ with $d < K$. Every such hit is transient and satisfies $\mu(x) = d + \mu(h)$.

Hardwire $\mu(h)$ with each checkpoint and resolve all prefix queries using Lemma 2.1. Either branch uses $O(n)$-bit records; comparing, selecting, and adding indices cost $O(nL)$ gates. The two prefixes and shared lookup cost $O(Ls + n^3(L + N/L))$. Use the same L for the projection and these operations, then optimize as in Equation (3.6). □

Lemma 7.3 (Compressed checkpoint forest)**.** *Let H be the checkpoint set from Lemma 2.3. Cut the outgoing edge of every cyclic vertex, and let S be the union of the paths from H to their first cyclic vertices, including both endpoints. There is a set J with $H \subseteq J \subseteq S$ and $|J| \le 2|H|$ such that:*

1. *Every root and every branching vertex of the forest on S lies in J.*
2. *From each nonroot $a \in J$, the next vertex of J is reached after some $1 \le d_a \le 2L$ steps. The segments $a, F(a), \ldots, F^{d_a - 1}(a)$ are pairwise disjoint.*
3. *Each $a \in J$ has labels $p(a), q(a) < N$ such that, for $a, h \in J$,*

$$p(a) \le p(h) \le q(a) \quad \iff \quad F^t(h) = a \text{ for some } 0 \le t \le \mu(h).$$

Proof. Take J to be H together with the roots and branching vertices of S. Every leaf belongs to H: any other vertex on a checkpoint path has a predecessor on that path. In a rooted forest, the number of roots plus the number of vertices with at least two children is at most the number of leaves. Indeed, summing the numbers of children gives the number of nonroot vertices; a branching vertex contributes at least two. Therefore $|J| \le |H| + |\text{leaves}| \le 2|H|$.

Apply checkpoint coverage to $F(a)$. A checkpoint hit gives a positive hit from a within $2L$ steps; a repetition gives cycle entry within L steps, where a root is marked. Hence the next marked vertex is at distance at most $2L$. Between marks, each vertex of S has a unique predecessor in S, so two such segments cannot merge. Finally order vertices by a preorder traversal of the cut forest. Descendants of a vertex occupy a contiguous interval; its first and last ranks give p, q. Only the labels of J need to be stored. The Lean proof obtains the same interval property by lexicographically ordering root-to-vertex paths, without formalizing a traversal algorithm. □

Theorem 7.4 (Reachability and first hitting times)**.** *A circuit of size $O(B)$, on input (x, y), decides whether $F^t(x) = y$ for some $t \geq 0$ and returns the least such t when it exists. The output may use a separate existence flag and n time bits.*

Proof. Use a common parameter L in the depth and cycle-label constructions. Write $K = 2L$ and construct the forest of Lemma 7.3. Store a dictionary of the marked vertices $a \in J$, with their states, depths, and interval labels. A second dictionary has one key $F^{d_a-1}(a)$ for each nonroot mark a, with the same payload for a. These last-predecessor keys are distinct by disjointness of the segments. Both dictionaries have $O(N/L)$ records of $O(n)$ bits.

Suppose first that y is transient. Generate the prefixes of x and y through time K. If y occurs in the prefix of x, accept. Otherwise select any marked vertex h in that prefix; if none exists, reject. To select a candidate ancestor a, use $a = y$ if $y \in J$. Otherwise query the second dictionary on the prefix of y, retain only matches with $\mu(a) \geq \mu(y)$, and select any retained record. Reject if there is no such record. Finally accept exactly when

$$y \in \{a, F(a), \ldots, F^K(a)\} \quad \text{and} \quad p(a) \leq p(h) \leq q(a).$$

The first test uses a third short prefix. It is essential for rejecting side branches outside S.

Every acceptance is sound: x reaches h, the interval test says that h reaches a, and the verification prefix reaches y. Conversely, suppose x reaches a transient y beyond its inspected prefix. That prefix cannot repeat and must contain a checkpoint. Every selected mark h precedes y, so $y \in S$. If y is unmarked, it lies in a unique compressed segment, whose last-predecessor key appears within K steps of y. Its lower endpoint passes the depth filter. Moreover, every retained key has the same property: if $F^j(y) = F^{d_a-1}(a)$ and $\mu(a) \geq \mu(y)$, decreasing depth gives $j \leq d_a - 1$. Unique predecessors inside the segment then give $y = F^{d_a-1-j}(a)$. A marked h can reach this segment only through a. Thus the selected record passes both final tests. The marked-target case takes $a = y$ directly.

Three prefixes cost $O(Ls)$ gates. Shared lookup costs $O(n^3(L + N/L))$, and depth filtering, record selection, prefix equality, and interval comparisons add $O(nL + n^3)$ gates. Depths and cycle labels fit the same tradeoff. Optimization therefore gives $O(B)$ gates. For a positive transient query, the first hitting time is simply $\mu(x) - \mu(y)$, since depth decreases by one on each transient step.

If y is cyclic, it is reachable exactly when $c(x) = c(y)$. On a positive instance its first hitting time is

$$\mu(x) + \big((\phi(y) - \phi(x) - \mu(x)) \bmod \ell(x)\big),$$

where the parenthesized residue lies in $[0, \ell(x) - 1]$. This is the first time on or after entry into the cycle with the required phase. Finally $\mu(x) + \ell(x) \leq N$, so every first hitting time is at most $N - 1$. Arithmetic may use $n + 1$ intermediate bits. The time field is unspecified on negative instances; an additional $O(n)$-gate mask can set it to zero if desired. □

Corollary 7.5 (Complete visit schedules)**.** *A circuit of size $O(B)$, on input (x, y), returns a success flag and two integers d, p describing every visit time. A zero flag denotes no visits. For a positive flag, $d < N$ is the first visit, $0 \leq p \leq N$, and*

$$\{t \geq 0 : F^t(x) = y\} = \{d + kp : k \geq 0\}.$$

Here $p = 0$ for transient y, giving the singleton $\{d\}$, and $p = \ell(y)$ for cyclic y. The period field uses $n + 1$ bits.

Proof. Use Theorem 7.4 for the flag and d, and Theorem 7.1 and an $O(B)$ cyclicity test for p. After the first visit, a later time $t \geq d$ is a visit exactly when $F^{t-d}(y) = y$. Return times of a cyclic point are precisely the multiples of its least positive period. A transient point has no positive return. This proves the descriptor. □

For example, a visit occurs by deadline T exactly when the flag is positive and $d \leq T$. The number of visits at times $0, \ldots, T$ is

$$\begin{cases} 0, & \text{flag zero or } T < d, \\ 1, & \text{flag positive, } T \geq d,\ p = 0, \\ 1 + \lfloor (T-d)/p \rfloor, & \text{flag positive, } T \geq d,\ p > 0. \end{cases}$$

These are arithmetic consequences of the returned descriptor; the descriptor bound does not include a circuit for processing an additional deadline input of unspecified length.

Relation to navigation data structures. Function powers and path queries have established data-structure formulations [15, 21]. In particular, nearest labeled successors include reachability of a fixed singleton target. Those results charge representation space and query time in a RAM model. Here the given object is a succinct transition circuit, and every gate accessing a nonuniform table is charged. These differences specify the claim being compared; they do not establish historical novelty of the present bounds.

7.1 Cycle spectra and cycle-length transfer

The fixed-power construction stores one payload per checkpoint. The following decomposition shows where the information about T actually resides: in one residue per distinct cycle length. Write ℓ_F for the map $x \mapsto \ell(x)$, with $(n+1)$-bit outputs, let $c(F)$ be the number of distinct cycle lengths of F, let $\lambda(F)$ be their least common multiple, and let $\mu(F) = \max_x \mu(x)$. Since distinct cycle lengths sum to at most N, always $c(F) \leq \sqrt{2N}$. The semantic identities of this subsection are checked in Lean, in `Proofs/Spectrum.lean`: the reduction $F^T(x) = F^{N+((T-N) \bmod \ell(x))}(x)$ for $T \geq N$ (`iterate_eq_card_add_mod`) and the bound $N + ((T-N) \bmod \ell(x)) < 2N$ (`card_add_mod_lt`); the spectrum test for maps and for permutations (`cycle_test_iff`, `cycleSpectrum_test`, `injective_spectrum_test`); and $P^{-1}(x) = P^{\ell(x)-1}(x)$ (`perm_symm_eq_iterate`). The squaring bound is the mechanized binary-powering construction behind `variable_time`. The gate counts below, the prime-power test, the order bound and the factoring example remain prose results outside the Lean formalization.

Theorem 7.6 (Spectrum decomposition)**.** *For every $T \geq 0$,*

$$\mathrm{C}(F^T) \leq \mathrm{C}(\ell_F) + \mathrm{C}(\mathrm{Iter}_{F,n+1}) + O(c(F)\, n).$$

Proof. If $T < N$, hardwire T as the $(n+1)$-bit time input of a circuit for $\mathrm{Iter}_{F,n+1}$. Otherwise let $\ell_1 < \cdots < \ell_c$ be the distinct cycle lengths and hardwire $r_i = (T-N) \bmod \ell_i$. Compute $\ell = \ell(x)$, the flags $e_i = [\ell = \ell_i]$, and $r = \bigvee_i e_i \wedge r_i$; exactly one flag is set, so $r = (T-N) \bmod \ell(x)$, and this costs $O(cn)$ gates. Since $r < \ell \leq N$, the integer $u = N + r < 2N$ is obtained by setting bit n above the n bits of r, with no gates. Both T and u are at least $N \geq \mu(x)$ and are congruent modulo $\ell(x)$, so $F^T(x) = F^u(x) = \mathrm{Iter}_{F,n+1}(x, u)$. □

Combined with Theorems 6.2 and 7.1 the right-hand side is $O(nB)$, so the decomposition is not a better general bound. Its content is structural: all dependence on T is confined to the $c(F)$-entry table $\ell \mapsto T \bmod \ell$. For P_n, $\ell_{P_n}(a,b)$ is a on active states and 1 otherwise, which costs $O(n)$ gates; Equation (5.9) gives $\mathrm{C}(\mathrm{Iter}_{P_n,n+1}) = O(n^2)$; and $c(P_n) = B$. Hence

$$\max_{T \geq 0} \mathrm{C}(P_n^T) = O\Big(n\, 2^{n/2}\Big),$$

a factor n^2 below the bound from Theorem 1.1, and within $O(n^2)$ of Theorem 5.4. A finer treatment of the residue table determines the leading constant; we do not pursue it here.

The next lemma relates the four quantities that the decomposition involves. Write $\Omega(\lambda)$ for the number of prime factors of λ counted with multiplicity, so $\Omega(\lambda) \leq \log_2 \lambda$.

Lemma 7.7 (Transfer lemmas)**.** *Let $\ell_1 < \cdots < \ell_c$ be the distinct cycle lengths of F and $\lambda = \lambda(F) = \prod_p p^{e_p}$.*

1. *For every $m \geq 1$, $\mathrm{C}(\mathrm{Iter}_{F,m}) \leq \sum_{j<m} \mathrm{C}(F^{2^j}) + O(mn)$.*
2. *$\mathrm{C}(\ell_F) \leq (c+1)\,\mathrm{C}(\mathrm{Iter}_{F,n+1}) + O(cn)$. For a permutation the factor $c+1$ may be replaced by c.*
3. *$\mathrm{C}(\ell_F) \leq \mathrm{C}(F^N) + \Omega(\lambda)\big(\max_{d\mid\lambda} \mathrm{C}(F^d) + O(n^2)\big)$. For a permutation the term $\mathrm{C}(F^N)$ may be omitted.*
4. *If $\mathrm{C}(F^T) \leq s$ for every $T \geq 0$, with $s \geq n$, then $\lambda(F) \leq 2^{O(s \log s)}$.*

Proof. Part 1 is the proof of Theorem 6.2, with the circuits for F^{2^j} taken optimal.

For the other parts, put $y = F^N(x)$. It is cyclic and $\ell(y) = \ell(x)$, and a cyclic state satisfies $F^d(y) = y$ exactly when $\ell(y) \mid d$. For a permutation every state is cyclic and we may take $y = x$.

Part 2. Compute $y = \mathrm{Iter}_{F,n+1}(x, N)$ and the flags $g_i = [\mathrm{Iter}_{F,n+1}(y, \ell_i) = y]$; all time inputs are at most N. Then $g_i = 1$ exactly when $\ell(y) \mid \ell_i$. Since $\ell(y)$ is itself one of the ℓ_i, and every multiple of it in the list is at least as large, the least ℓ_i with $g_i = 1$ is $\ell(x)$. Comparisons and the priority selection cost $O(cn)$ gates.

Part 3. For a prime $p \mid \lambda$ and $0 \leq f \leq e_p$, put $d_{p,f} = \lambda / p^{e_p - f}$. Its q-adic valuation is e_q for $q \neq p$ and f for $q = p$. Since $\ell(y) \mid \lambda$, the divisibility $\ell(y) \mid d_{p,f}$ holds exactly when $v_p(\ell(y)) \leq f$. Hence

$$v_p(\ell(x)) = \min\big(\{f < e_p : F^{d_{p,f}}(y) = y\} \cup \{e_p\}\big).$$

This uses $\sum_p e_p = \Omega(\lambda)$ fixed powers applied to y, each followed by an n-bit equality test. Select the hardwired constant $p^{v_p(\ell(x))}$ among $e_p + 1$ candidates and multiply it into a running product. Every partial product divides $\ell(x) \leq N$, so each multiplication has $(n+1)$-bit operands and costs $O(n^2)$ gates. There are at most $\Omega(\lambda)$ primes.

Part 4. Put $\mu = \mu(F)$. For $\mu \leq a < b < \mu + \lambda$ the difference $b - a$ is not divisible by λ, so some cycle length ℓ does not divide it. At a state y on such a cycle, $F^a(y) \neq F^b(y)$. Thus $F^\mu, \ldots, F^{\mu+\lambda-1}$ are λ distinct maps of complexity at most s, and Lemma 5.3 with $q = s \geq n$ gives $\lambda \leq (s+1)16^s(n+s+2)^{2s+n} = 2^{O(s \log s)}$. □

Call a function of a family *easy* if it has circuits of size polynomial in n.

Theorem 7.8 (Which maps have only easy powers)**.** *Let F_n be maps with $\mathrm{C}(F_n) = n^{O(1)}$.*

1. *If every F_n is a permutation, then $\max_T \mathrm{C}(F_n^T) = n^{O(1)}$ if and only if $\mathrm{Iter}_{F_n,n}$ and ℓ_{F_n} are easy and $\log_2 \lambda(F_n) = n^{O(1)}$. In particular, if all powers are easy then the cycle length is easy.*

2. *If* $c(F_n) = n^{O(1)}$*, then* $\max_T \mathrm{C}(F_n^T)$ *and* $\mathrm{C}(\mathrm{Iter}_{F_n,n+1})$ *are polynomially related:*

$$\mathrm{C}(\mathrm{Iter}_{F,n+1}) \le (n+1)\max_T \mathrm{C}(F^T) + O(n^2), \qquad \max_T \mathrm{C}(F^T) \le (c+2)\,\mathrm{C}(\mathrm{Iter}_{F,n+1}) + O(cn).$$

3. *For a permutation* P*,* $\mathrm{C}(P^{-1}) \le \mathrm{C}(\ell_P) + \mathrm{C}(\mathrm{Iter}_{P,n}) + O(n)$*.*

Proof. Part 1, forward direction: part 1 of Lemma 7.7 with $m = n$ bounds the iterator, part 4 bounds $\log_2 \lambda$ by $O(s \log s)$, and part 3 then bounds $\mathrm{C}(\ell_F)$ by $\log_2 \lambda$ times a polynomial. Reverse direction: hardwire $T' = T \bmod \lambda$, which has $n^{O(1)}$ bits. Since $\ell(x) \mid \lambda$, we have $T \bmod \ell(x) = T' \bmod \ell(x)$. Compute this remainder by streaming the bits of T' as in Theorem 6.3, at $O(n)$ gates per bit. The remainder is at most $N-1$, so it fits the n-bit time input of $\mathrm{Iter}_{F,n}$, and every state of a permutation is cyclic, so $F^T(x) = \mathrm{Iter}_{F,n}(x, T' \bmod \ell(x))$.

Part 2: the first inequality is part 1 of Lemma 7.7, and the second combines Theorem 7.6 with part 2 of Lemma 7.7.

Part 3: $P^{-1}(x) = P^{\ell(x)-1}(x)$, and $\ell(x) - 1 \le N - 1$ has n bits. □

More generally, a permutation with easy ℓ and an easy iterator has easy T-th power whenever the residues $\ell \mapsto T \bmod \ell$ on its spectrum are computed by a small circuit. Part 1 of Theorem 7.8 fails without the bound on λ: for P_n the cycle length and the iterator are easy, but some powers are hard. Conversely, the cycle length can be hard while the map, its inverse, and its iterator are all easy.

Proposition 7.9 (Hard cycle length, easy iterator and inverse)**.** *For* $w \ge 2$ *define* P *on triples* (M, a, x) *of* w*-bit integers, so* $n = 3w$*, by*

$$P(M,a,x) = \begin{cases} (M, a, ax \bmod M), & M \ge 2,\ a, x < M,\ \gcd(a, M) = \gcd(x, M) = 1, \\ (M, a, x), & \textit{otherwise.} \end{cases}$$

Then P *is a permutation, and* P*,* P^{-1}*, and* $\mathrm{Iter}_{P,m}$ *have circuits of size* $\mathrm{poly}(w)$ *and* $\mathrm{poly}(w, m)$*. If* ℓ_P *has circuits of size* q*, then a circuit of size* $O(w)(q + \mathrm{poly}(w))$ *outputs a nontrivial factor of every composite* w*-bit integer. Consequently, if no polynomial-size circuits find a nontrivial factor of every composite* w*-bit integer, then* ℓ_P *requires superpolynomial circuits.*

Proof. For fixed valid (M, a), multiplication by the unit a permutes the units below M, and invalid triples are fixed; so P is a permutation. Validity tests use gcd circuits, and modular products cost $\mathrm{poly}(w)$ gates. The inverse multiplies by $a^{-1} \bmod M$, computed by an extended-Euclid circuit. The iterator returns $(M, a, a^t x \bmod M)$ on valid triples, with $a^t \bmod M$ computed by square-and-multiply over the m bits of t. On a valid triple, $a^j x \equiv x$ exactly when $a^j \equiv 1$, so $\ell_P(M, a, x) = \mathrm{ord}_M(a)$.

Now let M be composite. If M is even, output 2. If $M = r^e$ with $e \ge 2$, output r; integer e-th roots for all $e \le w$ cost $\mathrm{poly}(w)$ gates. Otherwise M is odd with at least two distinct prime factors. Draw a uniformly from $[1, M-1]$ by rejection sampling on the bit length of M. If $\gcd(a, M) > 1$, output it. Otherwise put $r = \ell_P(M, a, 1) = \mathrm{ord}_M(a)$. If r is even and $a^{r/2} \not\equiv -1 \pmod M$, then $\gcd(a^{r/2} - 1, M)$ is a nontrivial factor, and for such M this happens with probability at least $1/2$ over a uniform unit a [41, 42]. One trial therefore succeeds with probability at least $1/4$, and $O(w)$ independent trials fail with probability below 2^{-w-1}. A union bound over all w-bit M and Adleman's argument [39] fix the random bits. Each trial uses one copy of the circuit for ℓ_P and $\mathrm{poly}(w)$ further gates. □

By Theorem 7.8, the permutation of Proposition 7.9 then has superpolynomially many distinct cycle lengths and some hard fixed powers, under the same hypothesis. The hardness obtained is only superpolynomial, since factoring has circuits of size $2^{\tilde{O}(w^{1/3})}$. Whether the cycle length of an explicit polynomial-size permutation can be hard without number-theoretic assumptions is a question about space-bounded computation; we do not pursue it here.

7.2 Recognizable targets and terminating computations

Corollary 7.10 (First recognizable target)**.** *Let $P : \{0,1\}^n \to \{0,1\}$ have circuit size a, and put $A_P = s + a + n^3$. Eventual target membership,*

$$H(x) = 1 \quad \Longleftrightarrow \quad \exists t \geq 0 : \ P(F^t(x)) = 1,$$

has a circuit of size $O(A_P + \sqrt{n^3 N A_P})$. Within the same bound, a circuit returns the first visited target state and its exact first visit time on positive instances.

Proof. Set $G(x) = x$ when $P(x) = 1$ and $G(x) = F(x)$ otherwise. This costs $O(s + a + n)$ gates. A first target, if one exists, is visited before step N, since an earlier repeated state would preclude a later first visit. Hence $G^N(x)$ is the first target state on positive instances, and $H(x) = P(G^N(x))$. Apply Theorem 1.1 to G. For the time, apply Theorem 7.2 to G. On a positive instance, the first target is an absorbing fixed point. A cyclic state cannot precede it on the trajectory: iteration is injective on cyclic states, so any cyclic predecessor of that fixed point is the same point. Consequently $\mu_G(x)$ is exactly the first target time. □

Corollary 7.11 (Complete-configuration simulation)**.** *Suppose a deterministic computation has its entire configuration encoded in b bits and a next-configuration circuit of size s. If halting configurations are absorbing and every valid initial configuration halts, its final-configuration map on valid initial states has circuit size*

$$O\left(s + b^3 + \sqrt{b^3 2^b (s + b^3)}\right).$$

The exact first halting time can also be returned within this bound.

Proof. Before halting, no configuration can repeat. The iterate at time 2^b is therefore the final configuration. Apply Theorem 1.1 with state width b. The first halt is the transient depth, by the same absorbing-state argument as in Corollary 7.10; use Theorem 7.2. Initialization and output decoding circuits are charged separately. □

In particular, when $s = \mathrm{poly}(b)$, both the final configuration and the first halting time have circuit size

$$2^{b/2}\,\mathrm{poly}(b).$$

For a nearly in-place computation on an n-bit input, with complete state width $b = n + O(\log n)$ and a polynomial-size transition, this specializes to $2^{n/2}\,\mathrm{poly}(n)$. Polynomial-size initialization and output decoding preserve that bound. The parameter b includes all persistent input information, control, and workspace; these bits are counted even when an abstract machine model calls part of its storage read-only. A general polynomial-space machine can have far more than $n + O(\log n)$ configuration bits, so the corollary gives no such input-length bound for all of PSPACE. For machines whose only storage is the input tape, Section 7.3 replaces the width b by the number of reachable configurations.

7.3 In-place Turing machines

Corollary 7.11 charges the complete configuration width b. For a Turing machine whose only storage is its input tape, the configurations actually visited can be far fewer than 2^b: a ternary tape alphabet already needs $b \geq 2n$ bits, while only $3^n \operatorname{poly}(n)$ configurations exist. Combining a locality bound for the step map with Theorem 3.4 on the reachable configurations gives bounds in terms of the number of reachable configurations, also for machines that need not halt, and the exponent is attained by an explicit two-head binary machine. The Lean module `Proofs/InPlace.lean` defines the configurations and the step map F_M of the model below and checks the counts of configurations and of reachable configurations used in Theorem 7.13 (`InPlace.card_config`, `reach_card_le_configs`, `reach_card_le_time`), the stabilization argument of Corollary 7.14 (`iterate_card_halting`), and the application of Theorem 3.4 to the reachable set, precomposed with an initialization circuit (`inplace_fixed_power_optimized`, `inplace_halting`). These circuit theorems take as hypotheses a Boolean encoding of configurations, injective on the reachable set, a circuit for F_M on that encoding, and a circuit for $x \mapsto \iota(x)$; Lemma 7.12, the width Equation (7.3) and the asymptotic simplifications are not mechanized. For Proposition 7.15, Lean checks items 2 and 3 for every machine that satisfies item 1 and whose encoding exposes the tape of an initial configuration by wiring (`reach_card_le_round`, `round_bounds`); the machine M^* and item 1 are prose.

7.3.1 Model and step circuit

An *in-place machine* $M = (Q, \Gamma, h, \delta, q_0, Q_{\text{halt}})$ has one tape whose cells are exactly the n input cells, a finite tape alphabet $\Gamma \supseteq \{0,1\}$, and $h \geq 1$ heads on that tape. In one step the control reads the state q, the h symbols under the heads, and for each head whether it is on cell 1 and whether it is on cell n. The transition function δ returns a new state, a symbol to write under each head, and a move in $\{-1, 0, +1\}$ for each head; a move past either end leaves the head in place. When several heads are on the same cell, the head with the largest index determines the written symbol. Halting states are absorbing: on them δ writes the symbols read and does not move or change state. Nothing is assumed about running time.

A configuration is $(w, p_1, \ldots, p_h, q)$ with $w \in \Gamma^n$, $p_j \in [n]$, and $q \in Q$. Encode it cell by cell in

$$c = c_M(n) = n \lceil \log |\Gamma| \rceil + h \lceil \log n \rceil + \lceil \log |Q| \rceil \tag{7.3}$$

bits, so that there are $|\Gamma|^n \, n^h \, |Q|$ valid encodings. Let $F_M : \{0,1\}^c \to \{0,1\}^c$ be the one-step map, extended to invalid encodings by the circuit below; valid encodings are mapped to valid encodings. An input $x \in \{0,1\}^n$ is identified with the initial configuration $\iota(x) = (x, 1, \ldots, 1, q_0)$; the map ι only inserts constants. Let $\operatorname{Reach}_M(n)$ be the number of configurations $F_M^t(\iota(x))$ with $t \geq 0$ and $x \in \{0,1\}^n$. Write $\mathrm{C}(\delta)$ for the circuit size of δ on its $O(\log |Q| + h \log |\Gamma| + h)$ input bits.

Lemma 7.12 (Step circuit)**.** $\mathrm{C}(F_M) = O(hn + \mathrm{C}(\delta))$*, where the constant depends only on* $|\Gamma|$*.*

Proof. Decode each head position into a one-hot vector $e_1^{(j)}, \ldots, e_n^{(j)}$ with $O(n)$ gates. The symbol under head j is $\bigvee_i (e_i^{(j)} \wedge w_i)$, computed bitwise in $O(n)$ gates, and the two end flags are $e_1^{(j)}$ and $e_n^{(j)}$. One copy of a circuit for δ produces the new state, the written symbols $\sigma'_1, \ldots, \sigma'_h$, and the moves. The new content of cell i is obtained from w_i by h successive multiplexers, the jth selecting σ'_j when $e_i^{(j)} = 1$, so that the largest index wins; this costs $O(h)$ gates per cell. Each head update is a clamped increment or decrement of a $\lceil \log n \rceil$-bit word, $O(\log n)$ gates. □

This is the only place where the structure of Turing machines is used. A cell changes only if a head is on it, which is why the step map is small even though it acts on $\Theta(n)$ bits.

7.3.2 Circuit bounds

Theorem 7.13 (In-place machines)**.** *Let M be an in-place machine, let c be as in Equation* (7.3)*, and put $A_M = hn + \mathrm{C}(\delta) + c^3$ and $R = \mathrm{Reach}_M(n)$. For every $T \geq 0$ the map $x \mapsto F_M^T(\iota(x))$ on $\{0,1\}^n$ has a circuit of size*

$$O\Big(A_M + c^{3/2}\sqrt{R\,A_M}\Big)\,.$$

Always $R \leq |\Gamma|^n n^h |Q|$. If M halts within $t(n)$ steps on every input in $\{0,1\}^n$, then also $R \leq 2^n(t(n)+1)$.

Proof. The set S of configurations reachable from $\iota(\{0,1\}^n)$ is forward invariant under F_M and has $|S| = R$. Apply Theorem 3.4 to F_M on c bits, with $s = O(hn + \mathrm{C}(\delta))$ by Lemma 7.12, and choose $L = \max\{1, \left\lceil \sqrt{c^3 R / A_M} \right\rceil\}$ as in Equation (3.6), with N replaced by R; then $1 \leq L \leq \left\lceil \sqrt{R} \right\rceil \leq 2^c$. Precompose with ι. Every valid configuration is counted in the first bound on R. Under the time bound, each trajectory from $\iota(x)$ visits at most $t(n)+1$ configurations, because halting configurations are absorbing. □

For constant h, $|Q|$ and $|\Gamma|$ we have $c = O(n)$ and $A_M = O(n^3)$, so the two bounds on R give

$$O\Big(n^{3+h/2}\,|\Gamma|^{n/2}\Big) \qquad \text{and} \qquad O\Big(n^3\,2^{n/2}\sqrt{t(n)+1}\Big)\,. \tag{7.4}$$

The circuit depends on M, n and T; the bound does not depend on T.

Corollary 7.14 (Predicates decided in place)**.** *Let M be an in-place machine with constant h, $|Q|$ and $|\Gamma|$, not necessarily halting. Whether M halts on x, whether it accepts x, and its final tape when it halts (arbitrary otherwise) all have circuits of size $O(n^{3+h/2}|\Gamma|^{n/2})$. In particular, over a binary tape alphabet these functions have circuits of size $2^{n/2+O(\log n)}$, and over a ternary alphabet of size $2^{0.7925n+O(\log n)}$.*

Proof. Let $T^* = |\Gamma|^n n^h |Q|$, the number of valid configurations. A run that halts does so within $T^* - 1$ steps, since the configurations before halting are distinct. A run that has not halted after T^* steps has repeated a configuration; it then cycles forever, and the cycle contains no halting configuration, because a halting configuration on a cycle is a fixed point. So $F_M^{T^*}(\iota(x))$ is a halting configuration exactly when M halts on x, and it is then the final configuration. Apply Theorem 7.13 with $T = T^*$ and read off the state and the tape. Finally $3^{1/2} = 2^{0.7925\ldots}$. □

Unlike Corollary 7.11, the bound is governed by the reachable configurations rather than by the encoding width: for a ternary alphabet $c \geq 2n$, so the width bound would give nothing below the truth table. For $|\Gamma| \geq 4$ the alphabet bound is no better than the $O(2^n/n)$ truth-table bound, and the reachable-set bound is the one to use. The time bound in Equation (7.4) beats direct unrolling, which costs $O(n\,t(n))$ gates, only when $t(n) \geq 2^n \operatorname{poly}(n)$. A machine with a separate read-only input and S work cells has circuits of size $2^{O(S)} \operatorname{poly}(n)$ by repeated squaring of its configuration graph; this is better than the present bounds whenever $S = o(n)$, and the bounds here are specific to the regime in which the input tape is the work space.

Several heads. The heads cost the factor $n^{h/2}$ in the alphabet bound. For a binary alphabet, $2^{n/2}n^{h/2}\operatorname{poly}(n)$ is below the truth-table bound $2^n/n$ exactly when $h \log n \leq n(1 - o(1))$, that is, for $h = o(n/\log n)$ heads. With $h = \Theta(n)$ heads the bound is vacuous, as it should be: n heads can hold n bits of information outside the tape. The model is in-place in the tape, not in the heads.

7.3.3 Sharpness

The permutation P_n of Equation (5.1) can be run in place, and the counting lower bound of Theorem 5.4 then transfers to the machine.

Proposition 7.15 (A two-head binary machine with a hard power)**.** *There is an in-place machine M^* with two heads, tape alphabet $\{0,1\}$ and a constant number of states, and for each even $n \geq 2$ an integer $\tau = \tau(n) = \Theta(n)$, such that:*

1. *for every $x \in \{0,1\}^n$, $F^{\tau}_{M^*}(\iota(x)) = \iota(P_n(x))$;*
2. $\mathrm{Reach}_{M^*}(n) \leq 2^n \tau$*;*
3. *for all sufficiently large even n,*

$$\Omega\left(\frac{2^{n/2}}{n}\right) \leq \max_{T \geq 0} \mathrm{C}(x \mapsto F^{T}_{M^*}(\iota(x))) \leq O\left(n^{7/2} 2^{n/2}\right).$$

Proof. Let $n = 2k$. The input is $x = (a, b)$ with a in cells $1, \ldots, k$ and b in cells $k+1, \ldots, 2k$, most significant bit first. The state consists of a phase, a parity bit, and three flags *lt*, *diff* and *carry*. We design δ so that the moves of both heads and the changes of phase and parity depend only on the phase, the parity, and the end flags, never on symbols or flags; symbols and flags influence only what is written and how the flags are updated. Consequently the sequence of head positions, phases and parities is the same for every input, and a round has an input-independent length τ. Head 2 never changes a symbol (it always writes what it reads), and head 1 changes symbols only in phases 3 and 5, during which it is on cells $k+1, \ldots, 2k$ and head 2 is on cells $1, \ldots, k$; so the write-conflict rule never matters. Each phase below processes the cells under the heads in every step, and moves on to the next phase in the step in which its stopping condition is read.

1. *Midpoint.* Both heads start on cell 1. In each step head 2 moves right and head 1 moves right when the parity bit is 0; the parity bit flips. Stop when head 2 is on cell $2k$. This takes $2k - 1$ moves of head 2, during which head 1 moves k times, so it stops on cell $k + 1$.
2. *Rewind head 2.* Move head 2 left until it is on cell 1, in $2k - 1$ moves. Set $lt = diff = 0$.
3. *Compare.* Heads 2 and 1 start on cells 1 and $k + 1$ and move right in lockstep, stopping when head 1 is on cell $2k$, so that they read the pairs (a_i, b_i) for $i = 1, \ldots, k$. If $diff = 0$ and $a_i \neq b_i$, set $diff = 1$ and $lt = a_i$. At the end, $lt = 1$ exactly when $b < a$, since the first differing bit decides the comparison. Set $carry = 1$ and $diff = 0$.
4. *Increment.* Move both heads left in lockstep from cells $(k, 2k)$, stopping when head 2 is on cell 1, so that head 1 visits cells $2k, 2k-1, \ldots, k+1$. At each cell head 1 writes $b_i \oplus carry$ if $lt = 1$ and b_i otherwise, and *carry* becomes $b_i \wedge carry$. If $lt = 1$ the block now holds $b + 1$; there is no overflow because $b + 1 \leq a \leq 2^k - 1$.
5. *Compare again.* As in phase 3, the heads move right from cells 1 and $k + 1$ to cells k and $2k$; set $diff = 1$ at the first i with $a_i \neq b'_i$, where b' is the current second block, and leave *lt* unchanged. At the end, $diff = 0$ exactly when $b' = a$.
6. *Zero.* As in phase 4, head 1 visits cells $2k, \ldots, k + 1$ and writes 0 if $lt = 1$ and $diff = 0$, and rewrites the symbol read otherwise.
7. *Return.* Move head 1 left until it is on cell 1, in k moves; head 2 is already there. Clear the flags and enter q_0.

Counting the stopping step of each phase, the round takes $\tau = 9k + O(1) = \Theta(n)$ steps, the same for every input. If $b \geq a$, which includes every state with $a = 0$, then $lt = 0$ and nothing is changed. If $b < a$, the second block becomes $b + 1$, and then 0 exactly when $b + 1 = a$; that is, it becomes $(b + 1) \bmod a$. The first block is never changed. Comparing with Equation (5.1), the tape after one round is $P_n(x)$, both heads are on cell 1, and the state is q_0, which is item 1.

For item 2, item 1 shows that every configuration reachable from $\iota(x)$ has the form $F_{M^*}^j(\iota(y))$ with $y = P_n^i(x)$ and $0 \leq j < \tau$. There are at most $2^n\tau$ such configurations.

For item 3, item 1 gives $F_{M^*}^{\tau T}(\iota(x)) = \iota(P_n^T(x))$. The tape bits of this configuration are $P_n^T(x)$, so a circuit for $x \mapsto F_{M^*}^{\tau T}(\iota(x))$ computes P_n^T as a projection, and Theorem 5.4 supplies T with $\mathrm{C}(P_n^T) = \Omega(2^{n/2}/n)$. For the upper bound apply Theorem 7.13 with $h = 2$, $|\Gamma| = 2$, $\mathrm{C}(\delta) = O(1)$, $c = n + O(\log n)$ and $R \leq 2^n\tau = O(n2^n)$: $O(c^{3/2}\sqrt{RA_M}) = O(n^{3/2} \cdot n^{1/2}2^{n/2} \cdot n^{3/2}) = O(n^{7/2}2^{n/2})$. □

The alphabet bound of Theorem 7.13 gives $O(n^4 2^{n/2})$ here; the reachable-set bound is slightly better. Both match the lower bound in the exponent. With a single head the same permutation can be run in place using a second marker track, that is, with $\Gamma = \{0,1\}^2$ and $\tau = \Theta(n^2)$; we omit the details. The alphabet bound is then $2^n \operatorname{poly}(n)$, and only the reachable-set bound $\sqrt{2^n\tau}\operatorname{poly}(n) = 2^{n/2}\operatorname{poly}(n)$ matches the lower bound.

As for P_n itself, the hard exponents of M^* are long, and its iterator with a short time input is easy. For an m-bit t write $t = \tau T + \rho$ with $0 \leq \rho < \tau$; dividing by the fixed integer τ costs $O(m \log n)$ gates. Then $F_{M^*}^t(\iota(x)) = F_{M^*}^{\rho}(\iota(P_n^T(x)))$: compute $P_n^T(x)$ with the iterator of Equation (5.9), generate the τ configurations of the next round with τ copies of F_{M^*}, and select entry ρ. This costs $\operatorname{poly}(n, m)$ gates, so every T witnessing the lower bound in item 3 has $2^{\Omega(n)}$ bits, exactly as in Corollary 5.12.

Everything in this subsection is nonuniform. The checkpoint payloads $F_M^{T-K}(h)$ may take time proportional to T to compute, and no bound for a specific, efficiently described T is obtained.

7.4 Synchronous and recurring queries

The projection $R = F^{N!}$ from the proof of Theorem 7.2 also supplies queries about the eventual dynamics.

Corollary 7.16 (Eventual synchronous coalescence)**.** *A circuit of size $O(B)$ decides, on input (x, y), whether there exists $t \geq 0$ such that $F^t(x) = F^t(y)$.*

Proof. The required equivalence is

$$\exists t \geq 0 :\ F^t(x) = F^t(y) \quad \Longleftrightarrow \quad F^N(x) = F^N(y).$$

Equality before time N persists to time N. Both states are cyclic by that time, and F restricts to a bijection on the cyclic states. Therefore later equality can be canceled back to time N. Compute two copies of F^N and compare their outputs. □

This query requires the same elapsed time on both trajectories. Orbit intersection permits different elapsed times and is a different predicate; for example, distinct points of one cycle never coalesce synchronously.

Corollary 7.17 (Recurring and permanent targets)**.** *For a fixed predicate $P : \{0,1\}^n \to \{0,1\}$ of circuit size a, each of*

$$\forall u\ \exists t \geq u :\ P(F^t(x)) = 1, \qquad \exists u\ \forall t \geq u :\ P(F^t(x)) = 1$$

is decidable by a circuit of size $O(A_P + \sqrt{n^3 N A_P})$, where $A_P = s + a + n^3$.

Proof. A target recurs arbitrarily late exactly when the eventual cycle contains an accepting state. Start at $R(x)$ and use the absorbing-target construction in Corollary 7.10. If G makes accepting states absorbing, the circuit outputs $P(G^N(R(x)))$. A successful visit is periodic and hence recurs arbitrarily late; every sufficiently late visit from x is a future visit from $R(x)$. The projection and the target circuit each obey the stated bound. Permanent eventual acceptance is the complement of recurrence of the predicate $1 - P$, which changes the bound only by a constant number of gates. □

7.5 Short exponents and polynomial space

The hard powers of Section 5 have exponentially long exponents (Corollary 5.12). For exponents with polynomially many bits, the question changes character: Corollary 7.11 has a vacuous bound when the configuration width is polynomial rather than $n + O(\log n)$. This subsection shows that hardness of short powers is exactly a circuit lower bound for PSPACE. None of its results is covered by the Lean formalization.

Families are indexed by the state width. A family of functions (G_n) is *superpolynomially hard* if for every k its complexity $\mathrm{C}(G_n)$ exceeds n^k for infinitely many n.

Theorem 7.18 (Short exponents and PSPACE)**.** *The following are equivalent.*

(i) $\mathsf{PSPACE} \not\subseteq \mathsf{P/poly}$.

(ii) There are maps F_n with $\mathrm{C}(F_n) = O(n)$ and exponents $T_n \le 2^n$ such that $(F_n^{T_n})$ is superpolynomially hard.

(iii) There are maps F_n with $\mathrm{C}(F_n) = n^{O(1)}$ and exponents $T_n \le 2^{n^{O(1)}}$ such that $(F_n^{T_n})$ is superpolynomially hard.

The same equivalence holds with P/poly *replaced by subexponential size: some language in* PSPACE *has, for some $\delta > 0$, complexity at least 2^{n^δ} infinitely often if and only if some such family has $\mathrm{C}(F_n^{T_n}) \ge 2^{n^{\delta'}}$ infinitely often for some $\delta' > 0$. In (ii) the maps can be taken to be permutations, with T_n a power of two.*

Proof. (ii)⇒(iii) is immediate.

(iii)⇒(i). Suppose $\mathsf{PSPACE} \subseteq \mathsf{P/poly}$. Let $\mathrm{C}(F_n) \le n^a$ and $T_n \le 2^{n^c}$. Consider the language U of tuples $\langle 1^n, D, t, x, i\rangle$ such that D encodes a circuit of size at most n^a on n inputs, t has $n^c + 1$ bits, $x \in \{0,1\}^n$, $i \le n$, and bit i of $D^{\mathrm{val}(t)}(x)$ is one. Use a fixed-length encoding of the fields for each n, so that the length $m(n)$ of a tuple is a polynomial in n. A machine decides U in polynomial space: it evaluates D repeatedly, storing the current state and a counter of $n^c + 1$ bits. By assumption U has circuits of size m^e for some e. Hardwiring the description of F_n, the exponent T_n, and each index i gives $\mathrm{C}(F_n^{T_n}) \le n\, m(n)^e = n^{O(1)}$ for all large n, so (iii) fails.

(i)⇒(ii). Let $L \in \mathsf{PSPACE} \setminus \mathsf{P/poly}$, decided by a deterministic one-tape machine M that uses the cells $1, \dots, S(n)$ with $S(n) = n^c$, halts on every input, and returns its head to cell 1 before halting. These normalizations cost only a polynomial in space. Encode a configuration cell by cell: each cell holds a tape symbol and a control field, which is zero when the head is absent and otherwise names the machine state. With $r = O(1)$ bits per cell, the state width is $b = rS(n)$. Choose the symbol encoding so that the initial configuration on w is obtained from w by inserting constant bits; this map ι costs no gates.

The next-configuration map acts on each cell by a fixed function of that cell and its two neighbours, so it has a circuit of size $O(b)$, and it fixes halting configurations. Use it on all 2^b strings; on strings that are not valid configurations its values are irrelevant. Valid configurations do not

repeat before halting, so, as in the proof of Corollary 7.11, $F_b^{2^b}(\iota(w))$ is the halting configuration of M on w. Acceptance is read from the control field of cell 1 with $O(1)$ gates. Hence

$$\mathrm{C}(L_n) \le \mathrm{C}(F_{b(n)}^{2^{b(n)}}) + O(1).$$

Since $b(n)$ is strictly increasing, define F_b to be the identity on widths outside its range. If $\mathrm{C}(F_b^{2^b}) \le b^k$ for all large b, then $\mathrm{C}(L_n) \le O(n^{ck})$ for all large n, contradicting the choice of L. The permutation strengthening is Proposition 7.19.

For the subexponential version, the argument for (iii)⇒(i) gives $\mathrm{C}(F_n^{T_n}) \le n\, 2^{m(n)^\delta}$ whenever every language in PSPACE has complexity at most 2^{m^δ} for all large m; since $m(n) \le n^e$, taking δ small rules out $\mathrm{C}(F_n^{T_n}) \ge 2^{n^{\delta'}}$ infinitely often. Conversely, if $\mathrm{C}(L_n) \ge 2^{n^\delta}$ infinitely often, the construction above gives $\mathrm{C}(F_b^{2^b}) \ge 2^{n^\delta} - O(1) \ge 2^{b^{\delta/(2c)}}$ for those n when n is large. □

The next construction uses reversible machines. Lange, McKenzie, and Tapp [18] show that every language decided in deterministic space $S(n) \ge \log n$ is decided by a reversible machine in space $O(S(n))$ that halts on every input. Each configuration of a reversible machine has at most one successor and at most one predecessor. Both, and whether they exist, are determined by the constant number of cells around the head [19, 18]. We count configurations without a successor in a nonhalting state as rejecting halts.

Proposition 7.19 (Hard permutations and cycle lengths)**.** *Suppose $L \in \mathsf{DSPACE}(S(n))$ with $S(n) \ge n$ space-constructible. There are permutations P_b of $\{0,1\}^b$ with $b = O(S(n))$ and $\mathrm{C}(P_b) = O(b)$, an exponent $T = 2^{b'}$ with $b' < b$, and a constant-insertion map ι from $\{0,1\}^n$ to $\{0,1\}^b$, such that for every $w \in \{0,1\}^n$:*

(a) one fixed output bit of $P_b^T(\iota(w))$ equals $L(w)$;

(b) $L(w) = 1$ exactly when $\ell(\iota(w)) \ge 2^{b'+1}$.

Consequently, $\mathsf{PSPACE} \not\subseteq \mathsf{P/poly}$ if and only if there are polynomial-time constructible permutations P_b with $\mathrm{C}(P_b) = O(b)$ whose cycle-length maps $x \mapsto \ell(x)$ are superpolynomially hard.

Proof. Let R be a reversible machine deciding L in space $O(S)$, encoded cell by cell as in Theorem 7.18. Let V be the set of valid configurations, those with exactly one head, and let 2^v be the number of strings of the configuration field, so $|V| \le 2^v$. Put $D = 2^{v+2}$. The state set consists of

$$(C,+), \quad (C,-) \qquad (C \in V), \qquad\qquad (C,j) \qquad (C \in V \text{ accepting halting},\ 0 \le j < D).$$

Define P by

$(C,+) \mapsto (R(C),+)$	if C has a successor,
$(C,+) \mapsto (C,-)$	if C is a rejecting halt,
$(C,+) \mapsto (C,0)$	if C is an accepting halt,
$(C,j) \mapsto (C,j+1)$	if $j < D-1$, and $(C,D-1) \mapsto (C,-)$,
$(C,-) \mapsto (R^{-1}(C),-)$	if C has a predecessor,
$(C,-) \mapsto (C,+)$	otherwise.

Every state has exactly one preimage. For $(C',+)$ it is $(R^{-1}(C'),+)$ if C' has a predecessor and $(C',-)$ otherwise. For $(C',-)$ it is $(R(C'),-)$, $(C',+)$, or $(C',D-1)$, according as C' has a successor, is a rejecting halt, or is an accepting halt. For (C',j) with $j \ge 1$ it is $(C',j-1)$, and for $(C',0)$ it is $(C',+)$. Thus P is a permutation of the state set.

Encode a state by the configuration field, a direction bit, a detour bit, and a $(v+2)$-bit counter, so $b = 2v + 4 = O(S)$; strings encoding no state are fixed points. Validity is a linear-size test, and the other conditions are local at the head, so $\mathrm{C}(P_b) = O(b)$. Fixing the strings that encode no state preserves bijectivity.

Let $C_0, \dots, C_k$ be the maximal configuration path through the initial configuration I_w, with C_0 having no predecessor and C_k halting. It has no repetition, since the forward path from I_w halts and R is injective, so $k + 1 \le 2^v$. The cycle of $(I_w, +)$ visits $(C_0, +), \dots, (C_k, +)$, then the D detour states if C_k accepts, then $(C_k, -), \dots, (C_0, -)$. Its length is $2(k+1) + D \cdot L(w)$. Since $2(k+1) \le 2^{v+1} < D$, part (b) holds with $b' = v + 1$, because $2^{b'+1} = D$. For part (a), take $T = 2^{b'} = D/2$. From $(I_w, +)$ the first detour state is reached after at most $k + 1 \le 2^v < D/2$ steps and the detour lasts D steps, so $P^T(\iota(w))$ is a detour state when $L(w) = 1$. When $L(w) = 0$ the cycle contains no detour states. The detour bit is the required output.

For the equivalence, if $\mathsf{PSPACE} \not\subseteq \mathsf{P/poly}$, apply (b) to a hard L with polynomial S; testing $\ell \ge 2^{b'+1}$ costs $O(b)$ gates, and the width bookkeeping is as in Theorem 7.18. Conversely, $\ell(x)$ is computable in polynomial space from a circuit for P_b by walking the cycle with a $(b+1)$-bit counter, and the advice argument of Theorem 7.18 gives polynomial circuits for ℓ when $\mathsf{PSPACE} \subseteq \mathsf{P/poly}$. □

Part (a) completes the permutation clause of Theorem 7.18. The exponent $T = 2^{b'}$ is explicit and has $O(b)$ bits. This contrasts with Section 5, where an explicit permutation has hard powers at exponents of exponential length, without identifying an explicit hard exponent. Here the permutation depends on a hard language, and its power at a single short exponent inherits hardness from that language.

Remark 7.20 (Linear space and exponential bounds). The polynomial loss in the state width prevents a $2^{\varepsilon n}$ version of Theorem 7.18. Linear space gives one up to advice. If $\mathsf{DSPACE}(O(n)) \not\subseteq \mathsf{SIZE}(2^{\varepsilon n})$ infinitely often, the construction of Theorem 7.18 with $S(n) = O(n)$ gives maps with $\mathrm{C}(F_b) = O(b)$, $T_b = 2^b$, and $\mathrm{C}(F_b^{T_b}) \ge 2^{\varepsilon' b}$ infinitely often, with $\varepsilon' > 0$ depending on ε and the machine. Conversely, for such a family the power $F_b^{T_b}$ is computable in space $O(b)$ from $O(b \log b)$ bits of advice describing F_b and T_b: store all gate values and a counter. Hence $\mathsf{DSPACE}(O(n))/O(n \log n) \not\subseteq \mathsf{SIZE}(2^{\varepsilon'' n})$ infinitely often for some $\varepsilon'' > 0$. The two statements differ only in the advice.

7.6 Machines with a separate work tape

Corollary 7.11 charges every configuration bit, including a read-only input. For a space-bounded machine this gives a bound in terms of input length plus work space. The results of this subsection are not mechanized in the Lean development.

Let M be a deterministic machine with a read-only input tape holding $x \in \{0,1\}^n$ between two endmarkers, one work tape over an alphabet Γ that contains the blank, h_1 input heads, h_2 work heads, and a finite state set Q; put $h = h_1 + h_2$. Halting states are absorbing, and on every input of length n the machine halts and uses only work cells $1, \dots, S = S(n)$. A configuration (x, w, p, q) consists of the input, the work contents $w \in \Gamma^S$, the head positions p, and the state. Encode it in

$$b = n + S \lceil \log |\Gamma| \rceil + h \lceil \log(n + S + 2) \rceil + \lceil \log |Q| \rceil$$

bits, and let $F_M : \{0,1\}^b \to \{0,1\}^b$ be the one-step map, with $\iota(x)$ the initial configuration on input x. Reading the symbols under the heads, writing, and moving the heads cost $s = O(h(n + S \log |\Gamma|) + h \log(n + S))$ gates for fixed Q and Γ: decode each head position to a one-hot vector, select the symbols by multiplexers, apply one circuit for the transition function, and update each work cell and head position locally.

Theorem 7.21 (Space-bounded machines)**.** *Let M be as above, with h, $|Q|$ and $|\Gamma|$ fixed. The language decided by M has circuits on $\{0,1\}^n$ of size*

$$2^{(n+S\log|\Gamma|)/2}\,(n+S)^{O(1)},$$

where the $O(1)$ depends on h.

Proof. Let R be the set of configurations $F_M^t(\iota(x))$ with $t \geq 0$ and $x \in \{0,1\}^n$. It is forward invariant, and every configuration in it has input part x, work contents in Γ^S and heads within their ranges, so

$$|R| \leq 2^n|\Gamma|^S(n+S+2)^h|Q|.$$

Before halting, a run visits distinct configurations of R, so $F_M^{|R|}(\iota(x))$ is the final configuration. Apply Theorem 3.4 with state width b, the set R, and $T = |R|$, choosing L as in Equation (3.6) with N replaced by $|R|$. With $A = s + b^3$ the size is $O(A + \sqrt{b^3|R|A})$. Since b and s are $O(n+S)$, this is $(n+S)^{O(1)}\sqrt{|R|}$. Precompose with ι, which inserts constants, and read the acceptance bit from the state. □

For a binary work alphabet the encoding has $b = n + S + O(\log(n+S))$ bits, and the theorem already follows from Corollary 7.11. For other alphabets the reachable set matters, since $S\lceil\log|\Gamma|\rceil$ can exceed $S\log|\Gamma|$ by a constant factor.

The bound is new only in a narrow range. Every Boolean function on n bits has a circuit of size $O(2^n/n)$. The standard simulation squares the configuration map for the given input about $S\log|\Gamma|$ times; each squaring composes two tables with $|\Gamma|^S\,\mathrm{poly}(n,S)$ entries, for $|\Gamma|^{2S}\,\mathrm{poly}(n,S)$ gates in total. For a binary work tape and $S = \sigma n$ with a constant $\sigma > 0$, the three exponents are $(1+\sigma)/2$, 1 and 2σ. The first is the smallest exactly when $1/3 < \sigma < 1$. For instance, a language decided in space $n/2$ with a binary work tape has circuits of size $2^{3n/4}\,\mathrm{poly}(n)$. We have not found this bound in the literature. It combines checkpointing with locality and may be known. At $S = n$ it gives only $2^n\,\mathrm{poly}(n)$; the square-root bound $2^{n/2}\,\mathrm{poly}(n)$ of Corollary 7.11 needs the input to serve as the workspace.

Improving the square-root bound for in-place computations. Improving the exponent $1/2$ for computations that work in place would improve the generic bound for linear space. We make this precise with the configuration-map setting of Corollary 7.11. A *nearly in-place computation* is a family of maps $G_n : \{0,1\}^{b(n)} \to \{0,1\}^{b(n)}$ with $n < b(n) \leq n + c\lceil\log(n+1)\rceil$ and $\mathrm{C}(G_n) \leq n^c + c$ for a constant c. Its acceptance predicate on $\{0,1\}^n$ is

$$\mathrm{acc}_G(x) = \text{last bit of } G_n^{2^{b(n)}}(x0^{b(n)-n}).$$

The time $2^{b(n)}$ is at least the number of states, so for a halting computation with absorbing halting states this is the last bit of the final state. Theorem 1.1 gives $\mathrm{C}(\mathrm{acc}_G) \leq 2^{n/2}\,\mathrm{poly}(n)$. An in-place machine with a binary tape alphabet and a constant number of heads is an example: its configuration consists of the n tape bits and $O(\log n)$ bits for heads and state.

Proposition 7.22 (Padding linear space)**.** *Fix $\varepsilon > 0$. Suppose that every nearly in-place computation satisfies $\mathrm{C}(\mathrm{acc}_G) \leq 2^{(1/2-\varepsilon)n}$ for all sufficiently large n. Let L be decided by a machine M as above with a binary work alphabet, constant h, and a nondecreasing space bound $S(n)$ with $1 \leq S(n) \leq n^{O(1)}$. Then*

$$\mathrm{C}(L \cap \{0,1\}^n) \leq 2^{(1/2-\varepsilon)(n+S(n))}$$

for all sufficiently large n. In particular, every language decided in space n with a binary work tape has circuits of size $2^{(1-2\varepsilon)n}$.

Proof. Put $m(n) = n+S(n)$, which is strictly increasing. For $m = m(n)$, let G_m act on configurations of M on inputs of length n, laid out as the input x, then the S work bits, then a field z that holds the head positions, the state, and an acceptance bit in its last position. Choose the encoding of z so that $z = 0 \cdots 0$ is the initial head positions and state, and encode the blank by 0. Let G_m be the step map of M that sets the acceptance bit when it enters an accepting halting state. The field z has $O(\log(n + S)) = O(\log m)$ bits, and $\mathrm{C}(G_m) = O(m)$ by locality, so for some constant c this is a nearly in-place computation once G_m is defined as the identity on $m + 1$ bits at the lengths m outside the range of $m(\cdot)$.

On $y = x0^S$, the state $y0 \cdots 0$ encodes the initial configuration of M on x. The machine halts within the number of states, so $\mathrm{acc}_G(x0^S) = 1$ exactly when M accepts x. Fixing the last S inputs of a circuit for acc_G on $\{0,1\}^m$ to zero gives a circuit for $L \cap \{0,1\}^n$ of size at most $2^{(1/2-\varepsilon)m}$. For $S(n) = n$ the size is $2^{(1/2-\varepsilon)2n}$. □

A bound of $2^{(1-\delta)n}$ for linear space with a binary work tape would beat the $\Theta(2^n/n)$ worst case for n-bit functions by an exponential factor. We are not aware of such a bound, and we do not know whether the hypothesis of Proposition 7.22 holds for any $\varepsilon > 0$. We do not prove the proposition with the hypothesis restricted to in-place Turing machines. That would require the padded machine to find the end of x itself, with no symbols to spare.

The hypothesis concerns one predicate per input length, at the fixed time $2^{b(n)}$. For the configuration map, or for a predicate at a time chosen for each n, the analogous improvement is false. The map P_n, with one idle extra bit, is a nearly in-place computation, and Theorem 5.4 and Corollary 5.6 give exponents T_n for which $P_n^{T_n}$, and some output bit of it, need $2^{n/2}/\operatorname{poly}(n)$ gates. Those exponents come from counting over T. Counting says nothing about a single predicate per length.

8 Structured transitions

The universal bound uses only the state width and the size of one transition. Additional structure can give compact circuits for whole iterates even when an orbit visits exponentially many states. Affine payload updates stay affine along each autonomous controller trajectory; sparse counter edits give orbit coordinates described by a few intervals. In both cases, selecting and evaluating the resulting description has a charged circuit cost that can improve on the universal bound.

8.1 Small controllers with affine payloads

The next result isolates all exponential cost in the autonomous controller, allowing a much larger affine payload.

Theorem 8.1 (Affine-controller powering)**.** *Let $q + p = n \geq 1$, and write a state as $(u, v) \in \{0,1\}^q \times \mathbb{F}_2^p$. Suppose*

$$F(u, v) = \big(g(u), A(u)v + b(u)\big),$$

where $g : \{0,1\}^q \to \{0,1\}^q$, $A(u)$ is a $p \times p$ binary matrix, and $b(u) \in \mathbb{F}_2^p$. For every fixed integer $T \geq 0$,

$$\mathrm{C}(F^T) = O\Big(2^q(q + p^2)\Big).$$

For an m-bit input exponent, the size bound is $O(m2^q(q + p^2))$. In particular, $q = O(\log n)$ gives polynomial-size circuits for every fixed power, uniformly in the magnitude of T. The matrices may be singular and need not commute.

Proof. Once the initial controller u is fixed, the controller trajectory and all affine maps applied to v are fixed. Composition of affine maps is affine, so for some hardwired matrix $M_T(u)$ and vector $d_T(u)$,

$$F^T(u, v) = (g^T(u), M_T(u)v + d_T(u)).$$

For each of the 2^q controller values, evaluate its fixed affine map using $O(p^2)$ gates and provide its fixed controller output. Compute each controller-equality flag once, and share it across the $q + p$ output bits. A scan selects the matching record using $O(2^q(q + p))$ additional gates. This proves the fixed-power bound, including a charge for every table-selection gate.

For an input exponent, conditionally apply the circuits for F^{2^j} to the current full state, as in Section 6.1. Each stage updates the controller as well as the payload and adds $O(q + p)$ selection gates. Summing over m stages gives the second bound. □

Autonomy is essential to this argument: if the controller update depends on v, its future choices need not be determined by the initial u. No efficient algorithm for obtaining $M_T(u)$ from a succinct transition is asserted here. The bound uses ordinary affine composition and finite selection; it does not claim a new algebraic closure principle.

The same reasoning applies beyond affine maps. Let $\mathcal{M}$ be a family of maps on p-bit payloads containing the identity and closed under composition, and suppose *every* member has a circuit of size at most κ. If the autonomous controller selects maps from $\mathcal{M}$, every fixed power costs

$$O(2^q(\kappa + q + p + 1)),$$

and an m-bit input time multiplies this by m. Closure alone is insufficient: the uniform evaluation bound on composite maps is a separate hypothesis. For nonuniform circuit existence, no efficient procedure for computing those composites is needed.

8.2 Sparse modifications of a counter

The state-space parameter is not the only source of compression. Identify $\{0, 1\}^n$ with $\mathbb{Z}/N\mathbb{Z}$, where $N = 2^n$, and write $R(x) = x + 1 \pmod N$. A permutation obtained by changing a small number of counter successors admits a bound linear in its sparse description, even when it is a single cycle through all N states.

Theorem 8.2 (Sparse-counter powers)**.** *Let $n \geq 1$, let F be a permutation of $\mathbb{Z}/N\mathbb{Z}$, and let $r = |\{x : F(x) \neq R(x)\}|$. For every fixed $T \in \mathbb{Z}$,*

$$\mathrm{C}(F^T) = O\big(n(r + 1)\big).$$

For an unsigned m-bit time supplied as input,

$$\mathrm{C}\big((x, t) \mapsto F^t(x)\big) = O\big(n(r + m + 1)\big).$$

Both constants are absolute. The dependence on time-input length is additive.

8.2.1 Commutation defects and displacement

The fixed-power assertion follows from a more general statement.

Proposition 8.3 (Sparse-counter centralizers)**.** *Let F be any map on $\mathbb{Z}/N\mathbb{Z}$ differing from R at r inputs. Every permutation H commuting with F has a displacement function $\delta_H(x) = H(x) - x \pmod N$ that changes on at most $2r$ counter edges. Consequently, $\mathrm{C}(H) = O(n(r + 1))$.*

Proof. Put $E = \{x : F(x) \neq R(x)\}$. If $x \notin E$ and $H(x) \notin E$, then

$$H(R(x)) = H(F(x)) = F(H(x)) = R(H(x)).$$

Thus the set of commutation failures for H and R lies in $E \cup H^{-1}(E)$ and has size at most $2r$. Commutation at x is exactly $\delta_H(x+1) = \delta_H(x)$. Cutting the ordinary linear order $0, \ldots, N-1$ at these changes gives at most $2r+1$ constant intervals. Compare the input with their endpoints, select a hardwired displacement, and add it modulo N, at cost $O(n(r+1))$.

An alternative implementation, used in Lean, avoids ordering the changes. Let $D = \{i : \delta_H(i+1) \neq \delta_H(i)\}$, with cyclic addition in the arguments and ordinary integer comparisons below. Telescoping gives

$$H(x) = x + \delta_H(0) + \sum_{\substack{i \in D \\ i < x}} \big(\delta_H(i+1) - \delta_H(i)\big) \pmod{N}.$$

Each summand uses one comparison, a conditional constant word, and a linear-size adder. The wraparound edge is never selected. All gates, including the shared carry chains, are charged. □

Every signed power of a permutation commutes with it, proving the first part of Theorem 8.2. The proposition only requires injectivity of H in its cardinality argument; it does not require F to be a permutation. Its set-containment argument works on arbitrary universes, before finiteness or coordinates are imposed.

The geometry relating commutation failures to blocks in cycles is established permutation theory. Moreno and Rivera characterize these blocks in their Theorem 3.8 and Corollary 3.10 [22]. Here we use the elementary defect containment and account for the Boolean cost of the resulting representation. Historical priority of these precise circuit bounds has not been determined.

8.2.2 A chart constructed from the edited successors

For the live-time bound, assume first $r > 0$ and cut the counter cycle immediately after each point of E. There are r nonempty circular arcs. Inside an arc F increments; its only possible change is at the last state. Since F and R are permutations agreeing off E,

$$F(E) = R(E).$$

Indeed, the images of the unedited inputs coincide, so their complements coincide. Therefore each edited edge leads to an arc start, and these edges induce a permutation of the r arcs.

Decompose that permutation into cycles and concatenate the arcs in each cycle, retaining their individual lengths. Assign to a state the triple (c, p, ℓ) consisting of its cycle identifier, its position within that cycle, and its total cycle length. The coordinate map and its inverse are piecewise translations on the original arcs and their concatenated images. At most one original arc crosses zero; splitting it there adds one linear interval. Comparisons, shared record selection, and addition therefore implement each direction in $O(n(r+1))$ gates. Identifiers and positions use n bits, while lengths use $n+1$ bits to allow $\ell = N$. All intermediate arithmetic uses $O(n)$ bits.

In these coordinates one step preserves c, ℓ and increments p modulo ℓ. For an m-bit input t, compute its remainder by scanning bits from most significant to least significant. With current remainder $u < \ell$, the next remainder is $2u + b$ or $2u + b - \ell$, according as $2u + b < \ell$ or not. A comparison and subtraction cost $O(n)$ per bit; one extra bit prevents overflow. Add p, reduce once more, and decode. The cost is $O(nr + nm + n)$. If $r = 0$, ordinary addition modulo N gives the

same bound. This proves the second part of Theorem 8.2. For fixed signed T, hardwire $T \bmod \ell$ for each cycle instead.

Corollary 8.4 (Labels and navigation for sparse counters)**.** *For a permutation differing from the counter at r states, the cycle length, numerically least cycle vertex, and phase of an input state can be computed with $O(n(r+1))$ gates. The same bound suffices, on input (x, y), for reachability, the exact first hitting time, and a descriptor of all visits.*

Proof. Cut the ordinary linear order after each exception, obtaining at most $r+1$ intervals. Within each interval $[a, b)$, the trajectory follows ordinary addition. Hardwire its cycle representative c, cycle length ℓ, and phase p_a at its first state. An input x in that interval has label $(c, \ell, (p_a + x - a) \bmod \ell)$. The interval length is at most ℓ, so one addition and conditional subtraction suffice for the phase. Comparisons and record selection cost $O(n)$ per interval.

Compute the two labels. Since every state is cyclic, y is reachable from x exactly when $c(x) = c(y)$. On a positive instance, the first hit is

$$d = (\phi(y) - \phi(x)) \bmod \ell(x),$$

and all visits occur at times $d + k\ell(x)$, $k \geq 0$. Both phases are already below the common cycle length, so the difference needs only comparison and subtraction: use $\phi(y) - \phi(x)$ when nonnegative, and $\ell(x) - (\phi(x) - \phi(y))$ otherwise. This costs $O(n)$ additional gates. The descriptor also determines the exact bounded visit count; this is an arithmetic interpretation of its output, with no deadline input charged. □

Proposition 8.5 (Polynomial-time construction from an edit list)**.** *Given n and an explicit list of the r edited successors of the n-bit counter, one can check permutation preservation and construct an orbit chart of $O(r+1)$ records, each of $O(n)$ bits, in time polynomial in n and $r+1$. For a valid list, the chart supplies the encoding and decoding used above. Given also a signed binary integer T, one can construct a piecewise-translation table for F^T with at most $2r+1$ linear intervals in time polynomial in n, $r+1$, and $\log(|T|+2)$.*

Proof. Compare the edited outputs with the freed counter outputs to check $F(E) = R(E)$, after checking distinct input keys. Sort the cuts, compute arc lengths, follow the induced permutation of the arcs, and form prefix sums on each of its cycles. These operations construct the chart using arithmetic on $O(n)$-bit integers. The case $r = 0$ is the ordinary counter chart. For a signed power, reduce T modulo each chart cycle length. Use the chart to evaluate F^{-T} on E, and test the $O(r)$ candidate change edges $E \cup F^{-T}(E)$ from Proposition 8.3. Sorting their successors and evaluating the displacement at the interval starts gives the table; discard boundaries without a displacement change. □

The input here is the explicit edit list, whose entries specify where counter edges have changed and their new successors. Given only a circuit for F, identifying that list is a separate task: the proposition asserts no polynomial-time extraction of exceptions from a transition circuit. The chart construction avoids enumeration of the $N = 2^n$ states. The explicit arithmetic and selection constructions above then compile an m-bit iterator in time polynomial in $n, r+1, m$ with the gate bound of Theorem 8.2.

The executable `sparse_counter.py` implements this chart and interval compiler. Independent finite checks and their scope are recorded in `REVIEW.md`; they do not prove the gate bound or compiler correctness.

8.2.3 Large transitions and charged coordinate changes

The improvement can apply to transitions with genuinely exponential minimum circuit size. Fix r nonempty counter arcs and reconnect them in any single cyclic order. This gives $(r-1)!$ distinct permutations, each differing from the counter at at most r states and having one orbit of length N. For $r = 2^{\lfloor \alpha n \rfloor}$ with fixed $0 < \alpha < 1$, the logarithm of this family size is $\Theta(r \log r)$. The circuit-description bound used in Section 5 implies that some members need $\Omega(r)$ gates, while all have $O(n(r+1))$-gate circuits. Consequently there exist families with

$$\mathrm{C}(F_n) = 2^{\alpha n + O(\log n)}, \qquad \sup_{T \in \mathbb{Z}} \mathrm{C}(F_n^T) = 2^{\alpha n + O(\log n)}.$$

The lower bound is existential counting, not an explicit hard transition. Using only their transition sizes in Theorem 1.1 would instead give an upper bound with exponential scale $2^{(1+\alpha)n/2}$.

Corollary 8.6 (Charged orbit coordinates)**.** *Suppose a bijection $\phi : \{0,1\}^n \to \mathbb{Z}/N\mathbb{Z}$ satisfies $\phi(G(x)) = \phi(x) + 1 \pmod N$. If a permutation F differs from G at r states, then*

$$\mathrm{C}(F^T) \le \mathrm{C}(\phi) + \mathrm{C}(\phi^{-1}) + O(n(r+1)) \quad (T \in \mathbb{Z}).$$

For an m-bit input time, the last term is $O(n(r+m+1))$.

Proof. Conjugate F by ϕ. The exceptional-set cardinality is unchanged, and the reference permutation becomes the counter. Apply Theorem 8.2 and compose with the two coordinate circuits. □

Both coordinate costs are essential. A long orbit does not itself supply cheaply computable coordinates. Conversely, these examples can visit the entire state space: the compressed object is the arrangement of arcs, without a reduction in the number of reachable states.

For the pair queries of Corollary 8.4, coordinate transport needs only the forward map: encode both x and y, then query the conjugated permutation. Reachability and visit times are preserved by conjugacy, so the bound is $2\mathrm{C}(\phi) + O(n(r+1))$. No state is returned, and no inverse-coordinate circuit is required.

A finite-field application, including the cost of discrete-logarithm coordinates, is given in Section B.

9 Distinguished points: small width and reversible circuits

The batched lookup of Lemma 2.1 is needed because membership in the packed checkpoint set of Lemma 2.3 has no cheap test: every state of the initial trajectory must be looked up. If membership in H can be decided by a small circuit, only the *first* checkpoint on the trajectory needs a payload, and one linear scan over the dictionary suffices. Distinguished points, an idea attributed to Rivest in the time–memory tradeoff literature [35], give such a set when they are selected by a hash function of limited independence. This section derives three consequences: a polynomial-factor improvement of Equation (1.2), circuits of width $O(n+s)$, and reversible circuits with few ancillas. The Lean development checks only the semantic correctness of the six-step procedure in the proof of Theorem 9.2, for an arbitrary set H with the hitting property of Lemma 9.1 (`distinguishedPower_correct`; see Section 12). The hash lemma, the gate counts and widths, and the reversible and pebbling results of this section are not mechanized.

9.1 Hashed checkpoints

Identify $\{0,1\}^n$ with the field $\mathbb{F}_{2^n}$ through a fixed irreducible polynomial of degree n over $\mathbb{F}_2$. Put $t = 2n$. For coefficients $a = (a_0, \ldots, a_{t-1}) \in \mathbb{F}_{2^n}^t$ and $0 \le l \le n$, let

$$g_a(x) = \sum_{j<t} a_j x^j, \qquad H_{a,l} = \{x : \text{the first } l \text{ coordinates of } g_a(x) \text{ vanish}\}.$$

Lemma 9.1 (Hashed checkpoint or repetition)**.** *Let $n \ge 3$, let F be any function on $\{0,1\}^n$, and let $L = 2^l$ with $0 \le l \le n-1$. Put $K = 16nL - 1$. There are coefficients a such that $H = H_{a,l}$ satisfies $|H| \le 2N/L$ and, for every x, either two of $x, F(x), \ldots, F^K(x)$ are equal or $F^i(x) \in H$ for some $i \le K$. Membership in $H_{a,l}$ has a circuit with $O(n^3)$ gates and width $O(n)$.*

Proof. Choose a uniformly. For distinct $x_1, \ldots, x_t$ the vector $(g_a(x_1), \ldots, g_a(x_t))$ is uniform on $\mathbb{F}_{2^n}^t$, since the Vandermonde matrix of distinct points is invertible; here $t \le N$. Hence the indicators $Y_x = [x \in H_{a,l}]$ are t-wise independent Bernoulli variables with mean $1/L$.

Size. The indicators are in particular pairwise independent, so $|H| = \sum_x Y_x$ has mean N/L and variance at most N/L. By Chebyshev's inequality, $\Pr[|H| > 2N/L] \le L/N \le 1/2$.

Hitting. Fix x and suppose that $x_i = F^i(x)$, $0 \le i \le K$, are distinct. The trajectory is determined by F alone, before a is drawn. Then $X = \sum_{i \le K} Y_{x_i}$ is a sum of $K + 1 = 16nL$ t-wise independent indicators with mean $\mu = 16n$. Bellare and Rompel's tail bound for even $t \ge 4$ [30],

$$\Pr[|X - \mu| \ge \Delta] \le 8 \left(\frac{t\mu + t^2}{\Delta^2} \right)^{t/2},$$

applies with $\Delta = 15n$, because $X = 0$ forces $|X - \mu| = 16n \ge \Delta$. Here $(t\mu + t^2)/\Delta^2 = (32n^2 + 4n^2)/(225n^2) = 0.16$, so $\Pr[X = 0] \le 8 \cdot 0.16^n$. If the first $K + 1$ states are not distinct there is nothing to prove. A union bound over the N starting states therefore bounds the failure probability of the hitting condition by $8 \cdot 0.32^n \le 0.27$ for $n \ge 3$. Since $0.27 + 1/2 < 1$, some a satisfies both conditions.

Membership. Evaluate $g_a(x)$ by Horner's rule: $t - 1$ products of two variable field elements and t additions of hardwired constants. A product is a schoolbook polynomial product followed by reduction modulo the fixed irreducible polynomial. Both are $\mathbb{F}_2$-bilinear or linear maps with $O(n^2)$ gates. Producing one product coefficient at a time and adding its fixed reduction vector into an n-bit accumulator keeps only $O(n)$ wires live. Finally test l bits for zero. The total is $O(tn^2) = O(n^3)$ gates and width $O(n)$. □

The coefficients are found nonuniformly, as are all tables in this paper. No attempt is made to reduce the $O(n^3)$ membership cost, for instance by fast field multiplication.

9.2 One lookup per input

The *width* of a circuit is the minimum, over topological orders of its gates, of the largest number of inputs and gate values that have already been produced and are still needed by a later gate or as an output. It is the number of registers of a straight-line program evaluating the circuit. Every circuit of size s for F has width at most $s + n$.

Theorem 9.2 (Distinguished-point fixed powers)**.** *Let $n \geq 3$, let F have a circuit with s gates and width w, and put $A = s + n^3$. For every $T \geq 0$ and every power of two $1 \leq L \leq N/2$,*

$$\mathrm{C}(F^T) = O\Big(nLA + \frac{nN}{L}\Big), \tag{9.1}$$

by a circuit of width $O(n + w)$. Consequently, every fixed power satisfies

$$\mathrm{C}(F^T) = O\Big(n\sqrt{NA}\Big), \tag{9.2}$$

again by circuits of width $O(n + w) \leq O(n + s)$.

Proof. Take H, $K = 16nL - 1$, and $x_i = F^i(x)$ as in Lemma 9.1. If $T < K$, unroll T copies of F, using $Ts \leq 16nLs$ gates and width $O(n + w)$. Assume $T \geq K$ and hardwire the payloads $z_h = F^{T-K}(h)$ for $h \in H$ and the residues $R(\lambda) = (T - K) \bmod \lambda$ for $1 \leq \lambda \leq K$. Each phase below is a straight-line program on an $O(n)$-bit configuration; stage indices are hardwired, so no counter is needed. All index fields have $O(n)$ bits, since $K < 8nN$.

1. *Scan.* The configuration is (u, ϕ, d, c), initially $(x, 0, 0, 0)$. At stage $i = 0, 1, \ldots, K$: if $\phi = 0$ and $u \in H$, set $\phi = 1$, $d = i$ and $c = u$; then, if $i < K$, replace u by $F(u)$. Finally $u = x_K$, and if $\phi = 1$, then d is the first index with $x_d \in H$ and $c = x_d$. Each stage costs $s + O(n^3) = O(A)$ gates.

2. *Lookup.* Put $z = \bigvee_{h \in H} [c = h] \cdot z_h$, bitwise. For each key this is an n-bit equality test and $O(n)$ further gates, $O(n|H|) = O(nN/L)$ in total, in one pass over the keys.

3. *Correction.* Starting from $v = z$, at stage $i = 0, \ldots, K - 1$ replace v by $F(v)$ exactly when $i \geq d$. This applies F exactly $K - d$ times, so if $\phi = 1$, then as in Equation (3.3), $v = F^{K-d}(F^{T-K}(x_d)) = F^T(x)$.

4. *Period.* In a separate register, starting from $w = x_K$, at stage $j = 1, \ldots, K$ replace w by $F(w)$, and record $\lambda = j$ the first time $w = x_K$; use the dummy value $\lambda = 1$ if this never happens. The register v from the correction step is left unchanged.

5. *Residue and cycle walk.* Read $r = R(\lambda)$ by a linear scan of the K table entries, $O(nK)$ gates. Starting from $v' = x_K$, at stage $i = 0, \ldots, K - 1$ replace v' by $F(v')$ exactly when $i < r$.

6. *Output.* Return v if $\phi = 1$ and v' otherwise.

If $\phi = 1$ the output is correct by step 3. If $\phi = 0$, then by Lemma 9.1 two of $x_0, \ldots, x_K$ coincide. The orbit of x then has tail length μ and cycle length λ^* with $\mu + \lambda^* \leq K$. Hence x_K lies on the cycle, step 4 returns $\lambda = \lambda^*$, and step 5 gives $v' = F^{K+r}(x)$ with $K + r \equiv T \pmod{\lambda^*}$ and $K + r, T \geq \mu$, so $v' = F^T(x)$. As in Section 3, both branches are evaluated on every input, with dummy values where needed. No division is performed.

The four walks cost $O(KA) = O(nLA)$ gates, the lookup $O(nN/L)$, and the residue scan and selections $O(nK) = O(n^2 L)$, which is absorbed. This proves Equation (9.1). Only $O(1)$ configurations, the $O(n)$-bit Horner state of Lemma 9.1, and one evaluation of F are live at any time, which gives width $O(n + w)$.

For Equation (9.2), if $A \leq N$ let L be the largest power of two with $L \leq \sqrt{N/A}$. Then $L \leq \sqrt{N/n^3} \leq N/2$, so $nLA \leq n\sqrt{NA}$ and $nN/L < 2n\sqrt{NA}$. If $A > N$, scan a complete truth table of F^T: for each state x' an n-bit equality test with the input and $O(n)$ gates to OR in the stored value. This uses $O(nN) \leq O(n\sqrt{NA})$ gates and width $O(n)$. □

Remark 9.3 (Comparison with sorting-based lookup). For $A \le N$, Equation (9.2) improves Equation (1.2) by a factor of order $\sqrt{n}$, with the same $A = s + n^3$. The saving comes from replacing a sort of $\Theta(N/L + L)$ records, at $O(n^3)$ gates per record, by one comparison per key at $O(n)$ gates; the price is a walk of length $16nL$ in place of $2L$. This is not the best known gate count. An $O(R \log R)$-comparator network [2] in Lemmas 2.1 and 2.2 reduces the sorting cost to $O(n^2 R)$ and gives $O(A_2 + n\sqrt{NA_2})$ with $A_2 = s + n^2$ (Theorem 6.8), which is smaller than Equation (9.2) by up to a factor $\sqrt{n}$ when $s < n^3$. The n^3 term here is the cost of the hash, not of lookup. The advantage of distinguished points is structural: no sorting network is used, and the width is $O(n+s)$. The sorting network on $\Theta(N/L + L)$ records used in Section 3 manipulates that many records together; we do not know how to evaluate it in width much below $\sqrt{N}$ at the balanced choice of L. Equivalently, Theorem 9.2 describes a machine with $O(n + s)$ bits of read/write memory and a read-only data stream that it reads in one left-to-right pass, performing $O(n\sqrt{NA})$ bit operations. The stream must repeat the data that the machine cannot retain: each of the $O(K)$ membership tests needs the $2n$ hash coefficients, which occupy $\Theta(n^2)$ bits, and each of the $O(K)$ evaluations of F needs a gate list of $O(s \log s)$ bits. The stream therefore consists of one block of coefficients and one copy of the gate list of F in front of every stage of the walks, followed at the appropriate point by the key–payload pairs and the residue table. Its length is $O(nN/L + nL(n^2 + s \log s))$, which is $O(n\sqrt{NA} \log s)$ at the balanced choice of L. If the machine may instead reread a fixed region of its input, one copy of the coefficients and of the gate list suffices, and the stream has length $O(nN/L + n^2 + s \log s)$. The circuit-width statement needs neither variant, since its constants are hardwired.

9.3 Reversible circuits with few ancillas

A reversible circuit uses NOT, CNOT and Toffoli gates on a fixed set of wires. A circuit *computes* $G : \{0,1\}^n \to \{0,1\}^q$ with a ancillas if it maps $(x, y, 0^a)$ to $(x, y \oplus G(x), 0^a)$ for all x and y. Any Boolean circuit with g gates gives such a circuit with $O(g + q)$ gates and g ancillas: compute every gate into a fresh zero wire, copy the outputs with CNOTs, and run the first part backwards. A fan-in-two gate into a fresh wire needs $O(1)$ reversible gates, since every Boolean function of two bits is a polynomial of degree at most two over $\mathbb{F}_2$. Applied to Theorem 9.2 directly, this gives as many ancillas as gates. Bennett's pebbling of the four walks removes almost all of them.

A walk with K stages is a sequence of configurations $c_0, \dots, c_K$ with $c_{i+1} = \mathrm{step}_i(c_i)$. A *move* XORs $\mathrm{step}_i(c_i)$ into a register, which either creates c_{i+1} from zeros or clears it. Let $k \ge 2$ and $m = \lceil \log_k K \rceil$, and pad the walk to k^m stages with identity steps. To advance k^j stages from a held configuration, advance k times by k^{j-1} stages, and then undo the first $k - 1$ of these advances in reverse order. If $M(j)$ moves suffice for level j, then $M(j) = (2k - 1)M(j - 1)$ and $M(0) = 1$, so $M(m) = (2k - 1)^m$. At most $(k - 1)m + 1$ configurations besides c_0 are held at once [31, 32]. Put

$$\varepsilon_k = \log_k(2k - 1) - 1 = \log_k\!\left(2 - \tfrac{1}{k}\right) \in \left(0, \tfrac{1}{\log k}\right].$$

Since $k^{m-1} < K$,

$$(2k - 1)^m = (k^m)^{1+\varepsilon_k} < (kK)^{1+\varepsilon_k} \le k^2 K^{1+\varepsilon_k}.$$

For example, $\varepsilon_2 = \log 3 - 1 \approx 0.585$, $\varepsilon_4 \approx 0.404$ and $\varepsilon_{16} \approx 0.238$.

Theorem 9.4 (Reversible fixed powers)**.** *Let $n \ge 3$, $\mathrm{C}(F) \le s$, $A = s + n^3$, and let $k \ge 2$ be an integer. For every $T \ge 0$ and every power of two $1 \le L \le N/2$, a reversible circuit computes F^T with*

$$O\!\left(\frac{nN}{L} + k^2 (nL)^{1+\varepsilon_k} A\right) \text{ gates and } O\!\left(kn \left\lceil \frac{n}{\log k} \right\rceil + s\right) \text{ ancillas.}$$

Proof. Use the phases of Theorem 9.2. Each of the four walks has at most $K+1 = 16nL$ stages on configurations of $b = O(n)$ bits.

One move. For the scan, a move XORs $\mathrm{step}_i(c)$ into a register c'. Compute the membership bit of u by Horner's rule, keeping all $t = 2n$ intermediate field elements; each product is formed by n^2 Toffoli gates into a $(2n-1)$-bit coefficient register, reduced by $O(n^2)$ CNOTs, after which the coefficient register is cleared by the same Toffoli gates. This costs $O(n^3)$ gates and $O(n^2)$ ancillas, all cleared after the bit is copied. Then XOR $F(u)$ into the u-field of c', with $O(s)$ gates and s ancillas, and XOR the $O(n)$-gate updates of ϕ, d and c into c'. Finally clear the membership bit. A move therefore costs $O(A)$ gates with $O(s+n^2)$ reusable ancillas. Moves of the other walks cost $O(s+n)$ gates and $O(s+n)$ ancillas; a conditional application of F is computed into a temporary register, selected bitwise into c', and then uncomputed.

Walks. By the pebbling count, each walk uses $(2k-1)^m \le k^2(16nL)^{1+\varepsilon_k}$ moves, hence $O(k^2(nL)^{1+\varepsilon_k}A)$ gates, and at most $(k-1)m+2$ configuration registers. Since $K < 8nN$, $m = \lceil \log_k K \rceil = O(\lceil n/\log k \rceil)$, so the registers use $O(kn\lceil n/\log k\rceil)$ wires. Afterwards only c_0 and c_K are held.

Assembly. Run the scan, keeping its final configuration. For the lookup, for each key h compute $[c = h]$ into one ancilla with $O(n)$ gates and $O(n)$ ancillas, CNOT it into the positions of the one bits of z_h in a fresh register, and clear it. Copy the needed fields with CNOTs and run the correction and period walks. Read the residue by the same comparison scan over the K table entries, using $O(nK)$ gates, and run the cycle walk. For each output bit, two Toffoli gates XOR the bit selected by ϕ into y. Finally run everything except the output step backwards, which clears every ancilla. If $T < K$, pebble the walk of T applications of F instead.

The lookups cost $O(nN/L)$ gates. The residue scans and copies cost $O(nK) = O(n^2L)$, which is absorbed by the walks because $A \ge n^3$. Besides the pebbling registers, $O(1)$ configurations and $O(s+n^2)$ workspace ancillas are used. □

Corollary 9.5 (Ancilla–gate tradeoffs)**.** *Under the assumptions of Theorem 9.4, write $\varepsilon = \varepsilon_k$ and assume $A \le N/(k^2n)$. A reversible circuit computes every fixed power F^T with*

$$O\Big(k\,n^{1+\varepsilon/2}A^{1/2}N^{(1+\varepsilon)/(2+\varepsilon)}\Big)$$

gates and $O(kn\lceil n/\log k\rceil + s)$ ancillas. In particular:

1. *for $k = 2$, with $O(n^2+s)$ ancillas, the gate count is $N^{\log 3/(1+\log 3)}\,\mathrm{poly}(n)A^{1/2}$, where $\log 3/(1+\log 3) \approx 0.6131$;*
2. *for fixed k, the exponent of N is $\frac{1}{2} + \varepsilon_k/(2(2+\varepsilon_k)) < \frac{1}{2} + 1/(4\log k)$, with $O_k(n^2+s)$ ancillas;*
3. *for $k = n$, with $O(n^3/\log n + s)$ ancillas, the gate count is $N^{1/2}\,2^{n/(4\log n)}\,\mathrm{poly}(n)A^{1/2} = N^{1/2+O(1/\log n)}A^{1/2}$.*

A reversible circuit with $O(nN)$ gates and $O(n)$ ancillas computes F^T for every F, which covers larger A.

Proof. Let L be the largest power of two with $L \le \Lambda := (N/(k^2n^\varepsilon A))^{1/(2+\varepsilon)}$. The assumption on A gives $\Lambda \ge 1$, and $\Lambda \le \sqrt{N} \le N/2$. At $L = \Lambda$ the two terms of Theorem 9.4 agree up to a constant, and halving L changes them by constant factors. So the gate count is

$$O\Big(\frac{nN}{\Lambda}\Big) = O\Big(nN^{(1+\varepsilon)/(2+\varepsilon)}(k^2n^\varepsilon A)^{1/(2+\varepsilon)}\Big),$$

which is at most the displayed bound because $k^2n^\varepsilon A \ge 1$ and $1/(2+\varepsilon) \le 1/2$. The exponent of N equals $\frac{1}{2} + \varepsilon/(2(2+\varepsilon))$, and $\varepsilon_k \le 1/\log k$. For $k = 2$, $(1+\varepsilon_2)/(2+\varepsilon_2) = \log 3/(1+\log 3)$. For

$k = n$, $N^{\varepsilon/(2(2+\varepsilon))} \le N^{\varepsilon/4} \le 2^{n/(4\log n)}$ and $kn\lceil n/\log k\rceil = O(n^3/\log n)$. For the last claim, scan the truth table of F^T: for each state x', compute $[x = x']$ into an ancilla with $O(n)$ gates and $O(n)$ ancillas, CNOT it into the one bits of $F^T(x')$ in y, and clear it. □

Corollary 9.6 (In-place permutations)**.** *Let P be a permutation of $\{0,1\}^n$ with $\mathrm{C}(P) \le s$. For each integer $k \ge 2$ satisfying the hypothesis of Corollary 9.5, there is a reversible circuit mapping $(x, 0^{n+a})$ to $(P(x), 0^{n+a})$, after a fixed relabelling of wires, with $a = O(kn\lceil n/\log k\rceil + s)$ and the gate count of Corollary 9.5 plus $O(s+n)$.*

Proof. First XOR $P(x)$ into a zero register with $O(s+n)$ gates and s temporary ancillas. Then apply the circuit of Corollary 9.5 for $P^{-1} = P^{\mathrm{ord}(P)-1}$ from the new register into the input register. It XORs $P^{-1}(P(x)) = x$ into x and leaves zero there. Relabel the two n-wire registers. □

Remark 9.7 (Lower bounds and the pebbling overhead)*.* A reversible circuit with g gates yields a Boolean circuit with $O(g)$ gates, whatever the number of ancillas: each reversible gate changes one wire by a fan-in-at-most-three function. So Theorem 5.4 applies to reversible circuits with any number of ancillas, and the exponent $1/2$ cannot be lowered for them either. Whether $N^{1/2}\,\mathrm{poly}(n,s)$ gates are possible with $\mathrm{poly}(n,s)$ ancillas is open. The overhead $N^{\Theta(\varepsilon_k)}$ comes only from pebbling the four walks. Bennett's pebble game on a path, which models any construction storing only whole configurations of a walk, was analyzed exactly by Knill [33], and Li, Tromp, and Vitányi [34] show that Bennett's strategy reaches the greatest distance for a given number of pebbles. Such results limit this strategy class only. They give no circuit lower bound, since a reversible circuit need not simulate a walk configuration by configuration.

10 Applications

This section connects the circuit bounds to exact program summaries, bounded choices, preprocessing, universal maps, and asymmetric iterators. The target and navigation circuits used below are mechanized, as are the simulation and hardwiring arguments of Sections 10.4 and 10.5. The local certificate characterization, SAT-oracle analyses, SETH reduction, and preprocessing runtime are prose results.

10.1 Exact program summaries and local certificates

Let $P : \{0,1\}^n \to \{0,1\}$ be a target predicate of size a, put $N = 2^n$, $A_P = s + a + n^3$, and $B_P = A_P + \sqrt{n^3 N A_P}$. Define the *target distance*

$$\tau_P(x) = \begin{cases} \min\{t \ge 0 : P(F^t(x)) = 1\}, & \text{if the target is reached,} \\ N, & \text{otherwise.} \end{cases}$$

Every finite first-hit time is less than N, so the sentinel N means *never* without ambiguity; the output uses $n+1$ bits.

Corollary 10.1 (Exact target-distance summaries)**.** *The function τ_P has a circuit of size $O(B_P)$.*

Proof. Use the existence flag and first-hit-time circuit of Corollary 7.10, and select N on negative instances with $O(n)$ additional gates. The time on positive instances is the transient depth of the absorbing map $G(x) = x$ when $P(x) = 1$ and $G(x) = F(x)$ otherwise. No clock is added to the dynamical state. □

There is also a proof using only Theorem 1.1, at the cost of a factor n. Compile G^{2^i} for $0 \le i < n$. For a nontarget input, start with $z = x$, $r = 0$, and consider these powers in decreasing order. Accept a jump when its endpoint is still outside P, updating z and adding 2^i to r. Absorption makes target membership monotone along the orbit. The accepted jumps find the largest $r < N$ still outside the target, so $r + 1$ is the first-hit time or N if no target is reached. Target inputs return zero. The time accumulator belongs to the query circuit, and does not enlarge the state width to which powering is applied.

Proposition 10.2 (Local characterization)**.** *For a function $R : \{0,1\}^n \to \{0,\dots,N\}$, one has $R = \tau_P$ if and only if, for every x,*

$$R(x) = \begin{cases} 0, & P(x) = 1, \\ \mathrm{inc}_N(R(F(x))), & P(x) = 0, \end{cases} \qquad \mathrm{inc}_N(u) = \min(N, u+1). \tag{10.1}$$

Proof. The target distance satisfies the equation. Conversely, if the first hit from x is at time $d < N$, the equation forces zero at that target and then $1,\dots,d$ backwards along the path. If the target is never reached, the orbit enters a target-free cycle. Going around a cycle of length $k > 0$ forces $u = \min(N, u+k)$, hence $u = N$. The same value propagates backwards to x. These cases determine R everywhere. □

For a proposed circuit R with q gates and $n+1$ output bits, a violation circuit checks the range $R(x) \le N$ and Equation (10.1) using $O(q + s + a + n)$ gates. Its unsatisfiability certifies the entire summary. This verification problem is coNP-complete in the length of the supplied circuit descriptions: a violating state is a witness for failure, and with F the identity and $R \equiv 0$, correctness is equivalent to P being a tautology. Thus this is a small semantic certificate with a coNP correctness check. It does not bound the length of a proof of the UNSAT obligation in a conventional proof system. Guessing a summary, checking it with a SAT oracle, and testing its value at the queried starting state gives certificates for either reachability answer, with description length $O(B_P \log(B_P + 2))$ and a verification circuit of size $O(B_P)$.

The rank is necessary for this exact characterization. The Boolean equation $E(x) = P(x) \vee E(F(x))$ also admits the spurious value one on a target-free cycle; a finite decreasing distance cannot do so. In particular,

$$I_{\mathrm{safe}}(x) = [\tau_P(x) = N]$$

is the largest inductive invariant avoiding P. Every safe initial set lies inside it. On states with $0 < \tau_P(x) < N$, the equation gives $\tau_P(F(x)) = \tau_P(x) - 1$, an exact termination rank when P denotes halting.

Once $R = \tau_P$ is certified, evaluating it gives the answer and exact time for any single starting state. For an initial-state predicate A:

- safety is the unsatisfiability of $A(x) \wedge [R(x) < N]$;
- with P a halting predicate, nontermination from some allowed state is the satisfiability of $A(x) \wedge [R(x) = N]$;
- if A is nonempty and all its states terminate, binary search with queries $\exists x :\ A(x) \wedge [R(x) \ge k]$ finds the exact worst-case first-halting time in $O(n)$ SAT calls.

An empty initial set is detected by a separate satisfiability query. The use of compact state predicates follows symbolic model checking [67]; here the representation-size guarantee is for general Boolean circuits. Existence, discovery, and validation remain different costs. Feldman, Immerman, Sagiv, and Shoham [68] give exponential query lower bounds for invariant inference even with short invariants; their Hoare-query model is a specific restriction on access to the system.

One certificate for all state pairs. Let $D_F(x, y)$ be the least t with $F^t(x) = y$, or N if no such t exists. By Theorem 7.4, a flag-to-sentinel conversion gives $\mathrm{C}(D_F) = O(B)$. For each fixed y, apply Proposition 10.2 with the target predicate $[x = y]$. Consequently D_F is the unique function into $\{0, \dots, N\}$ satisfying

$$D_F(x, y) = \begin{cases} 0, & x = y, \\ \mathrm{inc}_N(D_F(F(x), y)), & x \neq y. \end{cases} \tag{10.2}$$

For a candidate of q gates, one UNSAT check over x, y certifies all pairwise first-hit times using $O(q + s + n)$ gates. The circuit can then be reused with arbitrary initial and target predicates through $\exists x, y : A(x) \wedge [D_F(x, y) < N] \wedge P(y)$. The two query inputs do not double the dynamical state width. The existing lower bounds for arbitrary fixed powers do not establish matching lower bounds for these distance or invariant circuits.

10.2 A bounded number of choices

Let a deterministic computation have its configurations encoded in b bits, as in Corollary 7.11, and suppose that some configurations are *choice configurations* with two successors $\sigma_0(c)$ and $\sigma_1(c)$, while every other configuration c has the single successor $\sigma(c)$; halting configurations are absorbing. Suppose that a circuit of size s decides whether c is a choice configuration and computes $\sigma_0(c)$, $\sigma_1(c)$ and $\sigma(c)$, and that along every path from an initial configuration $\iota(x)$, $x \in \{0,1\}^n$, at most g choice configurations are visited. Let $\mathrm{Conf} \subseteq \{0,1\}^b$ be a set containing every configuration on such a path. In an alternating computation each choice configuration is labelled existential or universal by the same circuit. Acceptance is the least fixed point: a halting configuration accepts according to its state, an existential (universal) choice configuration accepts when some (every) successor accepts, a nonchoice configuration accepts when its successor does, and a configuration accepts only if this is forced after finitely many steps. A nondeterministic computation is the special case in which every choice is existential.

Proposition 10.3 (Bounded alternation)**.** *In this setting put* $b' = b + g + \lceil \log_2(g+2) \rceil$, $M = (g+1)|\mathrm{Conf}|$ *and* $A' = s + O(g \log(g+2)) + b'^3$. *Acceptance on* $\{0,1\}^n$ *has a circuit of size*

$$2^{g+1} \cdot O\Big(A' + \sqrt{b'^3 M A'}\Big) + O(2^g).$$

For an in-place machine with h heads, tape alphabet Γ, state set Q and $|\mathrm{Conf}| \leq |\Gamma|^n n^h |Q|$, and with h, Q and Γ fixed, this is $2^g |\Gamma|^{n/2} \operatorname{poly}(n, g)$.

Proof. For a choice word $y \in \{0,1\}^g$ let F_y act on pairs (c, i), where $0 \leq i \leq g$ counts the choices made so far:

$$F_y(c, i) = \begin{cases} (\sigma_{y_{i+1}}(c),\, i+1), & c \text{ a choice configuration and } i < g, \\ (\sigma(c),\, i), & c \text{ a nonchoice configuration}, \\ (c, i), & \text{otherwise}. \end{cases}$$

With y supplied as g further state bits that never change, these maps form one map on b' bits whose circuit has $s + O(g \log(g+2))$ gates: the bit y_{i+1} is selected by a multiplexer and the counter is incremented conditionally. The deterministic trajectory of $(\iota(x), 0)$ under F_y is the computation path that resolves its jth choice by y_j. Fix y. The set of pairs reachable from $\{(\iota(x), 0) : x \in \{0,1\}^n\}$ is forward invariant under F_y and has at most M elements. A trajectory in a set of M states that has not reached an absorbing state after M steps never does, so $F_y^M(\iota(x), 0)$ is a halting configuration if

and only if the path determined by y halts, and it is then the final configuration. By Theorem 3.4, with the bits of y hardwired and L optimized as in Equation (3.6) with N replaced by M, the bit

$$v_y(x) = [\, F_y^M(\iota(x), 0) \text{ is an accepting halting configuration} \,]$$

has a circuit of size $O(A' + \sqrt{b'^3 M A'})$.

For a prefix $u \in \{0,1\}^j$ with $j < g$, let $F^{(u)}$ be $F_{u0^{g-j}}$ modified to fix every pair (c, j) in which c is a choice configuration. The trajectory of $(\iota(x), 0)$ under $F^{(u)}$ follows the path with choices u until it reaches its $(j+1)$st choice configuration, and stops there. The same argument gives a circuit of the same size for the bit $e_u(x)$ that says whether $F^{(u)\,M}(\iota(x), 0)$ is a universal choice configuration with counter j.

Now evaluate the choice tree. Put $\mathrm{val}(y) = v_y(x)$ for $y \in \{0,1\}^g$, and for $|u| < g$ let $\mathrm{val}(u)$ be the AND of $\mathrm{val}(u0)$ and $\mathrm{val}(u1)$ if $e_u(x) = 1$ and their OR otherwise. This uses $O(2^g)$ further gates. We claim that $\mathrm{val}(\varepsilon)$ is the acceptance bit. By induction on $g - |u|$, $\mathrm{val}(u)$ is the value of the configuration at which the path with choices u makes its $(|u|+1)$st choice, if it makes one. If it makes none, every y extending u determines the same path, so all leaves below u equal its acceptance bit and so does $\mathrm{val}(u)$, whatever e_u is. A path that never halts determines the value 0, as the least fixed point requires, because only finitely many choices occur on it. The circuits v_y and e_u number $2^g + 2^g - 1$, which gives the bound. For the in-place case, $b = n\lceil \log_2 |\Gamma| \rceil + h\lceil \log_2 n \rceil + O(1)$ and a step circuit decodes each head position and updates the cells under the heads with $O(hn)$ gates, so $s = O(hn)$, b' and A' are $\mathrm{poly}(n, g)$ and $\sqrt{M} \le \sqrt{g+1}\,|\Gamma|^{n/2} n^{h/2} |Q|^{1/2}$. □

The factor 2^g is the price of enumerating the choice words; the dictionaries of different words y cannot be shared, because their invariant sets are disjoint. For a binary alphabet the bound $2^{n/2+g}\,\mathrm{poly}(n)$ is below the truth-table bound whenever $g \le (1/2 - \varepsilon)n$ for a fixed $\varepsilon > 0$ and n is large. Unbounded nondeterminism lies outside the method. When g is not bounded, the configuration graph has out-degree two and a computation is a tree rather than a trajectory. The single-trajectory arguments in Lemma 2.3 and the correction identity Equation (3.3) do not directly apply. Whether nondeterministic in-place machines admit circuits of size $|\Gamma|^{n/2}\,\mathrm{poly}(n)$ is open.

10.3 The cost of writing down the circuit

The circuits of Theorem 1.1 are nonuniform: the checkpoint payloads depend on all of F and on T. When F is given by a circuit, they can nevertheless be computed in time close to linear in N. We use a checkpoint set that is easier to compute than the path packing of Lemma 2.3.

Lemma 10.4 (Depth-residue checkpoints)**.** *Let F be a map on N states and $1 \le L \le N$. There is a set H of at most $2N/L$ states such that, for every x, the list $x, F(x), \ldots, F^{2L}(x)$ contains an element of H or a repeated state. Given the table of F, such a set is computed in $O(N)$ operations.*

Proof. Among the L residue classes of the depths of noncycle vertices modulo L, choose one of minimum size and put its vertices into H; this adds at most N_{tree}/L vertices. On every cycle of length $\ell \ge L$, number the vertices in the forward direction and put those with indices $0, L, 2L, \ldots$ below ℓ into H, at most $\lceil \ell/L \rceil \le 2\ell/L$ per cycle. Leave cycles shorter than L unmarked. Then $|H| \le 2N/L$. A trajectory starting at depth at least L meets every depth residue among its first L states, hence meets H. Otherwise it reaches its cycle in fewer than L steps, then either meets a mark within L further steps or, on a short cycle, repeats a state within L further steps. The cycles, the depths and the class sizes are found by one traversal of the functional graph and one breadth-first search from the cycles along reversed edges. □

Replacing Lemma 2.3 by this lemma in Section 3 changes only the constant in the size of the dictionary, so it gives the bound of Theorem 1.1 with the same $K = 2L$.

Proposition 10.5 (Preprocessing time)**.** *On a random-access machine with words of $\Theta(n)$ bits, given a circuit of size s for F and the binary expansion of T, a circuit for F^T of the size in Equation* (1.2) *is computed with*

$$O\Big(Ns + N + G + \sqrt{N}\,\lceil \log_2(T+2)/n \rceil\Big)$$

word operations, where $G = O(A + \sqrt{n^3 NA})$ is the size of the output.

Proof. If $T < K$ the circuit is T copies of F. Otherwise:

1. Evaluate F on all N states, $O(Ns)$ operations, and compute H by Lemma 10.4, $O(N)$.
2. The payloads are $z_h = F^j(h)$ with the common exponent $j = T - K$. Run a depth-first search on the reversed forest from each cycle vertex, keeping the current root-to-vertex path on a stack. At a vertex h of depth $d_h > j$ the payload is the stack entry j positions below the top. At a vertex of depth $d_h \le j$ attached to the cycle vertex c of a cycle of length ℓ, stored as an array, the payload is the cycle vertex at position $\mathrm{pos}(c) + \big((T \bmod \ell) - K - d_h\big) \bmod \ell$. Cycle vertices are handled by the same formula with $d_h = 0$. This takes $O(N)$ operations once $T \bmod \ell$ is known for every cycle length.
3. The distinct cycle lengths sum to at most N, so there are at most $\sqrt{2N}$ of them. The residue table Equation (3.5) needs $T \bmod p$ for $p \le K \le 2\left\lceil \sqrt{N} \right\rceil$. Each reduction of T modulo an $O(n)$-bit integer takes $O(\lceil \log_2(T+2)/n \rceil)$ word operations by schoolbook division, so all residues take $O(\sqrt{N}\,\lceil \log_2(T+2)/n \rceil)$ operations.
4. Emit the trajectory circuits, the Batcher network of Lemma 2.1 with the hardwired dictionary, the repetition and arithmetic circuits and the residue table, in time linear in their size. Batcher's network is generated recursively. □

In bit operations the bound is multiplied by $\mathrm{poly}(n)$. This refines the polynomial bound in N, s and $\log T$ recorded in the review notes accompanying the repository. In particular, if a uniform family F_n has circuits of size $2^{O(n)}$ computable in time $2^{O(n)}$, and T_n is computable in time $2^{O(n)}$, then the circuits for $F_n^{T_n}$ of size $2^{n/2}\,\mathrm{poly}(n, s)$ are computable in time $2^{O(n)}$, that is, they form an E-uniform family. The same holds for the final-configuration circuits of Corollary 7.11, where $T = 2^b$. Only the circuit for the first halting time, which uses Theorem 7.2, has not been analysed here.

Proposition 10.6 (A SETH obstruction to uniform analysis)**.** *Fix $0 < \varepsilon < 1$. Unless the Strong Exponential Time Hypothesis (SETH) fails, no deterministic uniform algorithm solves general n-bit iteration in time*

$$2^{(1-\varepsilon)n}\,\mathrm{poly}(s, n, \log(T+2)),$$

or general target reachability in time $2^{(1-\varepsilon)n}\,\mathrm{poly}(s, n, a)$. The same obstruction applies to an algorithm that constructs an exact Boolean circuit for the requested power, reachability summary, or exact target-distance summary within those respective time bounds.

Proof. Given an m-variable CNF formula φ, form the $(m+1)$-bit map

$$F_\varphi(u, b) = \big(u + 1 \bmod 2^m,\ b \vee \varphi(u)\big).$$

Its circuit has size $O(|\varphi| + m)$. Starting at $(0^m, 0)$, the assignment register visits every assignment during the next 2^m steps, and the last bit is then $\mathrm{SAT}(\varphi)$. Equivalently, the target $b = 1$ is reachable exactly when φ is satisfiable. The hypothesized algorithm, with $n = m + 1$ and $T = 2^m$, therefore decides CNF-SAT in $2^{(1-\varepsilon)m}\,\mathrm{poly}(|\varphi|, m)$ time, contrary to SETH [69]. A compiler has the same consequence: its explicit circuit output can be evaluated with only polynomial overhead in n per gate, which preserves the stated exponent on these instances. For a target-distance summary, compare its value with the never sentinel. □

For this example the entire powered map is already tiny:

$$F_\varphi^{2^m}(u, b) = \big(u,\ b \vee \mathrm{SAT}(\varphi)\big).$$

It is either the identity or the map that sets the last bit to one. Determining which summary is correct decides SAT. In particular, a general $2^{n/2}\,\mathrm{poly}(s, n)$-time compiler would give a $2^{m/2}\,\mathrm{poly}(|\varphi|, m)$-time SAT algorithm. This is an obstruction for algorithms inspecting the transition circuit, distinct from the oracle-model restrictions in Section 11. It already uses an exponent with $O(n)$ bits and is compatible with the near-full-state-space preprocessing bound of Proposition 10.5.

10.4 A universal linear-size map

Hardness of a fixed power of some linear-size map transfers to a single explicit map with a logarithmic loss in the state width.

Fix n and a constant $c \geq 1$, and let $G = (c + 1)n$. Normalize every circuit of size at most cn for a map on n bits to exactly G gates in topological order whose last n gates are the outputs, by inserting dummy gates and then appending n gates that copy the outputs. Such a circuit is described by a list D of G entries, each an operation and two operand indices in $[0, n + G)$, so $|D| = O(n \log n)$. Let $U = U_{n,c}$ act on states (D, x, w, pc) with $x \in \{0,1\}^n$, $w \in \{0,1\}^G$ and a counter $0 \leq pc \leq G$, of total width $m = O(n \log n)$:

- if $pc < G$, set w_{pc} to the operation of entry pc of D applied to its operands, where index i refers to x_i for $i < n$ and to w_{i-n} otherwise, and increment pc;
- if $pc = G$, copy the last n bits of w into x, clear w and reset pc to 0.

Other values of pc are treated as G, and D never changes.

Proposition 10.7 (Universal map)**.** *$U_{n,c}$ is computable in time* $\mathrm{poly}(n)$ *from (n, c) and has* $\mathrm{C}(U_{n,c}) = O(n \log n) = O(m)$*. Put $\tau = G + 1$. For every $F : \{0,1\}^n \to \{0,1\}^n$ with $\mathrm{C}(F) \leq cn$ and every $T \geq 0$,*

$$\mathrm{C}(F^T) \leq \mathrm{C}(U_{n,c}^{\tau T}).$$

Consequently, if some maps F_n with $\mathrm{C}(F_n) \leq cn$ have $\mathrm{C}(F_n^{T_n}) \geq 2^{\varepsilon n}$, then $\mathrm{C}(U_{n,c}^{\tau T_n}) \geq 2^{\Omega(m/\log m)}$, where the exponent τT_n has $O(\log T_n + \log n)$ bits.

Proof. One step selects entry pc of D with a multiplexer over G entries of $O(\log n)$ bits, $O(n \log n)$ gates; selects the two operands from the $n + G$ bits of x and w, $O(n)$ gates; and writes the result through a decoder of pc, $O(n)$ gates. The reset step and the selection between the two cases cost $O(n)$ gates. For the description D_F of F, gate pc reads only x and gates with smaller indices, which have been written since the last reset, so $U^\tau(D_F, x, 0, 0) = (D_F, F(x), 0, 0)$ and by induction $U^{\tau T}(D_F, x, 0, 0) = (D_F, F^T(x), 0, 0)$. Hardwiring D_F, $w = 0$ and $pc = 0$ in a circuit for $U^{\tau T}$ and projecting onto x gives a circuit for F^T of no larger size, since constants and projections are free. The final claim follows from $n = \Omega(m/\log m)$. □

Lean checks the simulation and hardwiring steps of this proof in `UniversalMap.lean`. The theorem `iterate_simulation` derives

$$U^{\tau T} \circ \mathrm{enc} = \mathrm{enc} \circ F^T$$

from the one-step identity. The theorem `simulation_circuit` turns a circuit for $U^{\tau T}$, correct on encoded states, into one for F^T when the encoding places the input bits among constant bits; in the native basis this costs two gates for the constants. `Universal.step_round` defines the step map on abstract states (D, x, w, pc) over any CSLib signature and proves $U^{G+1}(D_c, x, 0, 0) = (D_c, c(x), 0, 0)$ for every CSLib circuit c with at most G gates, where D_c lists the gates of c followed by padding. In this version the reset step reads the output wires through indices stored in D, which is equivalent to the appended copy gates above. `Universal.universal_power_circuit` combines these steps: under any bit encoding of the states that writes x directly and fixes the remaining bits of a start state to constants determined by D_c, a circuit for $U^{(G+1)T}$ gives one for c^T with at most two more gates. The gate count $\mathrm{C}(U_{n,c}) = O(n \log n)$, the bit encoding of D, w and pc, the uniformity claim and the asymptotic consequence are not mechanized.

Remark 10.8 (One explicit candidate). Let $W = U_{n,c} \times P_{2k}$ act on $m + 2k$ bits by $(u, z) \mapsto (U_{n,c}(u), P_{2k}(z))$. Every power satisfies $\mathrm{C}(W^T) \geq \max\{\mathrm{C}(U_{n,c}^T), \mathrm{C}(P_{2k}^T)\}$, since each factor is a projection of W^T with the other input fixed. So this single explicit linear-size map has a power of complexity $\Omega(2^k/k)$ by Theorem 5.4, unconditionally but at an exponent of exponential length, and at each exponent τT it is at least as hard as every map on n bits with at most cn gates is at T, whether or not that map is explicit. So if any such map has a hard power at an explicit exponent T of polynomial length, the explicit map W has one at the explicit exponent τT, with only a logarithmic loss in the state width. Lean checks the projection inequality, up to the two native constant gates, as `prodMap_power_circuit` and `prodMap_power_circuit_left`.

10.5 Asymmetric iterators

A permutation family can make all polynomially long iterations easy while its inverse stays hard only under strong conditions. For a permutation P let $\ell(x)$ be the length of the cycle of x, $c(P)$ the number of distinct cycle lengths, and $\lambda(P) = \mathrm{ord}(P)$ the least common multiple of the cycle lengths.

Definition 10.9. A family of permutations P_n of $\{0,1\}^n$ with $\mathrm{C}(P_n) = n^{O(1)}$ has an *asymmetric iterator* if $\mathrm{C}(\mathrm{Iter}_{P_n, p(n)}) = n^{O(1)}$ for every polynomial p, while $\mathrm{C}(P_n^{-1})$ is not bounded by any polynomial.

Proposition 10.10 (Necessary conditions)**.** *If P_n has an asymmetric iterator, then none of the following is bounded by a polynomial in n: the circuit complexity of $x \mapsto \ell(x)$; the number $c(P_n)$ of distinct cycle lengths; and $\log_2 \lambda(P_n)$. Every power $P_n^{T_n}$ whose complexity is not polynomially bounded, for instance $P_n^{-1} = P_n^{\lambda - 1}$, has an exponent whose bit length is not polynomially bounded.*

Proof. Since $\ell(x) - 1 < 2^n$, $P^{-1}(x) = \mathrm{Iter}_{P,n}(x, \ell(x) - 1)$, so $\mathrm{C}(P^{-1}) \leq \mathrm{C}(\ell) + \mathrm{C}(\mathrm{Iter}_{P,n}) + O(n)$. Let $\ell_1 < \cdots < \ell_c$ be the distinct cycle lengths. A point x satisfies $P^{\ell_i}(x) = x$ exactly when $\ell(x) \mid \ell_i$, and $\ell(x)$ is itself in the list, so $\ell(x) = \min\{\ell_i : \mathrm{Iter}_{P,n+1}(x, \ell_i) = x\}$. With the list hardwired this gives $\mathrm{C}(\ell) \leq c\,(\mathrm{C}(\mathrm{Iter}_{P,n+1}) + O(n)) + O(cn)$. If $\log_2 \lambda \leq p(n)$, then $P^{-1}(x) = \mathrm{Iter}_{P,p(n)}(x, \lambda - 1)$ with $\lambda - 1$ hardwired. Finally, $P^T = \mathrm{Iter}_{P,m}(\cdot, T)$ with T hardwired whenever T has m bits. Each implication contradicts the assumption on P^{-1} when the quantity in question is polynomially bounded. □

Proposition 10.11 (Quantum inversion)**.** *If* $\mathrm{Iter}_{P,2n+1}$ *has a circuit of size* q*, then* P^{-1} *has a quantum circuit of size* $\mathrm{poly}(n, q)$ *that is correct with probability at least* $2/3$ *on every input. If* P_n *and* $\mathrm{Iter}_{P_n,2n+1}$ *are computable in polynomial time, then* $P_n^{-1} \in \mathsf{FBQP}$*. Hence a uniform family with an asymmetric iterator would give* $\mathsf{FBQP} \not\subseteq \mathsf{FP/poly}$*, and also* $\mathsf{PSPACE} \not\subseteq \mathsf{P/poly}$*.*

Proof. The function $t \mapsto \mathrm{Iter}_{P,2n+1}(x, t)$ on $[0, 2^{2n+1})$ is periodic with least period $\ell(x) \leq 2^n$ and injective on each period, so Shor's period finding [42], or Kitaev's phase estimation [57], returns $\ell(x)$ with probability $\Omega(1/\log n)$ using $\mathrm{poly}(n)$ queries. Any candidate r with $1 \leq r \leq 2^n$ and $\mathrm{Iter}(x, r) = x$ is a multiple of $\ell(x)$, so $y = \mathrm{Iter}(x, r-1)$ satisfies $P(y) = x$; the circuit checks $P(y) = x$ and repeats $O(\log n)$ times. For the last claim, P_n^{-1} is computable in polynomial space by iterating P_n, so its bit language lies in PSPACE but, by assumption, not in $\mathsf{P/poly}$. □

So asymmetric iterators cannot be shown to exist unconditionally with current techniques. The natural candidates fail for a simple reason.

Remark 10.12 (Fixed group elements). Suppose $P(x) = g \cdot x$ for a group action given by a polynomial-size circuit on encodings of group elements and points, with a fixed element g. Then $P^T(x) = g^T \cdot x$, and hardwiring an encoding of g^T gives a polynomial-size circuit for every power, including P^{-1}. This covers $x \mapsto ax \bmod M$, the RSA permutation $x \mapsto x^e \bmod N$ on $(\mathbb{Z}/N)^\times$ with fixed N, where g^T is the exponent $e^T \bmod \lambda(N)$, Rabin squaring on the quadratic residues of a fixed Blum integer, and the action of a fixed ideal class: all are symmetric nonuniformly. Making the modulus part of the state, as in the time-lock puzzle of Rivest, Shamir and Wagner [58], removes the hardwired group order, but then the iterator $x \mapsto x^{2^t} \bmod N$ and the inverse are both believed hard without the factorization of N and both easy with it. Lean checks the hardwiring step: given a circuit for an encoded group action, `smul_zpow_circuit` gives a circuit for $x \mapsto g^T \cdot x$ for every integer T with at most two more gates, and `smul_iterate_circuit` and `smul_inverse_circuit` specialize it to P^T and P^{-1}. The examples are not mechanized.

The generator must therefore be part of the input, in a group in which multiplication is easy and inversion is not.

Proposition 10.13 (Groups with infeasible inversion)**.** *Let* $\mathcal{G}_n$ *be groups with injective encodings into* $\{0,1\}^n$*, such that the set of encodings is recognized, and multiplication is computed, by circuits of size* $n^{O(1)}$*, while the map* $g \mapsto g^{-1}$ *has no polynomial-size circuits. Then* $P_n(g, x) = (g, gx)$ *on pairs of encodings, and the identity on all other strings, is a permutation family with an asymmetric iterator, and* $\ell(g, x) = \mathrm{ord}(g)$*.*

Proof. Left multiplication by g is a bijection of $\mathcal{G}_n$, so P_n is a permutation of $\{0,1\}^{2n}$. For an m-bit t, square-and-multiply computes g^t with $O(m)$ multiplications, so $\mathrm{Iter}_{P_n,m}((g, x), t) = (g, g^t x)$ has size $\mathrm{poly}(n, m)$. With the encoding of the identity e hardwired, $P_n^{-1}(g, e) = (g, g^{-1})$, so a polynomial-size circuit for P_n^{-1} would invert group elements. □

Lean checks only the identity $P^t(g, x) = (g, g^t x)$ for any monoid action, as `pairAction_iterate`; the circuit bounds in this proof are prose.

Groups with infeasible inversion were introduced in the theses of Hohenberger and of Molnar and used for directed transitive signatures and broadcast encryption; Yamakawa, Yamada, Hanaoka and Kunihiro construct one from indistinguishability obfuscation and factoring [59], and Altuğ and Chen propose a candidate from isogenies over RSA moduli [60], which also surveys this history. Whether either candidate admits the injective encodings and recognizable encoding set that Proposition 10.13 needs is not settled here. Any such group is broken by a quantum computer, by order finding on g followed by $g^{\mathrm{ord}(g)-1}$, in accordance with Proposition 10.11.

Remark 10.14 (No separation from a random power oracle). Oracle separations need care with advice. A uniformly random permutation of N points has $\ln N + O(1)$ cycles in expectation, hence $O(n)$ distinct cycle lengths with high probability. Given the list of these lengths as $O(n^2)$ bits of advice and the power oracle $(t, y) \mapsto P^t(y)$, the spectrum test in the proof of Proposition 10.10 finds $\ell(x)$ with $O(n)$ queries, and one more query returns $P^{\ell(x)-1}(x) = P^{-1}(x)$. So a random permutation with its power oracle separates iterating from inverting only for algorithms without advice. By Proposition 10.10, an oracle separation against circuits needs a permutation with superpolynomially many distinct cycle lengths.

11 Black-box lower bounds for fast-forwarding

The constructions of Section 3 use the transition only through evaluations. This section asks how far that use is optimal for oracle algorithms that may store advice about the map. If an algorithm must handle every exponent, or the single exponent $\mathrm{ord}(P) - 1$, then it inverts the permutation P, and the inversion bounds of Yao and of De, Trevisan, and Tulsiani [50, 51] give $S\,t \geq \widetilde{\Omega}(\varepsilon N)$ in the notation below. Here we treat one exponent fixed in advance, for random maps and for random permutations. The results are prose results. Apart from the phase identities of Lemma 11.1, which are checked in Lean as `canonicalTime_apply` and `iterate_eq_canonical_iff`, nothing in this section is mechanized. Two lemmas about random maps are used as explicit hypotheses; they are supported by heuristics and by simulation but are not proved here.

Throughout this section log is to base 2 and ln is natural. The letter t denotes a number of oracle queries, not an input time, and S denotes a number of advice bits, not an invariant set.

11.1 Preprocessing algorithms and the upper bounds

Let V be a set of N states and let f be a random map on V, either uniform among all N^N maps or uniform among all $N!$ permutations. An (S, t)-*algorithm* is a pair (A_0, A_1). The preprocessing stage A_0 sees all of f and outputs an advice string $z \in \{0,1\}^S$. The online stage A_1 receives z and an input $x \in V$, makes at most t adaptive oracle queries to f, and outputs a state. It computes f^T with success ε if

$$\Pr_{f,x}\big[A_1^f(A_0(f), x) = f^T(x)\big] \geq \varepsilon,$$

where x is uniform and independent of f. Preprocessing time is not charged. Randomness may be fixed by averaging, so we assume that both stages are deterministic. A query to q returns $f(q)$; only forward queries are allowed.

The construction of Section 3 is such an algorithm. With $K = 2L$, the advice consists of the at most N/L checkpoint records (h, z_h) and the residue table Equation (3.5), so $S = O((N/L+L)\log N)$. The online stage walks K steps, looks up the first checkpoint, and walks at most K more steps, so $t \leq 4L$. Hence, for every map and every exponent,

$$S\,t = O(N \log N) \qquad (t \leq \sqrt{N}). \tag{11.1}$$

For maps whose images shrink, Corollary 3.5 first walks d steps and then uses checkpoints inside $\mathrm{im}(f^d)$. This gives $S = O((|\mathrm{im}(f^d)|/L + L)\log N)$ and $t \leq d + 4L$. For a uniformly random map and each fixed d, Flajolet and Odlyzko [49] show $\mathbb{E}|\mathrm{im}(f^d)|/N \to \tau_d$ as $N \to \infty$, where $\tau_0 = 1$, $\tau_{d+1} = 1 - e^{-\tau_d}$, and $\tau_d \sim 2/d$. If the profile $|\mathrm{im}(f^d)| = O(N/d)$ persists for d growing with N, which is a heuristic and not proved here, then $d = L = \lfloor t/5 \rfloor$ gives

$$S\,t^2 = O(N \log N) \qquad (t \leq N^{1/3}), \tag{11.2}$$

with balancing point $S = t = N^{1/3}$. The question is whether Equations (11.1) and (11.2) are optimal for a fixed exponent.

11.2 Phases and random-mapping lemmas

We use the notation of Section 7: $\mu(x)$ is the transient depth of x, $\ell(x)$ the length of its eventual cycle, and $c(x)$ the least state on that cycle. Let $\kappa(x)$ be the least $k \geq 0$ with $f^k(x) = c(x)$. Then $\kappa(x) \leq \mu(x) + \ell(x) - 1$, and $\kappa(x) \equiv -\phi(x) \pmod{\ell(x)}$.

Lemma 11.1 (Phase identities)**.** *Let f be any map on a finite set.*

(P1) *If x is transient, then $\kappa(f(x)) = \kappa(x) - 1$.*

(P2) *If $T \geq \mu(x)$ and $a \geq 0$, then $f^T(x) = f^a(c(x))$ if and only if $a + \kappa(x) \equiv T \pmod{\ell(x)}$.*

Proof. (P1) The states x and $f(x)$ have the same cycle. Since x is transient, $x \neq c(x)$, so $\kappa(x) \geq 1$ and $f^k(f(x)) = c(x)$ holds exactly when $f^{k+1}(x) = c(x)$.

(P2) Write $\ell = \ell(x)$ and $\kappa = \kappa(x)$, and choose r with $T + r\ell \geq \kappa$. Since $T \geq \mu(x)$, the state $f^T(x)$ is cyclic, so $f^T(x) = f^{T+r\ell}(x) = f^{T+r\ell-\kappa}(c(x))$. The state $c(x)$ has least period ℓ, so $f^i(c(x)) = f^j(c(x))$ if and only if $i \equiv j \pmod{\ell}$. □

Thus a correct value $f^T(x)$ with $T \geq \mu(x)$ determines $\kappa(x) \bmod \ell(x)$, and it carries no other information about x. By (P1), κ strictly decreases along every path to a cycle.

Now let f be a uniformly random map on V. Write $\rho(x) = \mu(x) + \ell(x)$ for the number of distinct states on the trajectory of x, and put $D_0 = 2\sqrt{N \ln N}$.

Lemma 11.2 (Rho lengths)**.** $\Pr[\max_x \rho(x) > D_0] \leq 1/N$.

Proof. Let $k = \lceil D_0 \rceil$. For a fixed x, the event $\rho(x) > k$ says that $x, f(x), \ldots, f^k(x)$ are distinct. Each new value is uniform and independent of the earlier ones, so this has probability $\prod_{i=1}^{k}(1 - i/N) \leq e^{-k(k+1)/(2N)} \leq N^{-2}$. Take a union bound over x. □

Lemma 11.3 (Cycle length and depth of one state)**.** *For every state x and every $m \geq 0$,*

$$\Pr[\ell(x) \leq m] \leq \frac{2m}{\sqrt{N}} \qquad \text{and} \qquad \Pr[\mu(x) < m] \leq \frac{2m}{\sqrt{N}}.$$

Proof. Let $K = \rho(x)$. Conditioned on $K = k$, the first k trajectory states are distinct and $f^k(x)$ is uniform among them. So $\mu(x)$ is uniform on $\{0, \ldots, k-1\}$ and $\ell(x) = k - \mu(x)$ is uniform on $\{1, \ldots, k\}$. Both probabilities are therefore at most $m\,\mathbb{E}[1/K]$. Now $\Pr[K = k] = \Pr[K \geq k]\,k/N$, so

$$\mathbb{E}[1/K] = \frac{1}{N}\sum_{k \geq 1} \Pr[K \geq k] = \frac{\mathbb{E}[K]}{N}.$$

Finally $\mathbb{E}[K] = \sum_{k \geq 1} \prod_{i<k}(1 - i/N) \leq 1 + \int_0^\infty e^{-u^2/(2N)}\,du = 1 + \sqrt{\pi N/2} \leq 2\sqrt{N}$. □

The proof of Theorem 11.5 also uses two statements that we do not prove.

Definition 11.4 (Random-mapping hypotheses)**.** Let f be a uniformly random map on N states.

(L1) There is an absolute constant C_1 such that, with probability at least $1 - 1/N$, every state w and every $0 \leq j \leq \sqrt{N}$ satisfy $|f^{-j}(w)| \leq C_1(j+1)\ln N$.

(L2) There is an absolute constant C_2 such that, with probability at least $1 - 1/N$, for every cycle and every $k \geq 0$ at most $C_2\sqrt{N}\ln N$ states with that eventual cycle satisfy $\kappa(x) = k$.

Heuristically, the in-tree above a state is close to a critical Poisson Galton–Watson tree. Its generation j survives with probability about $2/j$ and then has size about j times an exponential

variable, which suggests (L1). A class of equal κ takes one level from each tree hanging on the cycle, which suggests (L2). The script `verify_random_mapping_lemmas.py` checks (P1), (P2), and Lemma 11.2 on random maps with $N = 2^{12}, 2^{14}, 2^{16}$. In these runs, $\max_{w,\, j \le 64} |f^{-j}(w)|/((j+1)\ln N)$ is at most 0.93 and the largest class in (L2) has at most $1.5\sqrt{N}$ states. Such simulations test typical behaviour only, not the tail probability $1/N$.

11.3 Random maps

Theorem 11.5 (Random maps, assuming (L1) and (L2))**.** *Assume (L1) and (L2). There is an absolute constant C with the following property. Let f be a uniformly random map on N states, let $T \ge D_0$, $0 < \varepsilon \le 1$, and $1 \le t \le \varepsilon^2\sqrt{N}/1024$. Suppose an (S,t)-algorithm computes f^T with success ε. Put*

$$m_1 = \left\lfloor \frac{\varepsilon N}{C\, t^2 \ln N} \right\rfloor, \qquad m_2 = \left\lfloor \frac{\varepsilon N}{C\, t^3 \ln N} \right\rfloor,$$

and

$$\beta_1 = m_1\Big(\tfrac{1}{2}\log N - 2\log t - 3\log \tfrac{1}{\varepsilon} - 3\log\log N - C\Big),$$
$$\beta_2 = m_2\Big(\log N - 5\log t - \log \tfrac{1}{\varepsilon} - \log\log N - C\Big).$$

Then $S \ge \min\{\beta_1, \beta_2\} - \log(8/\varepsilon) - 3$. If, on every input x with $\mu(x) > t$, the online stage does not query $f^t(x)$, then $S \ge \beta_1 - \log(8/\varepsilon) - 3$.

The proof is an encoding argument in the style of Gennaro and Trevisan and of De, Trevisan, and Tulsiani [63, 51]: a typical map is described in fewer than $\log N^N$ bits by omitting some of its values, which the decoder recovers with the help of the advice.

Proof. Let C_1, C_2 be the constants of (L1) and (L2), and put $\ell_0 = \varepsilon^2\sqrt{N}/512$. The hypothesis on t gives $\varepsilon \ge 32N^{-1/4}$ and $t + 1 \le 2t \le \ell_0$.

Step 0: typical maps. Let $\mathcal{F}$ be the set of maps f with the following four properties.

(a) The algorithm is correct on a set G_f of at least $\varepsilon N/2$ inputs.

(b) The events of Lemma 11.2, (L1), and (L2) hold.

(c) At most $\varepsilon N/16$ states x satisfy $\ell(x) < \ell_0$.

(d) At most $\varepsilon N/16$ states x satisfy $\mu(x) \le t$.

By averaging, (a) holds with probability at least $\varepsilon/2$. Event (b) fails with probability at most $3/N \le \varepsilon/16$. By Lemma 11.3, the expected numbers of states violating (c) and (d) are at most $2\ell_0\sqrt{N} = \varepsilon^2 N/256$, so by Markov's inequality each of (c) and (d) fails with probability at most $\varepsilon/16$. Hence $|\mathcal{F}| \ge (\varepsilon/4)N^N$.

Fix $f \in \mathcal{F}$ and $z = A_0(f)$. Let $Q(x)$ be the set of states queried by $A_1(z,x)$, so $|Q(x)| \le t$. Let

$$G' = \{x \in G_f : \ \ell(x) \ge \ell_0,\ \mu(x) > t\},$$

so $|G'| \ge 3\varepsilon N/8$. For $x \in G'$ the states $x, f(x), \dots, f^t(x)$ are distinct and transient. Split G' into $G_1 = \{x : f^t(x) \notin Q(x)\}$ and $G_2 = G' \setminus G_1$. The map f has *type 1* if $|G_1| \ge 3\varepsilon N/16$, and *type 2* otherwise, in which case $|G_2| \ge 3\varepsilon N/16$. One type contains a set $\mathcal{F}_i \subseteq \mathcal{F}$ of at least $(\varepsilon/8)N^N$ maps. For each type we give an injective encoding of $\mathcal{F}_i$. Its length is then at least $N\log N - \log(8/\varepsilon)$. Under the last hypothesis of the theorem every map in $\mathcal{F}$ has type 1.

Step 1: selection for type 1. For $x \in G_1$ put $w(x) = f^t(x)$ and $\Pi(x) = \{x, f(x), \dots, f^{t-1}(x)\}$. By (L1) with $j = t$, at most $C_1(t+1)\ln N \le 2C_1 t \ln N$ inputs share a value $w(x)$. So the set $\mathcal{C}$ of these values has at least $3\varepsilon N/(32C_1 t \ln N)$ elements. For each $w \in \mathcal{C}$ fix one $x_w \in G_1$ with $w(x_w) = w$. Join $w \ne w'$ in $\mathcal{C}$ by an edge if $w' \in Q(x_w) \cup \Pi(x_w)$ or $w \in Q(x_{w'}) \cup \Pi(x_{w'})$. Each w names at most $2t$ others, so the average degree is at most $4t$. By Turán's theorem in the form $\alpha \ge n/(\bar{d}+1)$, there is an independent set of size at least $|\mathcal{C}|/(4t+1) \ge 3\varepsilon N/(160C_1 t^2 \ln N)$. Taking $C \ge 54C_1$, it contains a set W with exactly m_1 elements. Put $X = \{x_w : w \in W\}$. For $w \in W$, neither $Q(x_w)$ nor $\Pi(x_w)$ meets W: the state w is excluded by type 1 and by distinctness of the trajectory, and other members of W by independence.

Step 2: encoding for type 1. The code consists of the following parts.

(E1) The advice z, in S bits.

(E2) The set X, as one of $\binom{N}{m_1}$ subsets.

(E3) The $N - m_1$ values of f outside W, as one integer below N^{N-m_1}, in the following order. First, for $x \in X$ in increasing order, simulate $A_1(z, x)$ and list $f(q)$ for each query q whose value is not yet listed. Second, for $x \in X$ in increasing order, list the values on $\Pi(x)$ not yet listed. Third, list the remaining values outside W in increasing order of the argument.

(E4) For each $w \in W$, a lap number $a_w \le D_0/\ell_0$ and an index below $C_2\sqrt{N}\ln N$, all as one integer.

The decoder reads (E1) and (E2) and replays the simulations. Every query lies outside W, so each needed value is either known or next in (E3). The walks along $\Pi(x)$ also avoid W and end at $w = f^t(x)$. So the decoder learns W and the matching $x \leftrightarrow w$, and then the third part of (E3). It now knows f outside W.

All states of W are transient, so every cycle consists of known values; the decoder knows all cycles, their lengths, and their least states. Fix $w \in W$ and $x = x_w$, and write $\ell = \ell(x) \ge \ell_0$. The simulated output is $f^T(x)$, because $x \in G_f$. By Lemma 11.2, $T \ge D_0 \ge \mu(x)$, so $f^T(x) = f^a(c(x))$ for a known $a < \ell$, and (P2) gives $\kappa(x) \equiv T - a \pmod{\ell}$. The states $x, \dots, f^{t-1}(x)$ are transient, so (P1) gives $\kappa(w) = \kappa(x) - t$. Since $\kappa(w) \le \mu(w) + \ell - 1 < D_0 + \ell$, the lap number $a_w = \lfloor \kappa(w)/\ell \rfloor$ is at most D_0/ℓ_0, and with it the decoder knows $\kappa(w)$ exactly.

The decoder now recovers f on W in increasing order of κ, with ties broken by label. Let $u = f(w)$. By (P1), every transient state on the path from u to its cycle has $\kappa < \kappa(w)$, so every state of W on that path has already been recovered. Let U_w be the set of states v whose path to a cycle is currently known, whose cycle is the cycle of x, and which satisfy $\kappa(v) = \kappa(w) - 1$. Then $u \in U_w$ by (P1), and $|U_w| \le C_2\sqrt{N}\ln N$ by (L2). The index in (E4) identifies u. After the last step the decoder knows f, so the encoding is injective.

Step 3: counting for type 1. Using $\log\binom{N}{m} \le m\log(eN/m)$ and allowing three bits for rounding,

$$S + m_1 \log\frac{eN}{m_1} + (N - m_1)\log N + m_1 \log\Big(\big(\tfrac{D_0}{\ell_0} + 1\big)C_2\sqrt{N}\ln N\Big) + 3 \ge N\log N - \log\frac{8}{\varepsilon}.$$

Here $D_0/\ell_0 + 1 \le 1025\sqrt{\ln N}/\varepsilon^2$ and, since $m_1 \ge \varepsilon N/(2Ct^2 \ln N)$ whenever $m_1 \ge 1$, $\log(eN/m_1) \le 2\log t + \log(1/\varepsilon) + \log\ln N + \log(2eC)$. Rearranging gives $S \ge \beta_1 - \log(8/\varepsilon) - 3$, once C also satisfies $C \ge 11 + \log C_2 + \log(2eC)$. If $m_1 = 0$ the claim is trivial.

Step 4: type 2. For $x \in G_2$ the run queries $f^t(x)$ and at most $t-1$ of the states $x, \dots, f^{t-1}(x)$. Let $j(x)$ be the largest $j < t$ with $f^j(x) \notin Q(x)$, put $w(x) = f^{j(x)}(x)$, and let $\Pi(x) = \{x, \dots, f^{j(x)-1}(x)\}$. By maximality, $f(w(x)) \in Q(x)$: the run queries $f(w)$ without querying w. By (L1), at most $\sum_{j<t} C_1(j+1)\ln N \le C_1 t^2 \ln N$ inputs share a value $w(x)$. The selection of Step 1 then yields an independent set of size at least $3\varepsilon N/(80C_1 t^3 \ln N) \ge m_2$, and we fix W and X as before. The code

consists of (E1), (E2), (E3) with the new sets $\Pi(x)$, and, for each $w \in W$, the number $j(x_w) < t$ and the position, among at most t queries of $A_1(z, x_w)$, of the query $f(w)$. The decoder reads $j(x)$ before walking along $\Pi(x)$; it then knows w, and the stated position gives $f(w)$. This part does not use correctness of the outputs. The counting gives

$$S \geq m_2\Big(\log N - \log \frac{eN}{m_2} - 2\log t\Big) - \log \frac{8}{\varepsilon} - 3 \geq \beta_2 - \log \frac{8}{\varepsilon} - 3. \qquad \square$$

Corollary 11.6. *Assume (L1) and (L2), and let $T \geq D_0$. Fix $0 < \delta \leq 1/20$. There are $c > 0$ and $N_0(\delta)$ such that, for $N \geq N_0(\delta)$ and $\varepsilon \geq N^{-\delta/4}$, the following hold.*

(a) If $t \leq N^{1/5-\delta}$, then $S\,t^3 \geq c\,\delta\,\varepsilon N$.

(b) If $t \leq N^{1/4-\delta}$ and the online stage never queries $f^t(x)$ on an input x with $\mu(x) > t$, then $S\,t^2 \geq c\,\delta\,\varepsilon N$.

(c) For constant ε and $N \geq N_0(\varepsilon)$, $\max\{S, t\} \geq N^{1/5}/\log N$.

Proof. In each case $t \leq \varepsilon^2\sqrt{N}/1024$ for large N, and the relevant bracket in Theorem 11.5 is at least $\delta \log N$. For (a), the bracket of β_2 is at least $(19\delta/4)\log N - O(\log\log N)$, and that of β_1 is at least $(1/10 + 5\delta/4)\log N - O(\log\log N)$; also $m_2 \leq m_1$ and $m_2 \geq \varepsilon N/(2Ct^3 \ln N)$. For (b), the bracket of β_1 is at least $(5\delta/4)\log N - O(\log\log N)$. For (c), if $t \leq N^{1/5}/\log N$ then the bracket of β_2 is at least $4\log\log N - O_\varepsilon(1)$ and $m_2 \geq N^{2/5}$, while the bracket of β_1 is $\Omega(\log N)$; so $S \geq N^{1/5}$. $\square$

Part (b) covers the checkpoint algorithms of Equation (11.2): their online stage walks to a checkpoint and then walks from a stored payload far along the trajectory. It queries $f^i(x)$ only for $i < t$ or $i > T - t > t$. For an input of depth greater than t, the state $f^t(x)$ is transient and occurs only at time t, so it is not queried. In the range $t \leq N^{1/4-\delta}$, (b) matches Equation (11.2) up to a factor $O(\log N)$. For general algorithms, (a) loses a factor t. The loss has two sources. In type 2 the omitted state $w(x)$ may sit at any of t distances from x, so up to $\Theta(t^2 \log N)$ inputs can share it; and every selection loses a factor t to the conflict graph. Distinct inputs genuinely share omitted states: in the checkpoint algorithm all inputs below a checkpoint share it, and there are only $O(N/t^2)$ checkpoints.

Remark 11.7 (A barrier at $t = N^{1/4}$). The type-1 encoding recovers each omitted value from a correct value of f^T. By (P2), such a value carries only $\log \ell(x) \leq \frac{1}{2}\log N + O(\log\log N)$ bits about f beyond what the decoder knows, while naming m omitted states costs about $m \log(N/m)$ bits. An encoding of this type therefore proves a positive bound only when $m \geq N^{1/2-o(1)}$. For $t \geq N^{1/4+\delta}$, the advice of Equation (11.2) is $O(N^{1/2-2\delta}\log N)$, so no such encoding can be valid there, and this method cannot reach the balancing point $N^{1/3}$. This is an observation about the method, not a lower bound. Corrigan-Gibbs and Kogan [5] show that improving Yao's bound for function inversion would give new lower bounds for depth-two circuits, and that strong bounds for nonadaptive inversion would give lower bounds for linear-size logarithmic-depth circuits. We know of no analogous reduction for fast-forwarding at $S \leq \sqrt{N}$. The barrier above concerns the proof technique, and it is not evidence that Equation (11.2) is optimal or not.

The assumption $T \geq D_0$ makes every answer a cyclic state. For $t < T < \sqrt{N}$, most answers lie in trees, and we have no bound.

11.4 Random permutations and a fixed exponent

Now let P be a uniformly random permutation of V and fix $T \geq 0$, independent of P. On a cycle of length ℓ, $P^T = P^{T \bmod \ell}$, so the hardness of P^T depends on the divisors of numbers near T. For an

integer $h \geq 0$ put

$$\Lambda(T,h) = \{\ell \leq N : \ \ell \mid T-a \text{ for some integer } a \text{ with } |a| \leq h\}.$$

If $T \leq h$ then $\Lambda(T,h) = \{1,\dots,N\}$. If $h < T \leq N^{O(1)}$, each $T-a$ has $N^{o(1)}$ divisors, so $|\Lambda(T,h)| \leq (2h+1)N^{o(1)}$. If T is uniform modulo $\operatorname{lcm}(1,\dots,N)$, then $\mathbb{E}|\Lambda(T,h)| \leq (2h+1)\sum_{\ell \leq N} 1/\ell$. On the other hand $T = \operatorname{lcm}(1,\dots,N)$ gives $P^T = \mathrm{id}$ and $\Lambda = \{1,\dots,N\}$.

The following class contains the algorithm of Equation (11.1). A *path-following algorithm with one correction walk* with walk length t queries $x, P(x), \dots, P^{t-1}(x)$, then chooses a state y as a function of z and the values seen, queries $y, \dots, P^{t'-1}(y)$ for some $t' \leq t$, and outputs any function of everything seen. Write $R(x) = \{y, \dots, P^{t'-1}(y)\}$ for the correction walk. We require $P^t(x) \notin R(x)$ for every x whose cycle length is not in $\Lambda(T, 2t)$. The checkpoint algorithm satisfies this: its correction walk is $P^{[T-t+d,\,T-1]}(x)$ for a checkpoint at distance d, and it contains $P^t(x)$ only if $\ell(x) \mid T-a$ with $t < a \leq 2t$.

Theorem 11.8 (Fixed powers of a random permutation)**.** *Let $0 < \varepsilon \leq 1$ and $t \geq 1$, and put*

$$I = \lceil 16(t+1)/\varepsilon \rceil, \qquad h = 2t + I + 1, \qquad \sigma = 2(t + I + 1).$$

Assume $|\Lambda(T,h)| + 4\sigma \leq \varepsilon^2 N/64$. Suppose a path-following algorithm with one correction walk of length t and S advice bits computes P^T with success ε. Then one of the following holds.

(i) With the same advice and $2t + I + 1$ forward queries, some algorithm inverts P at a uniform point with probability at least $\varepsilon^2/128$. Consequently $S\,(2t + I + 1) \geq \widetilde{\Omega}(\varepsilon^2 N)$ [50, 51].

(ii) $S \geq m\big(\log N - 4\log t - 7\log(1/\varepsilon) - 44\big) - \log(8/\varepsilon) - 3$, where $m = \lfloor \varepsilon^2 N/(3948\,t) \rfloor$.

In particular, if $t \leq N^{1/4-\delta}$ and $\varepsilon \geq N^{-\delta/8}$, then $S\,t \geq \widetilde{\Omega}(\delta\varepsilon^3 N)$. For constant ε this is the shape of Equation (11.1).

Proof. Call a cycle *bad* if its length lies in $\Lambda(T,h)$ or is at most 4σ. A uniform permutation has, for each $\ell \leq N$, exactly one state on cycles of length ℓ in expectation. So the expected number of states on bad cycles is at most $\varepsilon^2 N/64$, and by Markov's inequality it exceeds $\varepsilon N/16$ with probability at most $\varepsilon/4$. Let $\mathcal{F}$ be the set of permutations on which the algorithm is correct on a set G of at least $\varepsilon N/2$ inputs and at most $\varepsilon N/16$ states lie on bad cycles. Then $|\mathcal{F}| \geq (\varepsilon/4)N!$. Fix $P \in \mathcal{F}$, its advice z, and let $G' \subseteq G$ be the inputs on good cycles, so $|G'| \geq 7\varepsilon N/16$. Distances below are measured along cycles, in either direction.

Step 1: inversion. Let B be the set of $y \in G'$ whose correction walk $R(y)$ contains $P^{-i}(y)$ for some $1 \leq i \leq I$. Suppose $|B| \geq \varepsilon N/16$ for at least half of $\mathcal{F}$. On challenge y, run $A_1(z,y)$ and then walk at most $I+1$ further steps from the last state of $R(y)$. If $y \in B$, the walk reaches y, and the preceding state is $P^{-1}(y)$. This inverter makes at most $2t + I + 1$ queries and succeeds with probability at least $(\varepsilon/8)(\varepsilon/16)$, which is case (i). Otherwise, let $\mathcal{F}'$ be the set of at least $(\varepsilon/8)N!$ permutations in $\mathcal{F}$ with $|B| < \varepsilon N/16$, and put $G_2 = G' \setminus B$, so $|G_2| \geq 3\varepsilon N/8$.

Step 2: a correct state below. For $x \in G_2$ let $w(x) = P^t(x)$. This map is injective, and $w(x)$ is not queried on input x: it is not on the first walk because the cycle is longer than t, and it is not on $R(x)$ by the class restriction. Let $i(x)$ be the least $i \in [1, I]$ with $P^{t+i}(x) \in G_2$. If there is none, the window $P^{[t+1,\,t+I]}(x)$ lies in a maximal run of at least I consecutive states outside G_2, and the window starts within $t+1$ steps after the start of that run, because $x \in G_2$. There are at most N/I such runs, so at most $(t+1)N/I \leq \varepsilon N/16$ inputs fail. For the others put $u'(x) = P^{t+i(x)}(x) \in G_2$ and

$$v(x) = P^T(u'(x)) = P^{t+i(x)}\big(P^T(x)\big).$$

Step 3: few wrong collisions. Let $\mathrm{wr}(v)$ be the number of inputs y with $A_1(z,y) = v \neq P^T(y)$. At most N/K states v have $\mathrm{wr}(v) > K$, and each v equals $v(x)$ for at most I inputs x. With $K = 16I/\varepsilon$, discarding the inputs with $\mathrm{wr}(v(x)) > K$ loses at most $\varepsilon N/16$ of them. The remaining values $w(x)$ form a set $\mathcal{C}$ with $|\mathcal{C}| \geq \varepsilon N/4$; for $w \in \mathcal{C}$ write x_w, i_w, u'_w, and v_w.

Step 4: selection. Choose greedily $W_1 \subseteq \mathcal{C}$ with pairwise distances greater than σ; then $|W_1| \geq |\mathcal{C}|/(2\sigma+1)$. For $w \in W_1$ the sets $\Pi(x_w) = P^{[-t,-1]}(w)$, $P^{[1,i_w-1]}(w)$, and $\Pi(u'_w) = P^{[i_w,i_w+t-1]}(w)$ lie within distance σ of w and do not contain w, so they avoid W_1. Join $w \neq w'$ if w' lies in $R(x_w)$, in $R(u'_w)$, or on the target walk $P^{[0,t+i_w-1]}(P^T(x_w))$, or symmetrically. Each of these sets is a cycle interval of fewer than σ states, so it contains at most one state of W_1; the average degree is at most 6, and Turán's theorem gives an independent set of size at least $|W_1|/7 \geq \varepsilon N/(28(2\sigma+1))$. Since $2\sigma+1 \leq 141t/\varepsilon$, this is at least $\varepsilon^2 N/(3948t)$, and we fix a subset W with exactly m elements. The three conflict sets of w do not contain w itself: $w \notin R(x_w)$ by the class restriction; $w \notin R(u'_w)$ because $u'_w \notin B$; and w on the target walk would give $\ell(w) \mid T - a$ with $-I \leq a \leq t$, so $\ell(w) \in \Lambda(T,h)$.

Step 5: encoding. The code consists of z; the set $X = \{x_w : w \in W\}$, as one of $\binom{N}{m}$ subsets; the restriction of P to $V \setminus W$, as an injection listed first in simulation order for X and then by argument, in $\log(N!/m!)$ bits; and, for each w, the number i_w and an index into

$$U_w = \{u'' :\ P^{[0,i_w-2]}(u'') \text{ is known and } A_1(z, P^{i_w-1}(u'')) \text{ can be simulated and equals } v_w\}.$$

The simulations on X avoid W and reveal $W = \{P^t(x)\}$, so the decoder can read the injection. It obtains v_w by walking $t + i_w$ steps from the simulated output $P^T(x_w)$. The true value $u = P(w)$ lies in U_w by Steps 2–4. Distinct $u'' \in U_w$ give distinct states $P^{i_w-1}(u'')$, each with output v_w; at most one of them is correct, because P^T is injective. So $|U_w| \leq 1 + \mathrm{wr}(v_w) \leq K + 1$.

Step 6: counting. Injectivity on $\mathcal{F}'$ gives

$$S \geq \log m! - m\log\frac{eN}{m} - m\log\bigl(I(K+1)\bigr) - \log\frac{8}{\varepsilon} - 3 \geq m\Bigl(2\log m - \log N - 3 - \log\bigl(I(K+1)\bigr)\Bigr) - \log\frac{8}{\varepsilon} - 3.$$

Here $I \leq 33t/\varepsilon$, so $I(K+1) \leq 17I^2/\varepsilon \leq 2^{15}t^2/\varepsilon^3$, and $m \geq \varepsilon^2 N/(7896t)$ whenever $m \geq 1$. Substituting gives case (ii). □

If the algorithm is correct on every input on a good cycle, the same proof works with $I = 1$ and $K = 0$. Inputs x with $P^{t+1}(x) \notin G_2$ are then at most $N - |G_2|$, because $x \mapsto P^{t+1}(x)$ is a bijection, and each U_w has one element. Under $|\Lambda(T, 2t+2)| + 8(t+2) \leq N/64$, this gives either $S(2t+2) \geq \widetilde{\Omega}(N)$ or

$$S \geq m\bigl(\log N - 2\log t - 20\bigr) - 5, \qquad m = \lfloor N/(182t) \rfloor .$$

So $S\,t \geq \widetilde{\Omega}(N)$ for $t \leq N^{1/2-\delta}$, the full range of Equation (11.1), and the second case gives $S\,t \geq \Omega(\delta N \log N)$. The checkpoint algorithm is of this kind.

For general adaptive algorithms, the same scheme takes the first unqueried state $P^{j(x)}(x)$ with $j(x) \leq t$ as the omitted state. All requirements become conflict edges, of out-degree $O(t+I)$, and the inversion branch walks I steps from each query. A sketch along these lines gives either $S\,t^2 \geq \widetilde{\Omega}(\varepsilon^2 N)$ by inversion, or $S \geq (\varepsilon^2 N/(Ct^2))(\log N - 8\log t - 7\log(1/\varepsilon) - C)$, hence $S\,t^2 \geq \widetilde{\Omega}(\delta\varepsilon^2 N)$ for $t \leq N^{1/8-\delta}$. We have not checked every inequality of this sketch.

The divisor condition also controls a simpler model. For integers T and N put

$$D_T = \max_{|a|<N} \bigl|\{\ell :\ |a| < \ell \leq N,\ \ell \mid T - a\}\bigr|.$$

Lemma 11.9 (Bit-fixing model)**.** *Suppose an adversary fixes P on p states, after which P is uniform among the permutations that extend these values. Let x be uniform and independent of P. If an algorithm makes t forward queries, with $p + t \le N/2$, then it outputs $P^T(x)$ with probability at most $(p + 2t + 2D_T)/N$.*

Proof. Let Q be the partial injection formed by the fixed values and the query answers, with $|Q| \le p + t$. Given Q and x, the permutation P is uniform among extensions of Q. The cycle of x can be closed within Q in two ways. Either x lies on a cycle of the fixed values, which has probability at most p/N; or some query q returns the first state of the path of Q through q. That answer is uniform over at least $N/2$ unused states, so each query closes a cycle with probability at most $2/N$.

Otherwise, contract each maximal path of Q, including single states, to a node. There are $M = N - |Q| \ge N/2$ nodes, and the extensions of Q correspond to uniform permutations of the nodes. The node cycle through the node C_0 of x can be sampled as a uniform ordering $C_1, \dots, C_{M-1}$ of the other nodes together with an independent uniform $r \in \{1, \dots, M\}$; the cycle is $C_0, C_1, \dots, C_{r-1}$. The cycle length $\ell(r) = |C_0| + \cdots + |C_{r-1}|$ is strictly increasing in r. Fix the output y and the ordering. If y lies in C_0 at offset a after x or b before x, then $P^T(x) = y$ means $\ell(r) \mid T - a$ or $\ell(r) \mid T + b$, with $\ell(r) > a$ or $\ell(r) > b$. If y lies in C_s with $s \ge 1$, it is on the cycle only when $r > s$, and then at a distance $a < \ell(r)$ from x that does not depend on r; again $\ell(r) \mid T - a$. In each case at most D_T values of r qualify, so the probability is at most $D_T/M \le 2D_T/N$. □

With $p = 0$ it bounds algorithms without advice. The heuristic of Coretti, Dodis, Guo, and Steinberger [64] replaces S advice bits and t queries by about $p = S\,t$ fixed values. Here it predicts $S\,t \ge \varepsilon N - 2D_T - 2t$, matching Equation (11.1). Their presampling theorem and its permutation version [65] do not apply directly, because they also charge the queries a challenger needs to verify an answer, and verifying $y = P^T(x)$ may require walking the whole cycle of x.

11.5 Open problems

The bounds above leave four questions.

1. For a fixed exponent with $D_T = N^{o(1)}$, for example a prime $T > N$, does every (S, t)-algorithm on a random permutation satisfy $S\,t \ge \widetilde{\Omega}(\varepsilon N)$, as for inversion? Theorem 11.8 proves this for path-following algorithms with $t \le N^{1/4-\delta}$ and constant ε, and Lemma 11.9 proves the analogue in the bit-fixing model.

2. For random maps and $T \ge D_0$, does every algorithm satisfy $S\,t^2 \ge \widetilde{\Omega}(\varepsilon N)$ for all $t \le N^{1/3}$? This would make the balancing point $N^{1/3}$ of Equation (11.2) optimal. Corollary 11.6 gives $S\,t^3$ in general, and $S\,t^2$ for algorithms that do not query $f^t(x)$, only for t below $N^{1/5}$ and $N^{1/4}$. Proving (L1) and (L2) is a smaller open task.

3. The bit-fixing heuristic with $p = S\,t$ predicts both conjectures. A presampling theorem that charges verification by the information it needs, about $\frac{1}{2}\log N$ bits for a phase, rather than by its query cost would settle them in that form. We know of none.

4. Multi-instance arguments, as used by Akshima, Guo, and Liu [66] for Merkle–Damgård collisions, may avoid Remark 11.7: bound the probability that k independent challenges all succeed, then convert with k of order S.

12 Lean formalization

The accompanying `lean/` project proves existence of actual Boolean circuits with evaluation semantics and gate bounds. It covers Table 1, the invariant-set, image-sensitive and active-mass refinements, permutation inversion, complete-configuration simulation, and the core matching-exponent lower bound. Further checked components include the exact number of distinct powers of a finite map (Lemma 5.19), the counting step of Proposition 5.20, and the semantic identities of Section 7.1 (but not their gate counts). It also covers the reachable-set bounds for in-place machines given a step circuit (Section 7.3). It also checks the in-forest duality Lemma 4.3, a combinatorial statement without circuits. The simulation and hardwiring arguments of Sections 10.4 and 10.5 are checked as well. The semantic lemmas also support arbitrary state types with finite invariant subsets and supplied Boolean encodings; initialization and decoding costs remain additional.

The development uses CSLib's circuits and simultaneous synthesis relation, Mathlib's dynamics and arithmetic, and the author's Circuit library through `complexitylib`, including its sorting and counting results [24, 25, 26]. Its native basis consists of Boolean constants, NOT, AND, and OR. Every gate is charged, including constants; wiring and fan-out are free. This differs from the paper's basis by an absolute factor and an additive constant. Section A states the native tradeoff and explains proof differences. The public theorem map, detailed constants, module guide, pinned revisions, and build commands are maintained in `lean/README.md`.

The exported theorems have guarded axiom lists containing only `propext`, `Classical.choice`, and `Quot.sound`, with no `sorry` placeholders or additional axioms. Classical choices supply checkpoints, enumerations, and circuit witnesses; the result is nonuniform existence. The sparse chart's existence and circuit cost are checked, but the edit-list compiler's algorithm and runtime, sparse-transition counting application, and finite-field corollary remain prose results. Only the combinatorial core of Section 5.7 is checked (Remark 5.26). In Section 3.7, Lean checks the exponent reduction for sequential squaring modulo a fixed squarefree modulus, with $\lambda(M)$ taken as the exponent of the unit group (`SequentialSquaring.iterate_square_reduced` and `pow_eq_pow_of_modEq`), and the exact powers of two in Table 2 (`crypto_table_entries`). The other cryptographic statements and the literature discussions are outside the mechanized boundary. For Section 9, Lean checks the scan, lookup, correction, period walk, cycle walk and output semantics under the checkpoint-hitting hypothesis. The period walk uses its own register. The hash construction, gate counts, width and reversible circuits remain prose results. Kernel checking establishes neither historical priority nor independent fidelity to every intended model convention.

13 Conclusion and open questions

Sparse checkpoints and charged lookup give square-root state-space bounds for arbitrary fixed iterates and functional-graph navigation. The counting example makes the exponent sharp for unrestricted fixed powers; it does not establish sharpness for polynomial-width input times or exact program summaries. The distance circuits have local coNP correctness checks, while Proposition 10.6 obstructs a general compiler with a fixed saving in the state-width exponent under SETH.

In the black-box model, Section 11 gives partial lower bounds for a fixed exponent: matching ones for permutations within restricted algorithm classes, and, under two random-mapping hypotheses, polynomial ones for random maps.

Two directions remain: the additive bound $s + \widetilde{O}(2^{n/2})$, and broader classes of transitions with cheaply constructible orbit coordinates. The sparse-counter compiler obtains these from an explicit edit list; extracting that list from a transition circuit is a separate task.

Provenance and draft status

This draft and the underlying argument were developed with assistance from ChatGPT (GPT-6 Astra Pro), including assistance with theorem formulation, proof development, exposition, and literature searches. The Lean development and the current revision also received assistance from Codex, including circuit synthesis proofs, axiom auditing, and review. Christian Reitwiessner contributed the manuscript extensions, Lean formalizations, and validation programs integrated from repository pull requests on October 17, 2026; those contributions disclose assistance from Claude Code. Codex assisted with their integration, the program-summary and certificate discussion, bibliography reconciliation, and combined validation. The author field is provisional. Historical priority and final attribution require author review; the draft does not represent independent peer verification.

A Correspondence with the formalization

The native circuit model charges constants as well as Boolean operations. Its foundational statement is the following.

Theorem A.1 (Mechanized tradeoff)**.** *Let c be a circuit over the Boolean constant, NOT, AND, and OR basis, with n inputs and n outputs, and let $S \subseteq \{0,1\}^n$ be finite and invariant under its evaluation F. For all natural numbers T and $1 \le L \le 2^n$, Lean proves the existence of a circuit d agreeing with F^T on S such that*

$$|d| \le 40000 \left[L|c| + (n+1)^3 \left(L + \lfloor |S|/L \rfloor + 1 \right) \right].$$

Here circuit size counts every gate, including constant gates.

The harmless +1s cover zero-width and empty-set cases. Define

$$\widehat{A} = s + (n+1)^3, \qquad \widehat{B} = \widehat{A} + \sqrt{(n+1)^3 2^n \widehat{A}}.$$

The checked real optimization gives the fixed-power bound $120000\widehat{B}$, and a separate theorem derives the $2^{\lceil n/2 \rceil}\,\mathrm{poly}(n)$ consequence for polynomial-size updates. The constants are deliberately loose accounting bounds. Free wiring and arbitrary fan-out permit sharing; constants are charged. Translating the paper's fixed fan-in-two basis to this basis changes size by at most an absolute factor and an additive constant.

The public theorem map in `lean/README.md` gives the remaining exact constants and exported names. Three implementation differences explain how the formalized constructions realize the paper's asymptotic bounds.

1. For a single fixed power, one useful collision suffices. The circuit sorts checkpoint and prefix records with tagged padding, then selects an adjacent equal-key pair witnessing either a checkpoint hit or a repeated state. Batches use the full shared dictionary lookup of Lemma 2.1. The repeated-state decoder adds the period to a hardwired residue until the index lies in the repeated interval; its $O(Kn^2)$ cost, with $K = 2L$, fits the main budget.

2. Cycle labels are computed after projecting by $F^{N!}$, which reaches the cycle and preserves its labels. A short cyclic prefix either meets a checkpoint or contains the whole cycle, allowing a scan for its minimum and period. This replaces the direct first-repetition argument in Theorem 7.1. The forest proof obtains ancestry intervals by ordering root-to-vertex paths, without implementing a traversal algorithm.

3. Sparse-counter charts use at most $r + 1$ ordinary linear intervals, with hardwired canonical cycle labels, rather than compiling the weighted arc permutation. Encoding and decoding handle phase wraparound within those intervals. Both coordinate circuits and one streaming remainder computation are charged. This proves the claimed circuit existence without verifying the edit-list compiler.

Internal arithmetic uses spare bits to represent full-length cycles and prevent overflow. The checked first-hitting-time circuit returns $n + 1$ time bits; its bound below 2^n on positive instances permits discarding the top bit. Visit descriptors use a success flag and two $(n + 1)$-bit fields, with period zero denoting a singleton. Their all-time semantics and bounded-count interpretation do not include an additional deadline-processing circuit.

B Sparse edits of a primitive binary-field cycle

Corollary B.1. *Represent $K = \mathbb{F}_{2^n}$ in a fixed polynomial basis and fix a primitive element ω. If a permutation F differs from $L(x) = \omega x$ at r states, then*

$$\mathrm{C}(F^T) \le n^{O(\log n)} + O(n(r + 1)) \quad (T \in \mathbb{Z}).$$

With an m-bit input time the bound is $n^{O(\log n)} + O(n(r + m + 1))$.

Proof. Insert zero into the nonzero multiplicative cycle by setting $G(0) = 1$, $G(\omega^{-1}) = 0$, and $G(x) = \omega x$ elsewhere. Then G is one cycle, and F differs from it at at most $r+2$ states. Coordinates are $\phi(0) = 0$ and $\phi(\omega^j) = j + 1$ for $0 \le j < 2^n - 1$. Decoding uses finite-field exponentiation and costs poly(n) gates. Encoding uses discrete logarithms.

Kleinjung and Wesolowski's Theorem 1.1 [23] gives expected time $n^{O(\log n)}$ for discrete logarithms in characteristic two. To obtain exact nonuniform circuits, let $\tau(n)$ bound this expectation for every input. Truncate each trial at $2\tau(n)$ steps and verify candidate logarithms by exponentiation. A trial succeeds with probability at least $1/2$. Among $n + 1$ independent random tapes, the probability that some one of the fewer than 2^n nonzero inputs has no successful trial is less than one. Fix tapes that work for every input and select the first verified answer. Circuit simulation and verification have polynomial overhead in $\tau(n)$ and n, preserving quasipolynomial size. Apply Corollary 8.6. The case of bounded n is absorbed by the constants. □

This corollary relies on the cited discrete-logarithm theorem and the nonuniform simulation argument; those ingredients are not formalized in our Lean development. It concerns sparse permutation-preserving edits to a primitive multiplicative cycle. It does not imply the unrestricted conjectural bound $s + \widetilde{O}(2^{n/2})$.

References

[1] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, AFIPS Conference Proceedings, volume 32, pages 307–314, 1968. doi:10.1145/1468075.1468121.

[2] Miklós Ajtai, János Komlós, and Endre Szemerédi. An $O(n \log n)$ sorting network. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1–9, 1983. doi:10.1145/800061.808726.

[3] Martin E. Hellman. A cryptanalytic time–memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980. Author-hosted full text.

[4] Amos Fiat and Moni Naor. Rigorous time/space trade-offs for inverting functions. *SIAM Journal on Computing*, 29(3):790–803, 2000. doi:10.1137/S0097539795280512. Preliminary version in STOC 1991.

[5] Henry Corrigan-Gibbs and Dmitry Kogan. The function-inversion problem: barriers and opportunities. In *Theory of Cryptography (TCC 2019)*, Part I, LNCS 11891, pages 393–421. Springer, 2019. doi:10.1007/978-3-030-36030-6_16. Full version: Cryptology ePrint Archive, Paper 2019/1046.

[6] Wolfgang J. Paul. Realizing Boolean functions on disjoint sets of variables. *Theoretical Computer Science*, 2(3):383–396, 1976. doi:10.1016/0304-3975(76)90089-X.

[7] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. In *Advances in Cryptology – CRYPTO 2018*, Part I, LNCS 10991, pages 757–788. Springer, 2018. doi:10.1007/978-3-319-96884-1_25. Definition numbers refer to Cryptology ePrint Archive, Paper 2018/601.

[8] Arjen K. Lenstra and Benjamin Wesolowski. A random zoo: sloth, unicorn, and trx. Cryptology ePrint Archive, Paper 2015/366, 2015. `https://eprint.iacr.org/2015/366`.

[9] Dmitry Khovratovich, Mary Maller, and Pratyush Ranjan Tiwari. MinRoot: candidate sequential function for Ethereum VDF. Cryptology ePrint Archive, Paper 2022/1626, 2022. `https://eprint.iacr.org/2022/1626`.

[10] Gaëtan Leurent, Bart Mennink, Krzysztof Pietrzak, Vincent Rijmen, et al. Analysis of MinRoot: public report (requested by the Ethereum Foundation). Report, September 18, 2023. `https://crypto.ethereum.org/events/minrootanalysis2023.pdf`.

[11] Mohammad Mahmoody, Tal Moran, and Salil Vadhan. Time-lock puzzles in the random oracle model. In *Advances in Cryptology – CRYPTO 2011*, LNCS 6841, pages 39–50. Springer, 2011. doi:10.1007/978-3-642-22792-9_3.

[12] Alex Biryukov and Adi Shamir. Cryptanalytic time/memory/data tradeoffs for stream ciphers. In *Advances in Cryptology – ASIACRYPT 2000*, LNCS 1976, pages 1–13. Springer, 2000. doi:10.1007/3-540-44448-3_1.

[13] Christophe De Cannière. Trivium: a stream cipher construction inspired by block cipher design principles. In *Information Security (ISC 2006)*, LNCS 4176, pages 171–186. Springer, 2006. doi:10.1007/11836810_13.

[14] Martin Hell, Thomas Johansson, and Willi Meier. Grain: a stream cipher for constrained environments. *International Journal of Wireless and Mobile Computing*, 2(1):86–93, 2007. doi:10.1504/IJWMC.2007.013798.

[15] J. Ian Munro, Rajeev Raman, Venkatesh Raman, and S. Srinivasa Rao. Succinct representations of permutations and functions. *Theoretical Computer Science*, 438:74–88, 2012. Preprint: arXiv:1108.1983, 2011.

[16] Shuichi Hirahara, Rahul Ilango, and R. Ryan Williams. Beating brute force for compression problems. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 659–670, 2024. doi:10.1145/3618260.3649778. Full version: ECCC TR23-171; section and theorem numbers refer to that version.

[17] David Eppstein. The complexity of iterated reversible computation. *TheoretiCS*, 2, Article 10, 1–41, 2023. doi:10.46298/theoretics.23.10.

[18] Klaus-Jörn Lange, Pierre McKenzie, and Alain Tapp. Reversible space equals deterministic space. *J. Comput. System Sci.*, 60(2):354–367, 2000. doi:10.1006/jcss.1999.1672.

[19] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. doi:10.1147/rd.176.0525.

[20] Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008. Author's draft, Chapter 7: one-way function definitions.

[21] Travis Gagie, Meng He, and Gonzalo Navarro. Path queries on functions. *Theoretical Computer Science*, 770:34–50, 2019. doi:10.1016/j.tcs.2018.10.021. Author-hosted full text.

[22] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5, article 1949, 2016. Theorem numbers refer to arXiv:1306.5708v3 (2014), `https://arxiv.org/abs/1306.5708v3`.

[23] Thorsten Kleinjung and Benjamin Wesolowski. Discrete logarithms in quasi-polynomial time in finite fields of fixed characteristic. *Journal of the American Mathematical Society*, 35(2):581–624, 2022. Full text: Cryptology ePrint Archive, Paper 2019/751, `https://eprint.iacr.org/2019/751`.

[24] The CSLib contributors. CSLib: a Lean library for computer science. `https://github.com/leanprover/cslib`. Pinned revisions are in the accompanying Lean dependency manifest.

[25] The mathlib community. The Lean mathematical library. In *Proceedings of CPP*, pages 367–381, 2020. doi:10.1145/3372885.3373824. Lean 4 implementation: `https://github.com/leanprover-community/mathlib4`.

[26] Samuel Schlesinger and contributors. Complexitylib, including the Circuit library and verified sorting networks. `https://github.com/SamuelSchlesinger/complexitylib`.

[27] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi:10.1002/j.1538-7305.1949.tb03624.x.

[28] Oleg B. Lupanov. A method of circuit synthesis. *Izvestiya VUZ, Radiofizika*, 1:120–140, 1958. In Russian.

[29] Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*. Algorithms and Combinatorics 27, Springer, 2012. doi:10.1007/978-3-642-24508-4.

[30] Mihir Bellare and John Rompel. Randomness-efficient oblivious sampling. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 276–287, 1994. doi:10.1109/SFCS.1994.365687.

[31] Charles H. Bennett. Time/space trade-offs for reversible computation. *SIAM Journal on Computing*, 18(4):766–776, 1989. doi:10.1137/0218053.

[32] Robert Y. Levine and Alan T. Sherman. A note on Bennett's time-space tradeoff for reversible computation. *SIAM Journal on Computing*, 19(4):673–677, 1990. doi:10.1137/0219046.

[33] Emanuel Knill. An analysis of Bennett's pebble game. Preprint, 1995. arXiv:math/9508218.

[34] Ming Li, John Tromp, and Paul Vitányi. Reversible simulation of irreversible computation. *Physica D*, 120(1–2):168–176, 1998. doi:10.1016/S0167-2789(98)00052-9. Preprint: arXiv:quant-ph/9703009.

[35] Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12(1):1–28, 1999. doi:10.1007/PL00003816.

[36] Russell Impagliazzo and Avi Wigderson. $\mathsf{P} = \mathsf{BPP}$ if E requires exponential circuits: derandomizing the XOR lemma. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC)*, pages 220–229, 1997. doi:10.1145/258533.258590.

[37] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004. doi:10.4007/annals.2004.160.781.

[38] Nicholas Pippenger and Michael J. Fischer. Relations among complexity measures. *Journal of the ACM*, 26(2):361–381, 1979. doi:10.1145/322123.322138.

[39] Leonard Adleman. Two theorems on random polynomial time. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 75–83, 1978. doi:10.1109/SFCS.1978.37.

[40] Hugh L. Montgomery and Robert C. Vaughan. *Multiplicative Number Theory I: Classical Theory*. Cambridge Studies in Advanced Mathematics 97, Cambridge University Press, 2007.

[41] Gary L. Miller. Riemann's hypothesis and tests for primality. *Journal of Computer and System Sciences*, 13(3):300–317, 1976. doi:10.1016/S0022-0000(76)80043-8.

[42] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. doi:10.1137/S0097539795293172.

[43] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Texts in Computer Science. Springer, fourth edition, 2019. doi:10.1007/978-3-030-11298-1.

[44] Richard E. Ladner and Michael J. Fischer. Parallel prefix computation. *Journal of the ACM*, 27(4):831–838, 1980. doi:10.1145/322217.322232.

[45] Stephen Wolfram. Random sequence generation by cellular automata. *Advances in Applied Mathematics*, 7(2):123–169, 1986. doi:10.1016/0196-8858(86)90028-X.

[46] Edmund Landau. Über die Maximalordnung der Permutationen gegebenen Grades. *Archiv der Mathematik und Physik*, 3rd series, 5:92–103, 1903.

[47] Paul Erdős and Pál Turán. On some problems of a statistical group-theory. III. *Acta Mathematica Academiae Scientiarum Hungaricae*, 18:309–320, 1967. doi:10.1007/BF02280290.

[48] Bernard Harris. The asymptotic distribution of the order of elements in symmetric semigroups. *Journal of Combinatorial Theory, Series A*, 15:66–74, 1973. doi:10.1016/0097-3165(73)90036-8.

[49] Philippe Flajolet and Andrew M. Odlyzko. Random mapping statistics. In *Advances in Cryptology – EUROCRYPT '89*, LNCS 434, pages 329–354. Springer, 1990. doi:10.1007/3-540-46885-4_34.

[50] Andrew C. Yao. Coherent functions and program checkers. In *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, STOC, pages 84–94, 1990. doi:10.1145/100216.100226.

[51] Anindya De, Luca Trevisan, and Madhur Tulsiani. Time space tradeoffs for attacks against one-way functions and PRGs. In *Advances in Cryptology – CRYPTO 2010*, LNCS 6223, pages 649–665. Springer, 2010. doi:10.1007/978-3-642-14623-7_35.

[52] Noam Mazor and Rafael Pass. The non-uniform perebor conjecture for time-bounded Kolmogorov complexity is false. In *15th Innovations in Theoretical Computer Science Conference (ITCS)*, LIPIcs 287, 80:1–80:20, 2024. ECCC TR23-175.

[53] Larry J. Stockmeyer. The polynomial-time hierarchy. *Theoretical Computer Science*, 3(1):1–22, 1976. doi:10.1016/0304-3975(76)90061-X.

[54] Boštjan Brešar, František Kardoš, Ján Katrenič, and Gabriel Semanišin. Minimum k-path vertex cover. *Discrete Applied Mathematics*, 159(12):1189–1195, 2011. doi:10.1016/j.dam.2011.04.008.

[55] Lijie Chen, Zhenjian Lu, Igor C. Oliveira, Hanlin Ren, and Rahul Santhanam. Polynomial-time pseudodeterministic construction of primes. In *64th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1261–1270, 2023. doi:10.1109/FOCS57990.2023.00074.

[56] Patrick X. Gallagher. On the distribution of primes in short intervals. *Mathematika*, 23(1):4–9, 1976. doi:10.1112/S0025579300016442.

[57] A. Yu. Kitaev. Quantum measurements and the Abelian stabilizer problem. Preprint, 1995. arXiv:quant-ph/9511026.

[58] Ronald L. Rivest, Adi Shamir, and David A. Wagner. Time-lock puzzles and timed-release crypto. Technical Report MIT/LCS/TR-684, Massachusetts Institute of Technology, 1996.

[59] Takashi Yamakawa, Shota Yamada, Goichiro Hanaoka, and Noboru Kunihiro. Self-bilinear map on unknown order groups from indistinguishability obfuscation and its applications. In *Advances in Cryptology – CRYPTO 2014*, LNCS 8617, pages 90–107. Springer, 2014. doi:10.1007/978-3-662-44381-1_6.

[60] Salim Ali Altuğ and Yilei Chen. Hard isogeny problems over RSA moduli and groups with infeasible inversion. In *Advances in Cryptology – ASIACRYPT 2019*, LNCS 11922, pages 293–322. Springer, 2019. doi:10.1007/978-3-030-34621-8_11.

[61] Dietmar Uhlig. On the synthesis of self-correcting schemes from functional elements with a small number of reliable elements. *Mathematical Notes of the Academy of Sciences of the USSR*, 15(6):558–562, 1974. doi:10.1007/BF01152835.

[62] Dietmar Uhlig. Networks computing Boolean functions for multiple input values. In M. S. Paterson, editor, *Boolean Function Complexity*, London Mathematical Society Lecture Note Series, pages 165–173. Cambridge University Press, 1992. doi:10.1017/CBO9780511526633.013.

[63] Rosario Gennaro and Luca Trevisan. Lower bounds on the efficiency of generic cryptographic constructions. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 305–313, 2000. doi:10.1109/SFCS.2000.892119.

[64] Sandro Coretti, Yevgeniy Dodis, Siyao Guo, and John Steinberger. Random oracles and non-uniformity. In *Advances in Cryptology – EUROCRYPT 2018*, LNCS 10820, pages 227–258. Springer, 2018. doi:10.1007/978-3-319-78381-9_9.

[65] Sandro Coretti, Yevgeniy Dodis, and Siyao Guo. Non-uniform bounds in the random-permutation, ideal-cipher, and generic-group models. In *Advances in Cryptology – CRYPTO 2018*, LNCS 10991, pages 693–721. Springer, 2018. doi:10.1007/978-3-319-96884-1_23.

[66] Akshima, Siyao Guo, and Qipeng Liu. Time-space lower bounds for finding collisions in Merkle–Damgård hash functions. In *Advances in Cryptology – CRYPTO 2022*, pages 192–221. Springer, 2022. doi:10.1007/978-3-031-15982-4_7.

[67] Jerry R. Burch, Edmund M. Clarke, Kenneth L. McMillan, David L. Dill, and L. James Hwang. Symbolic model checking: 10^{20} states and beyond. In *Proceedings of the Fifth Annual IEEE Symposium on Logic in Computer Science*, pages 428–439, 1990. LICS archive.

[68] Yotam M. Y. Feldman, Neil Immerman, Mooly Sagiv, and Sharon Shoham. Complexity and information in invariant inference. Preprint, revised 2020. arXiv:1910.12256v3.

[69] Marek Cygan, Holger Dell, Daniel Lokshtanov, Dániel Marx, Jesper Nederlof, Yoshio Okamoto, Ramamohan Paturi, Saket Saurabh, and Magnus Wahlström. On problems as hard as CNF-SAT. *ACM Transactions on Algorithms*, 12(3), Article 41, 1–24, 2016. doi:10.1145/2925416.